Imprimer ce cours

Scripting Bash et PowerShell

Formation de scripting système — 4 jours, 16 chapitres

Durée : 24 heures (6h/jour)

Public : Administrateurs système, étudiants en informatique

Prérequis : Connaissances de base en utilisation d'un ordinateur

Table des matières

- **Jour 1** — Bash (1/2) : Introduction, syntaxe, création de scripts, structures de contrôle
- **Jour 2** — Bash (2/2) : Manipulation de fichiers, gestion des erreurs, administration, scripts interactifs
- **Jour 3** — PowerShell (1/2) : Environnement, cmdlets, exécution, structures de contrôle, initiation Python
- **Jour 4** — PowerShell (2/2) : Fichiers, objets, processus, droits, cas pratique
- **Module Extra** — Python avancé, cryptographie, sauvegarde, MinIO S3

Scripting Bash et PowerShell — JOUR 1

Bash (1/2) — Introduction au shell et structures de contrôle (08/11/2026)

FIL CONDUCTEUR

Ce cours de 4 jours progresse du shell Bash sous Linux (J1-J2) vers PowerShell sous Windows (J3-J4), en passant par l'automatisation, les droits et la documentation professionnelle de scripts.

Objectifs du Jour

- Comprendre l'environnement du shell Bash et son rôle d'intermédiaire entre l'utilisateur et le noyau Linux
- Maîtriser la syntaxe de base : variables, types, commandes simples et redirections
- Créer, rendre exécutable et exécuter un premier script Bash

- Utiliser les structures de contrôle : conditions (if, case) et boucles (for, while)

Planning du Jour

Chapitre	Horaires	Thème	Durée
1	9h00-11h00	Introduction au shell Bash et à son environnement	2h
—	11h00-11h15	Pause	15min
2	11h15-12h00	Syntaxe de base : variables, types, commandes simples	45min
—	12h00-13h00	Pause déjeuner	1h
3	13h00-14h30	Création et exécution d'un script	1h30
—	14h30-14h45	Pause	15min
4	14h45-16h00	Structures de contrôle : conditions (if, case), boucles (for, while)	1h15

Chapitre 1 — Introduction au shell Bash et à son environnement (9h00-11h00)

Objectifs

- Définir ce qu'est un shell et le rôle de Bash comme intermédiaire entre l'utilisateur et le noyau
- Identifier les fichiers de configuration et d'environnement du shell
- Distinguer variables shell et variables d'environnement, et utiliser `export`
- Connaître les variables d'environnement courantes : HOME, PATH, SHELL, USER, LANG

Contenu théorique (1h45)

1. Qu'est-ce que le shell Bash ? (30min)

Le **shell** est l'interpréteur de commandes qui fait l'interface entre l'utilisateur et le noyau du système d'exploitation. Il lit les commandes saisies par l'utilisateur, les interprète, puis les exécute en lançant les programmes correspondants. **Bash** (Bourne Again SHell) est le shell par défaut sur la plupart des distributions Linux. Il constitue une surcouche qui traduit les commandes texte en appels système compris par le noyau Linux.

Le noyau Linux gère directement le matériel, la mémoire, les processus et les systèmes de fichiers, mais il n'interagit pas directement avec l'utilisateur. C'est le shell qui assure ce rôle d'intermédiaire : il attend une saisie, analyse la commande, résout les chemins, lance les programmes et affiche les résultats.

```
echo $SHELL
# echo : affiche la valeur d'une variable ou d'une chaîne de caractères
# $SHELL : variable d'environnement contenant le chemin de l'interpréteur par défaut
ps -p $$
# ps : affiche des informations sur les processus en cours
# -p : sélectionne un processus par son identifiant (PID)
# $$ : variable spéciale contenant le PID du shell courant
```

CONCEPT CLÉ

Le shell agit comme un intermédiaire : il traduit les commandes texte en appels système compris par le noyau Linux. Sans shell, l'utilisateur devrait interagir directement avec le noyau via des appels système de bas niveau.

BON À SAVOIR

Bash est un shell compatible avec le shell Bourne historique (`sh`), tout en y ajoutant des fonctionnalités avancées : complétion, historique, alias, fonctions, tableaux et arithmétique intégrée.

2. Fichiers de configuration et d'environnement (45min)

Lorsqu'une session shell s'ouvre, plusieurs fichiers de configuration sont lus dans un ordre précis. Ils permettent de personnaliser l'environnement de l'utilisateur : variables d'environnement, alias, fonctions, invite de commandes, etc.

Fichier	Rôle	Moment de chargement
/etc/profile	Configuration système globale, valide pour tous les utilisateurs	Ouverture de session (login shell)
~/.bash_profile	Configuration utilisateur spécifique aux sessions de connexion	Ouverture de session (login shell)
~/.bashrc	Configuration utilisateur pour les shells interactifs non-login	Chaque nouveau shell interactif
~/.bash_history	Historique des commandes exécutées par l'utilisateur	Continu, écrit à la fin de la session

Le fichier `/etc/profile` est lu une seule fois à l'ouverture de session. Le fichier `~/.bash_profile` est lu ensuite pour l'utilisateur connecté. Si `~/.bash_profile` n'existe pas, Bash peut lire `~/.bash_login` ou `~/.profile`. Le fichier `~/.bashrc` est lu à chaque lancement d'un shell interactif (par exemple, à l'ouverture d'un nouveau terminal).

```
cat /etc/profile | head -20
# cat : affiche le contenu d'un fichier sur la sortie standard
# /etc/profile : fichier de configuration système globale
# head -20 : limite l'affichage aux 20 premières lignes
# | : tube (pipeline), redirige la sortie de cat vers l'entrée de head
cat ~/.bashrc
# cat : affiche le contenu du fichier de configuration utilisateur
# ~/.bashrc : fichier de configuration du shell interactif de l'utilisateur courant
```

POURQUOI

La distinction entre fichiers de connexion (login) et fichiers interactifs permet de séparer ce qui doit être configuré une seule fois par session (variables d'environnement, PATH) de ce qui doit être disponible à chaque nouveau terminal (alias, fonctions, invite de commandes).

PIÈGE

Sur certaines distributions, `~/.bash_profile` source `~/.bashrc` s'il existe. Sur d'autres, ce n'est pas le cas. Il est recommandé de vérifier le comportement sur la cible de déploiement avant d'écrire un script portable.

3. Variables shell et variables d'environnement (45min)

Bash distingue deux catégories de variables :

- Les **variables shell** (shell variables) : définies dans le shell courant, visibles uniquement dans ce shell. Elles ne sont pas transmises aux programmes lancés depuis ce shell.
- Les **variables d'environnement** (environment variables) : des variables shell qui ont été rendues accessibles aux processus enfants (child processes) grâce à la commande `export`. Tout programme ou script lancé depuis le shell hérite d'une copie de ces variables.

Cette distinction est documentée dans The Linux Command Line - A Complete Introduction, 3e (Shotts, William), page 144, qui précise que les variables shell sont des données placées par l'instance courante de bash, tandis que les variables d'environnement englobent tout le reste.

Pour rendre une variable accessible aux sous-processus, on utilise la commande `export` (Linux Command Line and Shell Scripting Bible, 5e, Blum & Bresnahan, pp. 154, 155, 169). Un shell enfant (subshell) hérite des variables exportées de son shell parent. Lorsqu'un subshell se termine, sa copie de l'environnement est perdue : les modifications apportées dans le subshell n'affectent pas le shell parent.

Les variables d'environnement courantes sont :

Variable	Signification	Exemple de valeur
HOME	Répertoire personnel de l'utilisateur	/home/eleve
PATH	Liste des répertoires où le shell recherche les commandes	/usr/local/sbin:/usr/bin:/bin
SHELL	Interpréteur de commandes par défaut	/bin/bash
USER	Nom de l'utilisateur connecté	eleve
LANG	Langue et paramètres régionaux (locale)	fr_FR.UTF-8

```
echo $HOME
# echo : affiche la valeur d'une variable
# $HOME : répertoire personnel de l'utilisateur courant
echo $PATH
# $PATH : liste des répertoires de recherche des commandes, séparés par deux-points
echo $SHELL
# $SHELL : chemin de l'interpréteur de commandes par défaut
echo $USER
# $USER : nom de l'utilisateur connecté
echo $LANG
# $LANG : locale du système (langue, encodage, formats)
MON_PROJET=scripting-b2
# MON_PROJET=scripting-b2 : assigne la chaîne "scripting-b2" à la variable shell MON_PROJET
# Aucune espace n'est autorisée autour du signe =
export MON_PROJET
# export : rend la variable MON_PROJET accessible aux processus enfants (child processes)
echo $MON_PROJET
# echo : affiche la valeur de la variable dans le shell courant
```

CONCEPT CLÉ

Sans `export`, une variable reste locale au shell courant et n'est pas visible par les scripts ou programmes qu'il lance. Avec `export`, elle devient une variable d'environnement héritée par tous les sous-processus.

CONTRE-MESURE

Pour éviter de polluer l'environnement global, n'exportez que les variables strictement nécessaires aux sous-processus. Utilisez des noms en majuscules pour les variables d'environnement afin de les distinguer visuellement des variables locales.

Fig 1 — Environnement d'exécution d'un script Bash

Diagramme		
B	interprétation	
C	appel système	
D	résultat	
B	affichage	Utilisateur

Figure — Diagramme Mermaid

4. Commandes d'information système et de surveillance (30min)

Un administrateur système doit disposer d'une vision en temps réel de l'état de la machine : utilisation du processeur, de la mémoire, de l'espace disque, liste des processus en cours, utilisateurs connectés. Les commandes présentées ci-dessous permettent d'obtenir ces informations sans installer d'outils supplémentaires, car elles font partie du ensemble de base des systèmes Linux (coreutils, procps-ng, util-linux).

4.1. Informations sur les utilisateurs et les sessions (who, w, last, uptime)

La commande `who` liste les utilisateurs actuellement connectés au système, avec leur terminal, leur heure de connexion et l'adresse IP distante le cas échéant. La commande `w` fournit des informations similaires, complétées par l'activité en cours de chaque utilisateur et la charge moyenne du système. La commande `last` consulte le fichier des connexions (`/var/log/wtmp`) pour afficher l'historique des ouvertures et fermetures de session. Enfin, `uptime` indique depuis combien de temps le système fonctionne sans redémarrage, ainsi que la charge moyenne sur 1, 5 et 15 minutes.

```
who
# who : affiche la liste des utilisateurs connectés (nom, terminal, date/heure, hôte distant)
# Aucun flag requis : affichage par défaut des sessions actives

w
# w : affiche les utilisateurs connectés, leur activité et la charge système
# Aucun flag requis : combine qui, uptime et activité des processus

uptime
# uptime : affiche l'heure actuelle, la durée de fonctionnement et la charge moyenne
# Aucun flag requis : résultat synthétique sur une seule ligne

last | head -10
# last : lit le fichier /var/log/wtmp pour l'historique des connexions
# | : tube qui redirige la sortie de last vers l'entrée de head
# head -10 : limite l'affichage aux 10 entrées les plus récentes
```

CONCEPT CLÉ

La charge moyenne (load average) affichée par `uptime` et `w` représente le nombre moyen de processus en attente d'exécution sur les 1, 5 et 15 dernières minutes. Une charge supérieure au nombre de cœurs du processeur indique une saturation du CPU.

BON À SAVOIR

La commande `last` peut être vide si le fichier `/var/log/wtmp` n'existe pas ou a été purgé. Elle nécessite généralement des privilèges de lecture sur ce fichier journal.

4.2. Utilisation de la mémoire et du disque (free, df, du)

La commande `free` affiche la quantité de mémoire vive (RAM) disponible, utilisée et libre, ainsi que l'espace d'échange (swap). La commande `df` (disk free) indique l'espace disque disponible sur chaque système de fichiers monté. La commande `du` (disk usage) mesure l'espace occupé par un répertoire ou un fichier spécifique.

```
free -h
# free : affiche l'utilisation de la mémoire et du swap
# -h : format lisible par un humain (Ko, Mo, Go au lieu d'octets bruts)

df -h
# df : affiche l'espace disque disponible sur les systèmes de fichiers montés
# -h : format lisible par un humain (Ko, Mo, Go)

du -sh /home/eleve
# du : estime l'espace disque utilisé par un fichier ou répertoire
# -s : résumé, affiche uniquement le total pour le répertoire spécifié
# -h : format lisible par un humain
# /home/eleve : répertoire dont on mesure l'occupation
```

CONCEPT CLÉ

`df` montre l'espace disponible par système de fichiers (point de montage), tandis que `du` montre l'espace consommé par un répertoire ou fichier particulier. Ces deux commandes se complètent : `df` pour savoir s'il reste de la place, `du` pour savoir qui l'occupe.

PIÈGE

Sur certains systèmes de fichiers (notamment Btrfs ou ZFS), `df` peut afficher des valeurs inattendues en raison du partage de blocs ou des instantanés (snapshots). Privilégier `du` pour une mesure précise de la consommation d'un répertoire.

4.3. Gestion des processus (ps, top, kill)

Un processus est une instance d'un programme en cours d'exécution, identifiée par un PID (Process ID). La commande `ps` affiche un instantané des processus actifs. La commande `top` affiche une vue dynamique et actualisée des processus les plus consommatifs. La commande `kill` envoie un signal à un processus, par exemple pour le terminer.

```
ps aux
# ps : affiche un instantané des processus en cours
# a : affiche les processus de tous les utilisateurs
# u : affiche l'utilisateur propriétaire et le pourcentage de CPU/mémoire
# x : inclut les processus sans terminal associé (démons, services)

top
# top : affiche une vue dynamique triée par consommation CPU
# Aucune option requise : actualisation automatique toutes les 3 secondes
# Touche q pour quitter, touche k pour tuer un processus par son PID

kill 1234
# kill : envoie un signal à un processus identifié par son PID
# 1234 : identifiant du processus cible
# Par défaut, envoie le signal SIGTERM (15) qui demande une terminaison propre

kill -9 1234
# kill -9 : envoie le signal SIGKILL (9), terminaison immédiate et forcée
# 9 : numéro du signal SIGKILL, ne peut pas être intercepté par le processus
# À utiliser en dernier recours, car le processus ne peut pas nettoyer ses ressources
```

CONCEPT CLÉ

Un signal est une notification asynchrone envoyée à un processus. SIGTERM (15) demande une terminaison gracieuse ; le processus peut intercepter ce signal pour fermer ses fichiers et libérer ses ressources. SIGKILL (9) force l'arrêt immédiat par le noyau, sans possibilité pour le processus de réagir.

CONTRE-MESURE

Toujours tenter `kill PID` (SIGTERM) avant d'utiliser `kill -9 PID` (SIGKILL). Le signal SIGKILL ne permet pas au processus de sauvegarder son état ni de fermer proprement ses fichiers ouverts, ce qui peut entraîner une perte de données.

5. Commandes réseau et diagnostic (30min)

Les commandes réseau permettent de vérifier la connectivité, d'inspecter les interfaces, de consulter les tables de routage et de résoudre des noms de domaine. Certaines commandes classiques (`ifconfig` , `netstat` , `route`) sont progressivement remplacées par des outils plus modernes (`ip` , `ss`), mais restent présentes sur de nombreux systèmes.

5.1. Test de connectivité (ping, traceroute)

La commande `ping` envoie des paquets ICMP ECHO_REQUEST à un hôte distant et attend une réponse ECHO_REPLY, permettant de vérifier qu'un hôte est joignable sur le réseau. La commande `traceroute` affiche le chemin emprunté par les paquets pour atteindre une destination, routeur par routeur.

```
ping -c 4 192.168.1.1
# ping : envoie des requêtes ICMP pour tester la connectivité réseau
# -c 4 : limite à 4 paquets envoyés, puis s'arrête automatiquement
# 192.168.1.1 : adresse IP de l'hôte cible

traceroute 192.168.1.1
# traceroute : affiche chaque saut (routeur) sur le chemin vers la destination
# Aucun flag requis : utilise UDP par défaut avec des TTL croissants
```

5.2. Interfaces et configuration réseau (ifconfig, ip, route)

La commande `ifconfig` (interface configurer) affiche et configure les interfaces réseau. Elle est considérée comme obsolète sur les distributions modernes au profit de la commande `ip` du paquet `iproute2` . La commande `route` affiche et manipule la table de routage IP.

```
ifconfig
# ifconfig : affiche la configuration des interfaces réseau (adresse IP, masque, MTU)
# Aucun flag requis : liste toutes les interfaces actives

ip addr
# ip : outil moderne de gestion réseau et routage (paquet iproute2)
# addr : sous-commande affichant les adresses IP des interfaces
# Remplace ifconfig sur les systèmes récents

ip route
# ip route : affiche la table de routage IP du système
# route : sous-commande dédiée à l'affichage et à la modification des routes
```

CONCEPT CLÉ

La commande `ip` du paquet `iproute2` unifie la gestion des adresses, des routes et des liens réseau dans un seul outil, remplaçant les commandes historiques `ifconfig`, `route` et `arp`. Sur les systèmes anciens, `ifconfig` et `route` peuvent être les seuls outils disponibles.

BON À SAVOIR

L'option `-a` avec `ifconfig` affiche également les interfaces désactivées. Avec `ip`, la commande `ip link` affiche l'état des interfaces (up/down) indépendamment de leur configuration IP.

5.3. Connexions et sockets (netstat, ss)

La commande `netstat` affiche les connexions réseau actives, les tables de routage et les statistiques d'interface. La commande `ss` (socket statistics) est son équivalent moderne, plus rapide et plus détaillé, car elle lit directement les informations du noyau via l'interface Netlink.

```
netstat -tuln
# netstat : affiche les connexions réseau, tables de routage et interfaces
# -t : connexions TCP uniquement
# -u : connexions UDP uniquement
# -l : écoute uniquement (listening sockets)
# -n : affichage numérique des adresses et ports (pas de résolution DNS)

ss -tuln
# ss : affiche les statistiques des sockets (remplace netstat)
# -t : TCP, -u : UDP, -l : écoute, -n : numérique
# Plus rapide que netstat car lit directement depuis le noyau
```

CONTRE-MESURE

`netstat` peut être lent sur les systèmes avec beaucoup de connexions car il interroge `/proc` pour chaque socket. Préférer `ss` pour les diagnostics rapides en production.

5.4. Résolution de noms (nslookup, dig)

La commande `nslookup` et la commande `dig` (domain information groper) permettent de interroger les serveurs DNS pour résoudre un nom de domaine en adresse IP, ou inversement. `dig` est plus détaillé et plus flexible que `nslookup`.

```
nslookup example.com
# nslookup : interroge un serveur DNS pour résoudre un nom de domaine
# example.com : nom de domaine à résoudre
# Affiche l'adresse IP associée et le serveur DNS utilisé

dig example.com
# dig : outil avancé de requête DNS (domain information groper)
# example.com : nom de domaine à résoudre
# Affiche la réponse complète du serveur DNS, y compris les enregistrements et le temps de réponse
```

CONCEPT CLÉ

Le DNS (Domain Name System) traduit les noms de domaine lisibles par les humains (exemple.com) en adresses IP numériques (93.184.216.34) compréhensibles par le réseau. Sans DNS, il faudrait mémoriser l'adresse IP de chaque service.

Fig 5 — Topologie de surveillance système et réseau

Diagramme		
B	ps, top, free	
B	df, du	
B	ping, ip, ss	
B	who, w, last	
C	résultat	
D	résultat	
E	résultat	
F	résultat	B

Figure — Diagramme Mermaid

Lab 01 — Inspection de l'environnement shell (45min)

Environnement

Élément	Valeur
Poste	Linux (Bash)
Shell	/bin/bash
Éditeur	nano / vim

Exercices

Exercice 01 — Afficher les variables d'environnement (10min)

```
env | head -20
# env : affiche toutes les variables d'environnement du processus courant
# head -20 : limite l'affichage aux 20 premières lignes pour une lecture concise
# | : tube qui redirige la sortie de env vers l'entrée de head
```

- La liste des variables d'environnement s'affiche

- Les variables HOME, PATH, SHELL, USER, LANG sont identifiables

Exercice 02 — Créer et exporter une variable (10min)

```
MON_PROJET=scripting-b2
# MON_PROJET=scripting-b2 : assigne une valeur à la variable shell MON_PROJET
# Pas d'espace autour du signe =
export MON_PROJET
# export : rend MON_PROJET accessible aux processus enfants (child processes)
echo $MON_PROJET
# echo : affiche la valeur de MON_PROJET dans le shell courant
bash -c 'echo $MON_PROJET'
# bash -c : lance un nouveau shell Bash enfant qui exécute la commande entre guillemets
# $MON_PROJET : dans le shell enfant, la variable est visible car elle a été exportée
```

- La variable MON_PROJET est créée avec la valeur scripting-b2
- La variable est exportée et visible depuis un shell enfant

Exercice 03 — Inspecter le fichier /etc/profile (10min)

```
cat /etc/profile | head -30
# cat : affiche le contenu du fichier de configuration système globale
# /etc/profile : fichier lu à l'ouverture de session pour tous les utilisateurs
# head -30 : limite l'affichage aux 30 premières lignes
```

- Le fichier /etc/profile est lisible
- Les lignes affichées concernent la configuration système (PATH, umask, etc.)

Solution

Solution — Exercices 01, 02 et 03

```
# Exercice 01
env | head -20

# Exercice 02
MON_PROJET=scripting-b2
export MON_PROJET
echo $MON_PROJET
bash -c 'echo $MON_PROJET'

# Exercice 03
cat /etc/profile | head -30
```

Résultat attendu pour l'exercice 02 : la valeur `scripting-b2` s'affiche deux fois, une fois dans le shell courant et une fois dans le shell enfant lancé par `bash -c`.

Exercices supplémentaires — Analyse des fichiers de données fictives (donnees_fictives/)

Exercice 01b — Inspecter le fichier system.log (10min)

Le fichier `donnees_fictives/system.log` contient des entrées de journal système simulées. Utilisez les commandes d'information système pour l'analyser.

```
wc -l donnees_fictives/system.log
# wc : compte les lignes, mots et octets d'un fichier
# -l : compte uniquement le nombre de lignes
# donnees_fictives/system.log : fichier de journal système fictif

head -5 donnees_fictives/system.log
# head : affiche les premières lignes d'un fichier
# -5 : limite à 5 lignes
# donnees_fictives/system.log : fichier cible

tail -5 donnees_fictives/system.log
# tail : affiche les dernières lignes d'un fichier
# -5 : limite à 5 lignes
# donnees_fictives/system.log : fichier cible
```

- Le fichier system.log contient 23 lignes
- Les 5 premières lignes s'affichent correctement
- Les 5 dernières lignes s'affichent correctement

Exercice 02b — Lister les fichiers de données fictives (10min)

```
ls -l donnees_fictives/
# ls : liste le contenu d'un répertoire
# -l : format détaillé (droits, propriétaire, taille, date)
# donnees_fictives/ : répertoire contenant les fichiers de données

du -sh donnees_fictives/
# du : estime l'espace disque utilisé
# -s : résumé, affiche uniquement le total
# -h : format lisible par un humain
# donnees_fictives/ : répertoire cible
```

- Tous les fichiers de donnees_fictives/ sont listés
- La taille totale du répertoire s'affiche

Solution

Solution — Exercices 01b et 02b

```
# Exercice 01b
wc -l donnees_fictives/system.log
head -5 donnees_fictives/system.log
tail -5 donnees_fictives/system.log

# Exercice 02b
ls -l donnees_fictives/
du -sh donnees_fictives/
```

Résultat attendu pour l'exercice 01b : `23 donnees_fictives/system.log`, puis les 5 premières et 5 dernières lignes du fichier.

Résultat attendu pour l'exercice 02b : liste détaillée des 6 fichiers du répertoire, puis la taille totale (ex. `4,0K donnees_fictives/`).

Démonstration commentée

L'exécution de `env` liste l'environnement complet du processus shell courant. Chaque ligne a le format `NOM=VALEUR`. Le shell transmet cet environnement à tout programme qu'il lance, permettant ainsi aux scripts de connaître l'utilisateur (`$USER`), son répertoire (`$HOME`), son shell (`$SHELL`) et sa locale (`$LANG`). Lorsqu'une variable est exportée avec `export`, elle est copiée dans l'environnement de chaque processus enfant. Si un script modifie une variable exportée, cette modification n'affecte que la copie du script, pas le shell parent.

Correction et résolution pas à pas

- Ouvrez un terminal Bash.
- Tapez `env | head -20` pour visualiser un extrait des variables d'environnement. Observez les lignes `HOME=...`, `PATH=...`, `SHELL=...`, `USER=...`, `LANG=...`.
- Créez une variable locale avec `MON_PROJET=scripting-b2` (sans espaces autour de `=`).
- Rendez-la accessible aux sous-processus avec `export MON_PROJET`.
- Vérifiez dans le shell courant avec `echo $MON_PROJET`.
- Vérifiez dans un shell enfant avec `bash -c 'echo $MON_PROJET'`. La valeur doit s'afficher, confirmant l'héritage par le processus enfant.
- Inspectez `/etc/profile` avec `cat /etc/profile | head -30` pour voir la configuration système globale.

Bonnes pratiques

Pratique	Description
Nommage explicite	Utiliser des noms de variables décrivant leur contenu (ex. MON_PROJET, CHEMIN_SAUVEGARDE)
Export sélectif	N'exporter que les variables nécessaires aux sous-processus
Majuscules pour l'environnement	Utiliser des noms en majuscules pour les variables d'environnement

BONNES PRATIQUES

Privilégiez les noms en majuscules pour les variables d'environnement afin de les distinguer des variables locales. N'exportez jamais une variable contenant des informations sensibles (mot de passe, clé API) sans réfléchir aux implications de sécurité.

Script de correction : `env_check.sh`

Exercices — Chapitre 1

Exercice 04 — Vérifier le shell courant (5min)

```
echo $SHELL
# $SHELL : contient le chemin vers l'interpréteur de commandes par défaut de l'utilisateur
```

- Quel est le chemin de votre shell par défaut ?
- Est-ce Bash ?

Solution

Solution — Exercice 04

```
echo $SHELL
```

Attendu : `/bin/bash` (ou un chemin équivalent selon la distribution, par exemple `/usr/bin/bash`). Si le chemin affiché contient `bash`, le shell par défaut est bien Bash.

- Le chemin du shell est affiché
- Le shell est identifié comme Bash

Exercice 05 — Comparer variable locale et variable d'environnement (10min)

```
MA_VAR_LOCALE="visible_ici_seulement"
export MA_VAR_ENV="visible_partout"
bash -c 'echo "Locale : $MA_VAR_LOCALE"'
bash -c 'echo "Environnement : $MA_VAR_ENV"'
```

- Quelle commande affiche une valeur et laquelle affiche une chaîne vide ?
- Pourquoi ?

Solution

Solution — Exercice 05

La première commande `bash -c 'echo "Locale : $MA_VAR_LOCALE"'` affiche une chaîne vide car `MA_VAR_LOCALE` n'a pas été exportée : elle n'est pas transmise au shell enfant. La deuxième commande `bash -c 'echo "Environnement : $MA_VAR_ENV"'` affiche `visible_partout` car `MA_VAR_ENV` a été exportée et est donc héritée par le processus enfant.

- La différence entre variable locale et variable d'environnement est comprise
- Les deux commandes sont testées et le résultat est interprété

CE QU'ON A RETENU

Le shell Bash est l'intermédiaire entre l'utilisateur et le noyau Linux. Les fichiers de configuration (`/etc/profile`, `~/.bashrc`, `~/.bash_profile`) personnalisent l'environnement. Une variable shell reste locale tant qu'elle n'est pas exportée ; `export` la transforme en variable d'environnement héritée par les processus enfants. Les variables courantes HOME, PATH, SHELL, USER et LANG fournissent des informations essentielles sur l'environnement d'exécution.

Chapitre 2 — Syntaxe de base : variables, types, commandes simples (11h15-12h00)

Objectifs

- Déclarer et utiliser des variables en Bash sans erreur de syntaxe
- Comprendre que toutes les variables Bash sont des chaînes de caractères
- Maîtriser l'interpolation : `"$var"` vs `'$var'`

- Utiliser les commandes simples et les redirections

Contenu théorique (40min)

1. Assignation de variable et typage (25min)

En Bash, toutes les variables sont des chaînes de caractères (strings). Il n'existe pas de typage strict comme dans un langage compilé (C, Java, etc.). Une variable contenant `30` est traitée comme le texte `"30"` ; les opérations arithmétiques nécessitent une syntaxe spécifique (`$(( ))`).

L'assignation d'une variable suit une règle stricte : il ne doit **jamais** y avoir d'espace autour du signe `=`. `VAR=valeur` est correct ; `VAR = valeur` est une erreur de syntaxe que Bash interprétera comme une tentative d'exécution de la commande `VAR` avec des arguments.

```
NOM="Alice"
# NOM="Alice" : assigne la chaîne de caractères "Alice" à la variable NOM
# Les guillemets doubles protègent la valeur et permettent l'interpolation ultérieure
AGE=30
# AGE=30 : assigne la chaîne "30" à la variable AGE (tout est chaîne en Bash)
NOM_ESPACE="Alice Dupont"
# NOM_ESPACE="Alice Dupont" : les guillemets doubles permettent d'inclure des espaces dans la valeur
```

CONCEPT CLÉ

En Bash, il n'y a jamais d'espace autour du signe `=` lors d'une assignation : `VAR=valeur`, jamais `VAR = valeur`. Un espace autour de `=` transforme l'assignation en tentative d'exécution de commande.

PIÈGE

Si vous oubliez les guillemets autour d'une valeur contenant des espaces (`NOM=Alice Dupont`), Bash tente d'exécuter `Dupont` comme une commande avec `NOM=Alice` comme affectation temporaire, ce qui produit une erreur `command not found`.

2. Interpolation et protection des variables (15min)

L'interpolation consiste à remplacer le nom d'une variable par sa valeur au sein d'une chaîne de caractères. Le comportement dépend du type de guillemets utilisé :

- Entre guillemets doubles (`"$var"`), Bash remplace `$var` par sa valeur.
- Entre guillemets simples (`'$var'`), Bash affiche le texte littéralement, sans interprétation.

```
NOM="test"
echo '$NOM'
# echo : affiche la chaîne passée en argument
# '$NOM' : guillemets simples, pas d'interprétation, affiche littéralement $NOM
echo "$NOM"
# echo : affiche la chaîne passée en argument
# "$NOM" : guillemets doubles, Bash remplace $NOM par sa valeur test
```

Résultat attendu :

```
$NOM
test
```

3. Commandes simples et redirections (40min)

Une commande simple se compose du nom de la commande suivi d'arguments et d'options (flags). Les commandes de base du système Linux (coreutils) incluent `echo`, `printf`, `date`, `whoami`, `hostname`, `pwd`, `ls`, `cat`, etc.

Les redirections permettent de modifier l'origine ou la destination des flux de données :

- `>` : redirige la sortie standard vers un fichier (écrasement)
- `>>` : redirige la sortie standard vers un fichier (ajout à la fin)
- `<` : redirige l'entrée standard depuis un fichier
- `|` : tube (pipeline), redirige la sortie d'une commande vers l'entrée d'une autre

```
echo "Bonjour le monde"
# echo : affiche la chaîne de caractères sur la sortie standard
date
# date : affiche la date et l'heure système actuelles
whoami
# whoami : affiche le nom de l'utilisateur courant
hostname
# hostname : affiche le nom de la machine (hôte)
pwd
# pwd : affiche le chemin complet du répertoire de travail courant
ls -l /home
# ls : liste le contenu d'un répertoire
# -l : format détaillé (droits, propriétaire, taille, date de modification)
# /home : répertoire cible à lister
cat /etc/hostname
# cat : affiche le contenu d'un fichier sur la sortie standard
# /etc/hostname : fichier contenant le nom de la machine
```

```
ls -l /home > listing.txt
# ls -l : liste détaillée du répertoire /home
# > listing.txt : redirige la sortie standard vers le fichier listing.txt (écrasement si existant)
date >> listing.txt
# date : affiche la date et l'heure
# >> listing.txt : ajoute la sortie à la fin du fichier listing.txt sans écraser
wc -l < listing.txt
# wc -l : compte le nombre de lignes
# < listing.txt : redirige l'entrée standard depuis le fichier listing.txt
ls -l /home | head -5
# ls -l : liste détaillée du répertoire /home
# | : tube qui redirige la sortie de ls vers l'entrée de head
# head -5 : affiche les 5 premières lignes
```

CONCEPT CLÉ

Les redirections `>` et `>>` agissent sur la sortie standard (stdout, descripteur de fichier 1). Le tube `|` relie la sortie standard d'une commande à l'entrée standard (stdin, descripteur de fichier 0) de la commande suivante, formant une chaîne de traitement (pipeline).

BON À SAVOIR

Le descripteur de fichier 2 correspond à la sortie d'erreur (stderr). Pour rediriger les erreurs vers un fichier, utilisez `2> fichier` ou `2>&1` pour fusionner stderr sur stdout.

Fig 2 — Pipeline d'écriture et d'exécution d'un script Bash

Diagramme		
B	chmod +x	
C	./script.sh	
D	interprétation	E

Figure — Diagramme Mermaid

4. Commandes de lecture de fichiers (cat, less, head, tail, wc) (25min)

La lecture de fichiers est une opération fondamentale en administration système. Selon le fichier et le besoin, différentes commandes sont appropriées : `cat` pour afficher un fichier entier, `less` pour une navigation paginée, `head` et `tail` pour extraire des parties, et `wc` pour compter.

4.1. Affichage complet et paginé (cat, less)

La commande `cat` (concatenate) affiche le contenu d'un fichier sur la sortie standard. Elle est adaptée aux fichiers courts. Pour les fichiers longs, `less` permet une navigation interactive page par page sans charger l'intégralité du fichier en mémoire.

```
cat donnees_fictives/system.log
# cat : affiche le contenu complet d'un fichier sur la sortie standard
# donnees_fictives/system.log : fichier de journal système à afficher
# Inconvénient : défile tout le contenu, inadapté aux fichiers volumineux

less donnees_fictives/system.log
# less : affiche un fichier de manière paginée, navigation interactive
# donnees_fictives/system.log : fichier à consulter
# Touches : Espace (page suivante), b (page précédente), / (recherche), q (quitter)
```

CONCEPT CLÉ

`cat` lit et affiche le fichier d'un seul bloc, ce qui convient aux petits fichiers. `less` lit le fichier par morceaux et permet de naviguer, de rechercher et de quitter sans charger tout le contenu en mémoire, ce qui est essentiel pour les fichiers journaux volumineux.

BON À SAVOIR

Dans `less`, la touche `/` suivie d'un motif permet de rechercher dans le fichier. La touche `n` passe à l'occurrence suivante, `N` à l'occurrence précédente.

4.2. Extraction de parties de fichier (head, tail, wc)

La commande `head` affiche les premières lignes d'un fichier, `tail` les dernières lignes, et `wc` (word count) compte les lignes, mots et octets.

```
head -10 donnees_fictives/system.log
# head : affiche les premières lignes d'un fichier
# -10 : limite à 10 lignes
# donnees_fictives/system.log : fichier cible

tail -10 donnees_fictives/system.log
# tail : affiche les dernières lignes d'un fichier
# -10 : limite à 10 lignes
# donnees_fictives/system.log : fichier cible

tail -f donnees_fictives/system.log
# tail -f : suit le fichier en temps réel (affiche les nouvelles lignes au fur et à mesure)
# -f : mode follow, utile pour surveiller un fichier de log en cours d'écriture
# Ctrl+C pour arrêter

wc -l donnees_fictives/system.log
# wc : compte les lignes, mots et octets
# -l : compte uniquement le nombre de lignes
# donnees_fictives/system.log : fichier cible

wc -w donnees_fictives/employes.csv
# wc : compte les lignes, mots et octets
# -w : compte uniquement le nombre de mots
# donnees_fictives/employes.csv : fichier CSV des employés
```

CONCEPT CLÉ

`tail -f` est particulièrement utile pour surveiller un fichier de journal en temps réel : chaque nouvelle ligne ajoutée au fichier s'affiche immédiatement à l'écran. C'est l'outil de prédilection pour le dépannage de services en production.

PIÈGE

`wc -l` compte les caractères de retour à la ligne (`\n`). Si le dernier d'un fichier ne se termine pas par un retour à la ligne, il ne sera pas compté comme une ligne complète.

5. Commandes de recherche (find, grep) (25min)

La recherche de fichiers et de contenus textuels est une tâche courante. `find` permet de localiser des fichiers selon leurs attributs (nom, taille, date, permissions), tandis que `grep` recherche des motifs textuels à l'intérieur des fichiers.

5.1. Recherche de fichiers (find)

La commande `find` parcourt une arborescence de répertoires et sélectionne les fichiers correspondant à des critères de recherche.

```
find donnees_fictives/ -name "*.txt"
# find : recherche des fichiers dans une arborescence
# donnees_fictives/ : répertoire racine de la recherche
# -name : filtre par nom de fichier (motif avec jokers)
# "*.txt" : tous les fichiers se terminant par .txt

find donnees_fictives/ -type f -name "*.log"
# find : recherche des fichiers dans une arborescence
# donnees_fictives/ : répertoire racine
# -type f : ne sélectionne que les fichiers réguliers (exclut les répertoires)
# -name "*.log" : filtre par nom se terminant par .log

find /var/log -mtime -1
# find : recherche des fichiers modifiés récemment
# /var/log : répertoire des journaux système
# -mtime -1 : fichiers modifiés il y a moins de 24 heures
# Le signe - signifie "moins de", + signifierait "plus de"
```

CONCEPT CLÉ

`find` est récursif par défaut : il explore le répertoire de départ et tous ses sous-répertoires. Les options `-name`, `-type`, `-mtime`, `-size`, `-user` peuvent être combinées pour affiner la recherche.

BON À SAVOIR

L'option `-name` utilise des jokers shell (`*`, `?`, `[...]`). Pour rechercher une correspondance exacte sans joker, omettre les caractères spéciaux ou utiliser `-name "fichier_exact"`.

5.2. Recherche de texte (grep)

La commande `grep` (global regular expression print) recherche des lignes contenant un motif donné dans un ou plusieurs fichiers.

```
grep "ERROR" donnees_fictives/system.log
# grep : recherche des lignes contenant un motif
# "ERROR" : motif à rechercher (chaîne littérale)
# donnees_fictives/system.log : fichier dans lequel chercher
# Affiche toutes les lignes contenant le mot ERROR

grep -i "error" donnees_fictives/system.log
# grep : recherche des lignes contenant un motif
# -i : insensibilité à la casse (majuscules/minuscules ignorées)
# "error" : motif à rechercher, correspond aussi à ERROR, Error, etc.

grep -c "ERROR" donnees_fictives/system.log
# grep : recherche des lignes contenant un motif
# -c : compte le nombre de lignes correspondantes au lieu de les afficher
# "ERROR" : motif à rechercher
# donnees_fictives/system.log : fichier cible

grep -n "WARN" donnees_fictives/system.log
# grep : recherche des lignes contenant un motif
# -n : affiche le numéro de ligne devant chaque résultat
# "WARN" : motif à rechercher
# donnees_fictives/system.log : fichier cible
```

CONCEPT CLÉ

`grep` utilise des expressions régulières (regex) pour définir des motifs de recherche. Par défaut, le motif est traité comme une chaîne littérale, mais des caractères spéciaux comme `.`, `*`, `^`, `$` peuvent être utilisés pour des recherches avancées.

CONTRE-MESURE

Toujours protéger le motif entre guillemets doubles pour éviter que le shell n'interprète les caractères spéciaux (notamment `*`, `?`, `$`). Utiliser `grep -i` pour une recherche insensible à la casse.

6. Commandes de traitement de texte (awk, sed, cut, sort, uniq, diff, patch) (25min)

Ces commandes permettent de transformer, extraire et comparer des données textuelles. Elles sont particulièrement puissantes lorsqu'elles sont combinées dans des pipelines.

6.1. Extraction de colonnes (cut)

La commande `cut` extrait des colonnes ou des champs à partir de lignes de texte, selon un délimiteur ou des positions de caractères.

```
cut -d',' -f1,3 donnees_fictives/employes.csv
# cut : extrait des sections de chaque ligne d'un fichier
# -d',' : définit la virgule comme délimiteur de champs
# -f1,3 : sélectionne les champs 1 (nom) et 3 (salaire)
# donnees_fictives/employes.csv : fichier CSV des employés

cut -c1-10 donnees_fictives/noms.txt
# cut : extrait des sections de chaque ligne
# -c1-10 : extrait les caractères 1 à 10 de chaque ligne
# donnees_fictives/noms.txt : fichier contenant des noms
```

6.2. Tri et dédoublement (sort, uniq)

La commande `sort` ordonne les lignes d'un fichier. La commande `uniq` supprime les lignes consécutives identiques (nécessite un tri préalable).

```
sort donnees_fictives/noms.txt
# sort : trie les lignes d'un fichier par ordre alphabétique
# donnees_fictives/noms.txt : fichier contenant des noms (avec doublons)

sort donnees_fictives/noms.txt | uniq
# sort : trie les lignes pour regrouper les doublons
# | : tube qui redirige la sortie de sort vers l'entrée de uniq
# uniq : supprime les lignes consécutives identiques
# Résultat : liste des noms sans doublons, chaque nom apparaissant une seule fois

sort donnees_fictives/doublons.txt | uniq -c
# uniq -c : compte le nombre d'occurrences de chaque ligne unique
# -c : préfixe chaque ligne par le nombre d'occurrences
# donnees_fictives/doublons.txt : fichier contenant des mots en doublon
```

> **CONCEPT CLÉ**
>
> `uniq` ne détecte que les doublons consécutifs. Il est donc impératif de trier le fichier avec `sort` avant d'appliquer `uniq` pour éliminer tous les doublons, quelle que soit leur position dans le fichier.

6.3. Éditeur de flux (sed)

La commande `sed` (stream editor) effectue des transformations sur un flux de texte, notamment des substitutions de motifs.

```
sed 's/Informatique/IT/g' donnees_fictives/employes.csv
# sed : éditeur de flux pour filtrer et transformer du texte
# 's/Informatique/IT/g' : commande de substitution
# s : substituer, Informatique : motif à remplacer, IT : texte de remplacement
# g : global, remplace toutes les occurrences sur chaque ligne (pas seulement la première)
# donnees_fictives/employes.csv : fichier cible
```

6.4. Traitement de colonnes et rapport (awk)

La commande `awk` est un langage de programmation complet pour le traitement de texte, particulièrement adapté à l'extraction de colonnes et au calcul de totaux.

```
awk -F',' '{print $1, $3}' donnees_fictives/employes.csv
# awk : langage de traitement de texte orienté colonnes
# -F',' : définit la virgule comme séparateur de champs
# '{print $1, $3}' : affiche le champ 1 (nom) et le champ 3 (salaire)
# $1, $3 : références aux champs séparés par le délimiteur
# donnees_fictives/employes.csv : fichier CSV des employés

awk -F',' 'NR>1 {somme+=$3} END {print "Somme des salaires :", somme}' donnees_fictives/employes.csv
# NR : numéro de ligne courant (Number of Record)
# NR>1 : saute l'en-tête (première ligne)
# somme+=$3 : additionne le champ 3 (salaire) à chaque ligne
# END : bloc exécuté après la lecture de toutes les lignes
# print : affiche le résultat final
```

6.5. Comparaison de fichiers (diff, patch)

La commande `diff` compare deux fichiers ligne par ligne et affiche les différences. La commande `patch` applique un fichier de différences (produit par `diff`) à un fichier original.

```
diff donnees_fictives/noms.txt donnees_fictives/doublons.txt
# diff : compare deux fichiers ligne par ligne
# donnees_fictives/noms.txt : premier fichier de référence
# donnees_fictives/doublons.txt : second fichier à comparer
# Affiche les lignes à ajouter, modifier ou supprimer pour passer de l'un à l'autre
```

CONCEPT CLÉ

Le pipeline (`|`) permet de chaîner ces commandes pour effectuer des traitements complexes en une seule ligne. Par exemple, `sort fichier | uniq -c | sort -nr` compte les occurrences de chaque ligne et trie par fréquence décroissante.

BON À SAVOIR

`awk` est plus puissant que `cut` car il permet des calculs arithmétiques, des conditions et des blocs de début/fin (BEGIN/END). `sed` est idéal pour des substitutions simples, tandis que `awk` convient aux traitements structurés.

Fig 6 — Pipeline de traitement de texte

Diagramme		
B	grep	
C	awk	
D	sort	
E	uniq	
F	wc	G

Figure — Diagramme Mermaid

Lab 02 — Manipulation de variables et commandes (45min)

Environnement

Élément	Valeur
Poste	Linux (Bash)
Éditeur	nano / vim

Exercices

Exercice 06 — Créer des variables et les afficher (10min)

```
UTILISATEUR="admin"
# UTILISATEUR="admin" : assigne la chaîne "admin" à la variable UTILISATEUR
SERVEUR="web01"
# SERVEUR="web01" : assigne la chaîne "web01" à la variable SERVEUR
echo "L'utilisateur $UTILISATEUR est connecté sur le serveur $SERVEUR."
# echo : affiche le message avec interpolation des variables $UTILISATEUR et $SERVEUR
# Les guillemets doubles permettent l'interprétation des variables et protègent les espaces
```

- Les variables UTILISATEUR et SERVEUR sont créées
- Le message s'affiche avec les valeurs interpolées correctement

Exercice 07 — Tester l'absence d'interprétation (10min)

```
NOM="test"
echo '$NOM'
# '$NOM' : guillemets simples, pas d'interprétation, affiche littéralement $NOM
echo "$NOM"
# "$NOM" : guillemets doubles, Bash remplace $NOM par sa valeur test
```

- Quelle est la différence entre les deux lignes ?
- Pourquoi ?

- La différence d'interprétation est comprise
- Les deux formes sont testées

Exercice 08 — Rediriger la sortie vers un fichier (15min)

```
date > rapport.txt
# date : affiche la date et l'heure système
# > rapport.txt : redirige la sortie vers le fichier rapport.txt (écrasement)
whoami >> rapport.txt
# whoami : affiche le nom de l'utilisateur courant
# >> rapport.txt : ajoute la sortie à la fin du fichier sans écraser
cat rapport.txt
# cat : affiche le contenu du fichier rapport.txt
```

- Le fichier rapport.txt est créé avec la date
- Le nom de l'utilisateur est ajouté à la fin du fichier
- Le contenu du fichier s'affiche correctement

Solution

Solution — Exercices 06, 07 et 08

```
# Exercice 06
UTILISATEUR="admin"
SERVEUR="web01"
echo "L'utilisateur $UTILISATEUR est connecté sur le serveur $SERVEUR."

# Exercice 07
NOM="test"
echo '$NOM'
echo "$NOM"

# Exercice 08
date > rapport.txt
whoami >> rapport.txt
cat rapport.txt
```

Résultat attendu pour l'exercice 07 :

```
$NOM
test
```

Résultat attendu pour l'exercice 08 : le fichier `rapport.txt` contient d'abord la date et l'heure, puis le nom de l'utilisateur sur une ligne supplémentaire.

Exercices supplémentaires — Traitement de texte sur les données fictives (donnees_fictives/)

Exercice 08b — Lire et compter les lignes du fichier system.log (10min)

```
wc -l donnees_fictives/system.log
# wc : compte les lignes, mots et octets
# -l : compte uniquement le nombre de lignes
# donnees_fictives/system.log : fichier de journal système

head -5 donnees_fictives/system.log
# head : affiche les premières lignes d'un fichier
# -5 : limite à 5 lignes
# donnees_fictives/system.log : fichier cible
```

- Le nombre de lignes de system.log s'affiche
- Les 5 premières lignes du fichier s'affichent

Exercice 09b — Rechercher des erreurs dans system.log avec grep (10min)

```
grep "ERROR" donnees_fictives/system.log
# grep : recherche des lignes contenant un motif
# "ERROR" : motif à rechercher (majuscules)
# donnees_fictives/system.log : fichier de journal système

grep -i "error" donnees_fictives/system.log
# grep : recherche des lignes contenant un motif
# -i : insensibilité à la casse (majuscules/minuscules)
# "error" : motif à rechercher, correspond aussi à ERROR, Error
# donnees_fictives/system.log : fichier cible

grep -c "ERROR" donnees_fictives/system.log
# grep : recherche des lignes contenant un motif
# -c : compte le nombre de lignes correspondantes
# "ERROR" : motif à rechercher
# donnees_fictives/system.log : fichier cible
```

- Les lignes contenant ERROR s'affichent
- La recherche insensible à la casse trouve les mêmes lignes
- Le nombre de lignes ERROR s'affiche

Exercice 10b — Extraire des colonnes du fichier employes.csv avec cut et awk (15min)

```
cut -d',' -f1,3 donnees_fictives/employes.csv
# cut : extrait des sections de chaque ligne
# -d',' : définit la virgule comme délimiteur de champs
# -f1,3 : sélectionne les champs 1 (nom) et 3 (salaire)
# donnees_fictives/employes.csv : fichier CSV des employés

awk -F',' '{print $1, $3}' donnees_fictives/employes.csv
# awk : langage de traitement de texte orienté colonnes
# -F',' : définit la virgule comme séparateur de champs
# '{print $1, $3}' : affiche le champ 1 (nom) et le champ 3 (salaire)
# donnees_fictives/employes.csv : fichier CSV des employés
```

- Les noms et salaires s'affichent avec cut
- Les noms et salaires s'affichent avec awk
- Les deux commandes produisent le même résultat

Exercice 11b — Trier et dédoubler le fichier noms.txt (15min)

```
sort donnees_fictives/noms.txt
# sort : trie les lignes d'un fichier par ordre alphabétique
# donnees_fictives/noms.txt : fichier contenant des noms (avec doublons)

sort donnees_fictives/noms.txt | uniq
# sort : trie les lignes pour regrouper les doublons
# | : tube qui redirige la sortie de sort vers l'entrée de uniq
# uniq : supprime les lignes consécutives identiques
# donnees_fictives/noms.txt : fichier source

sort donnees_fictives/doublons.txt | uniq -c
# uniq -c : compte le nombre d'occurrences de chaque ligne unique
# -c : préfixe chaque ligne par le nombre d'occurrences
# donnees_fictives/doublons.txt : fichier contenant des mots en doublon
```

- Les noms sont triés par ordre alphabétique

- Les doublons sont supprimés après sort | uniq
- Le nombre d'occurrences de chaque mot s'affiche avec uniq -c

Solution

Solution — Exercices 08b, 09b, 10b et 11b

```
# Exercice 08b
wc -l donnees_fictives/system.log
head -5 donnees_fictives/system.log

# Exercice 09b
grep "ERROR" donnees_fictives/system.log
grep -i "error" donnees_fictives/system.log
grep -c "ERROR" donnees_fictives/system.log

# Exercice 10b
cut -d',' -f1,3 donnees_fictives/employes.csv
awk -F',' '{print $1, $3}' donnees_fictives/employes.csv

# Exercice 11b
sort donnees_fictives/noms.txt
sort donnees_fictives/noms.txt | uniq
sort donnees_fictives/doublons.txt | uniq -c
```

Résultat attendu pour l'exercice 09b : 5 lignes contenant ERROR dans system.log.

Résultat attendu pour l'exercice 11b : `sort | uniq` affiche 15 noms uniques (les 15 noms de la liste, sans les 5 doublons). `sort | uniq -c` affiche chaque mot de doublons.txt avec son nombre d'occurrences (pomme: 5, banane: 4, orange: 3, kiwi: 3, mangue: 3, raisin: 2).

Démonstration commentée

L'interpolation de variable (`$NOM`) permet d'insérer la valeur d'une variable dans une chaîne entre guillemets doubles. Entre guillemets simples, l'interprétation est désactivée et `$NOM` s'afficherait littéralement. Les redirections `>` et `>>` permettent respectivement d'écraser ou d'ajouter du contenu dans un fichier, tandis que le tube `|` enchaîne plusieurs commandes pour traiter un flux de données de manière séquentielle.

Correction et résolution pas à pas

- Déclarez `UTILISATEUR="admin"` et `SERVEUR="web01"` sans espaces autour de `=`.
- Affichez un message avec `echo "L'utilisateur $UTILISATEUR est connecté sur le serveur $SERVEUR."`.
- Testez `echo '$NOM'` puis `echo "$NOM"` avec `NOM="test"` pour observer la différence d'interprétation.
- Redirigez la sortie de `date` vers `rapport.txt` avec `>`, puis ajoutez `whoami` avec `>>`.
- Affichez le contenu du fichier avec `cat rapport.txt`.

Bonnes pratiques

Pratique	Description	
Guillemets doubles	Encadrer les valeurs de variables pour gérer les espaces et caractères spéciaux	
Pas d'espaces autour de =	Respecter `VAR=valeur` sans espaces	
Redirection appropriée	Utiliser `>` pour écraser, `>>` pour ajouter, `	` pour chaîner

BONNES PRATIQUES

Utilisez toujours des guillemets doubles autour des variables interpolées pour éviter les erreurs liées aux espaces dans les valeurs. Préférez `>>` à `>` lorsque vous souhaitez conserver l'historique d'un fichier de log.

Script de correction : `vars_check.sh`

Exercices — Chapitre 2

Exercice 09 — Afficher des informations système (5min)

```
echo "Utilisateur : $(whoami)"
# echo : affiche la chaîne
# $(whoami) : substitution de commande, exécute whoami et insère le résultat
echo "Répertoire : $(pwd)"
# $(pwd) : substitution de commande, exécute pwd et insère le chemin courant
```

- Que fait la syntaxe `$(commande)` ?
- Quel est l'avantage par rapport à l'utilisation de guillemets simples ?

Solution

Solution — Exercice 09

La syntaxe `$(commande)` est une substitution de commande : Bash exécute la commande entre parenthèses et remplace l'expression par le résultat de la commande. Contrairement aux guillemets simples qui affichent le texte littéralement, `$(commande)` permet d'insérer dynamiquement le résultat d'une commande dans une chaîne.

- La substitution de commande est comprise
- Les commandes whoami et pwd sont testées

CE QU'ON A RETENU

En Bash, toutes les variables sont des chaînes de caractères et l'assignation ne tolère aucun espace autour de `=`. L'interpolation fonctionne entre guillemets doubles (`"$var"`) mais est désactivée entre guillemets simples (`'$var'`). Les commandes simples (`echo`, `date`, `whoami`, `hostname`, `pwd`, `ls`, `cat`) peuvent être combinées avec des redirections (`>`, `>>`, `<`, `|`) pour diriger les flux de données.

Chapitre 3 — Création et exécution d'un script (13h00-14h30)

Objectifs

- Créer un fichier de script Bash valide avec shebang
- Rendre le script exécutable avec `chmod 755` et `chmod u+x`
- Exécuter un script de trois façons : `./script.sh`, `bash script.sh`, `source script.sh`
- Vérifier le code de retour `$?` (0 = succès, non-zéro = erreur)

Contenu théorique (1h15)

1. Le shebang et la structure d'un script (30min)

Un script Bash commence par une ligne appelée **shebang** (désignée par `#!`) qui indique au noyau quel interpréteur doit être utilisé pour exécuter le fichier. Cette ligne doit impérativement être la toute première ligne du fichier. Selon How Linux Works, 2nd Edition (Ward, Brian), page 251, « The #! part is called a [shebang] ».

Le shebang `#!/bin/bash` indique d'utiliser l'interpréteur Bash situé dans `/bin/bash`. Sans shebang, le shell par défaut de l'utilisateur serait utilisé, ce qui peut comporter des risques de compatibilité si le script utilise des fonctionnalités spécifiques à Bash.

```
#!/bin/bash
# #!/bin/bash : shebang, indique d'utiliser l'interpréteur Bash situé dans /bin/bash
# Ce script affiche un message de bienvenue et la date du jour

echo "Bienvenue dans ce script Bash"
# echo : affiche le message de bienvenue sur la sortie standard
date
# date : affiche la date et l'heure système actuelles
```

2. Rendre un script exécutable avec chmod (30min)

Par défaut, un nouveau fichier créé par un utilisateur n'est pas exécutable. Il faut modifier ses droits d'accès avec la commande `chmod` (change mode). Selon The Linux Command Line - A Complete Introduction, 3e (Shotts, William), page 353, il faut « make it executable » avec `chmod 755`. Selon Mastering Linux Security and Hardening (Tevault, Donald A.), page 80, on peut aussi utiliser `chmod u+x` pour ajouter la permission d'exécution au propriétaire.

Le mode `755` signifie :

- `7` (propriétaire) : lecture (4) + écriture (2) + exécution (1) = 7
- `5` (groupe) : lecture (4) + exécution (1) = 5
- `5` (autres) : lecture (4) + exécution (1) = 5

```
chmod 755 mon_script.sh
# chmod : modifie les droits d'accès au fichier
# 755 : lecture/écriture/exécution pour le propriétaire, lecture/exécution pour le groupe et les autres
# mon_script.sh : fichier cible à rendre exécutable
./mon_script.sh
# ./ : exécute le script situé dans le répertoire courant
# mon_script.sh : nom du script à exécuter
```

L'exécution directe avec `./mon_script.sh` nécessite que le répertoire courant soit explicitement indiqué, car pour des raisons de sécurité, le répertoire courant n'est généralement pas inclus dans la variable `PATH`.

```
chmod u+x mon_script.sh
# chmod : modifie les droits d'accès au fichier
# u+x : ajoute la permission d'exécution (x) pour le propriétaire (u) du fichier
# mon_script.sh : fichier cible
```

POURQUOI

Le répertoire courant n'est généralement pas dans le `PATH` pour des raisons de sécurité. `./mon_script.sh` indique explicitement d'exécuter le fichier situé dans le répertoire courant, évitant ainsi d'exécuter par erreur un programme système du même nom.

BON À SAVOIR

`bash mon_script.sh` ne requiert pas de droits d'exécution, car c'est l'interpréteur Bash qui lit et exécute le fichier. Le shebang est alors ignoré. Cette méthode est utile pour tester un script sans le rendre exécutable, mais elle ne fonctionne que pour Bash.

3. Les différentes façons d'exécuter un script (15min)

Il existe trois méthodes principales pour exécuter un script Bash :

- `./script.sh` : exécution directe, nécessite le shebang et les droits d'exécution
- `bash script.sh` : exécution via l'interpréteur Bash, ne nécessite pas de droits d'exécution, le shebang est ignoré
- `source script.sh` (ou `. script.sh`) : exécution dans le shell courant, sans lancer de processus enfant. Les variables et fonctions définies dans le script restent disponibles après l'exécution.

```
bash mon_script.sh
# bash : invoque l'interpréteur Bash pour exécuter le fichier mon_script.sh
# mon_script.sh : fichier de script à exécuter
source mon_script.sh
# source : exécute le script dans le shell courant sans lancer de processus enfant
# mon_script.sh : fichier de script à exécuter
# Les variables définies dans le script restent accessibles après l'exécution
```

4. Codes de retour et variable $? (15min)

Chaque commande ou script termine son exécution avec un **code de retour** (exit code). Selon How Linux Works, 2nd Edition (Ward, Brian), page 254, le code de sortie de la dernière commande détermine la façon dont le shell traite le conditionnel. Par convention :

- `0` : succès
- Toute valeur non-zéro (1 à 255) : erreur ou avertissement

La variable spéciale `$?` contient le code de retour de la dernière commande exécutée.

```
./mon_script.sh
# ./mon_script.sh : exécute le script situé dans le répertoire courant
echo $?
# $? : variable spéciale contenant le code de retour de la dernière commande exécutée
# 0 indique un succès, toute autre valeur indique une erreur
```

Fig 3 — Flux de création et exécution d'un script Bash

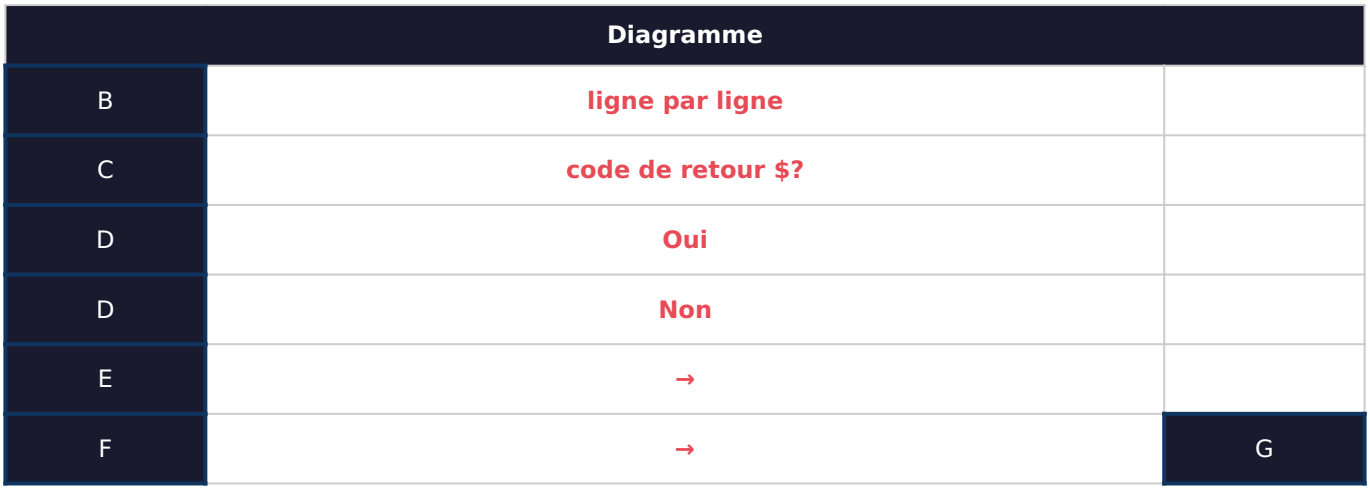


Figure — Diagramme Mermaid

5. Archivage et compression (tar, gzip, zip) (25min)

L'archivage consiste à regrouper plusieurs fichiers dans un seul conteneur, tandis que la compression réduit la taille des données. Ces opérations sont essentielles pour les sauvegardes, les transferts de fichiers et la distribution de logiciels.

5.1. Création et extraction d'archives (tar)

La commande `tar` (tape archive) regroupe plusieurs fichiers et répertoires dans un seul fichier archive, avec ou sans compression.

```
tar -cvf archive.tar donnees_fictives/
# tar : crée, liste ou extrait des archives
# -c : crée une nouvelle archive
# -v : mode verbeux, affiche les fichiers traités
# -f : spécifie le nom du fichier archive
# archive.tar : nom de l'archive à créer
# donnees_fictives/ : répertoire à archiver

tar -xvf archive.tar
# tar : extrait le contenu d'une archive
# -x : extrait les fichiers de l'archive
# -v : mode verbeux
# -f : spécifie le fichier archive
# archive.tar : archive à extraire

tar -czvf archive.tar.gz donnees_fictives/
# tar : crée une archive compressée avec gzip
# -c : crée une nouvelle archive
# -z : compresse avec gzip
# -v : mode verbeux
# -f : spécifie le nom du fichier archive
# archive.tar.gz : nom de l'archive compressée
# donnees_fictives/ : répertoire à archiver et compresser

tar -tzvf archive.tar.gz
# tar : liste le contenu d'une archive compressée sans l'extraire
# -t : liste le contenu de l'archive
# -z : l'archive est compressée avec gzip
# -v : mode verbeux
# -f : spécifie le fichier archive
```

CONCEPT CLÉ

`tar` ne compresse pas par défaut ; il regroupe simplement les fichiers. L'option `-z` ajoute la compression gzip, `-j` ajouterait la compression bzip2, et `-J` la compression xz. L'extension `.tar.gz` ou `.tgz` indique une archive tar compressée avec gzip.

BON À SAVOIR

L'option `-t` permet de lister le contenu d'une archive sans l'extraire, utile pour vérifier ce qu'elle contient avant une restauration.

5.2. Compression individuelle (gzip, zip)

La commande `gzip` compresse un fichier individuel, remplaçant l'original par une version compressée avec l'extension `.gz`. La commande `zip` crée des archives compatibles avec Windows.

```
gzip donnees_fictives/system.log
# gzip : compresse un fichier individuellement
# donnees_fictives/system.log : fichier à compresser
# Résultat : donnees_fictives/system.log.gz (l'original est remplacé)

gzip -d donnees_fictives/system.log.gz
# gzip -d : décompresse un fichier gzip
# -d : décompression (décompress)
# donnees_fictives/system.log.gz : fichier compressé à décompresser
# Résultat : donnees_fictives/system.log (l'original est restauré)

zip archive.zip donnees_fictives/*.txt
# zip : crée une archive compressée au format ZIP
# archive.zip : nom de l'archive à créer
# donnees_fictives/*.txt : tous les fichiers .txt du répertoire à inclure
# Format compatible avec Windows et macOS
```

CONCEPT CLÉ

`gzip` compresse un seul fichier et le remplace par sa version compressée. Pour compresser plusieurs fichiers individuellement, il faut appliquer `gzip` à chacun. `zip` peut regrouper plusieurs fichiers dans une seule archive, comme `tar`, mais avec un format plus largement interopérable.

PIÈGE

`gzip` supprime le fichier original après compression. Pour conserver l'original, utiliser l'option `-k` (keep) si disponible, ou copier le fichier avant compression.

6. Gestion des utilisateurs et groupes (chown, chgrp, umask, sudo, su, passwd, useradd, usermod, userdel, groupadd, groupdel) (25min)

La gestion des utilisateurs et des groupes constitue le cœur de la sécurité et du contrôle d'accès sur un système Linux. Chaque fichier appartient à un utilisateur et à un groupe, avec des droits d'accès distincts pour le propriétaire, le groupe et les autres.

6.1. Propriétaire et groupe (chown, chgrp)

La commande `chown` (change owner) modifie le propriétaire d'un fichier ou répertoire. La commande `chgrp` (change group) modifie le groupe propriétaire.

```
chown eleve fichier.txt
# chown : modifie le propriétaire d'un fichier
# eleve : nouveau propriétaire (nom d'utilisateur)
# fichier.txt : fichier cible

chown eleve:eleve fichier.txt
# chown : modifie le propriétaire et le groupe
# eleve:eleve : propriétaire et groupe, séparés par deux-points
# fichier.txt : fichier cible

chgrp developpeurs fichier.txt
# chgrp : modifie le groupe propriétaire d'un fichier
# developpeurs : nouveau groupe propriétaire
# fichier.txt : fichier cible
```

CONCEPT CLÉ

Sur Linux, chaque fichier a un propriétaire (user) et un groupe propriétaire (group). Les droits d'accès sont définis séparément pour ces trois catégories : propriétaire, groupe, et autres. Seul le propriétaire du fichier ou le superutilisateur (root) peut modifier ces attributs.

CONTRE-MESURE

L'utilisation de `chown` et `chgrp` nécessite généralement des privilèges de superutilisateur (via `sudo`). Ne jamais rendre un fichier appartenant à root accessible en écriture par tous les utilisateurs.

6.2. Masque de création (umask)

La commande `umask` (user mask) définit les permissions par défaut appliquées lors de la création d'un nouveau fichier ou répertoire. Elle soustrait des permissions du maximum théorique (666 pour les fichiers, 777 pour les répertoires).

```
umask
# umask : affiche le masque de création actuel
# Aucun flag requis : affiche la valeur octale (ex. 0022)

umask 0022
# umask : définit le masque de création
# 0022 : masque octal, retire l'écriture pour le groupe et les autres
# Résultat : fichiers créés avec 644 (rw-r--r--), répertoires avec 755 (rwxr-xr-x)
```

CONCEPT CLÉ

Le `umask` fonctionne par soustraction : permissions maximales (666 pour un fichier, 777 pour un répertoire) moins le umask donnent les permissions effectives. Avec umask 0022, un fichier créé aura les droits 644 (666 - 022) et un répertoire 755 (777 - 022).

BON À SAVOIR

Le `umask` s'applique uniquement aux nouvelles créations ; il ne modifie pas les fichiers existants. Pour modifier les droits d'un fichier existant, utiliser `chmod`.

6.3. Élévation de privilèges (sudo, su)

La commande `sudo` (superuser do) permet d'exécuter une commande avec les privilèges d'un autre utilisateur, généralement root. La commande `su` (switch user) permet de changer d'utilisateur dans le shell courant.

```
sudo ls /root
# sudo : exécute une commande avec les privilèges du superutilisateur
# ls : liste le contenu d'un répertoire
# /root : répertoire personnel de root (normalement inaccessible)
# Nécessite l'authentification de l'utilisateur courant

su - utilisateur
# su : change d'utilisateur dans le shell courant
# - : charge l'environnement complet de l'utilisateur cible (comme une nouvelle connexion)
# utilisateur : nom de l'utilisateur cible
# Nécessite le mot de passe de l'utilisateur cible
```

CONCEPT CLÉ

`sudo` est préféré à `su` dans l'administration moderne car il permet d'exécuter des commandes individuelles avec des privilèges élevés sans ouvrir une session root complète, et il journalise toutes les actions dans `/var/log/auth.log` ou via `journalctl`.

CONTRE-MESURE

Configurer `sudo` via `visudo` pour accorder des privilèges limités et spécifiques plutôt qu'un accès root complet. Éviter de partager le mot de passe root entre plusieurs administrateurs.

6.4. Gestion des mots de passe et comptes (passwd, useradd, usermod, userdel)

La commande `passwd` gère les mots de passe des utilisateurs. `useradd` crée un nouveau compte, `usermod` modifie un compte existant, et `userdel` supprime un compte.

```
passwd
# passwd : modifie le mot de passe de l'utilisateur courant
# Aucun flag requis : demande le mot de passe actuel puis le nouveau

sudo passwd utilisateur
# passwd : modifie le mot de passe d'un autre utilisateur
# sudo : élévation de privilèges requise pour modifier un autre compte
# utilisateur : nom du compte dont le mot de passe est modifié

sudo useradd -m -s /bin/bash nouvel_utilisateur
# useradd : crée un nouveau compte utilisateur
# -m : crée le répertoire personnel (/home/nouvel_utilisateur)
# -s /bin/bash : définit le shell par défaut de l'utilisateur
# nouvel_utilisateur : nom du compte à créer

sudo usermod -aG developpeurs utilisateur
# usermod : modifie un compte utilisateur existant
# -aG : ajoute l'utilisateur au groupe supplémentaire spécifié (append Group)
# developpeurs : groupe auquel ajouter l'utilisateur
# utilisateur : nom du compte à modifier

sudo userdel -r utilisateur
# userdel : supprime un compte utilisateur
# -r : supprime également le répertoire personnel et le spool de courrier
# utilisateur : nom du compte à supprimer
```

CONCEPT CLÉ

L'option `-aG` de `usermod` est cruciale : sans `-a`, l'option `-G` remplacerait tous les groupes supplémentaires de l'utilisateur par le seul groupe spécifié, lui retirant ses autres appartenances. Toujours utiliser `-aG` pour ajouter un groupe sans supprimer les existants.

PIÈGE

`userdel` sans l'option `-r` conserve le répertoire personnel de l'utilisateur, ce qui peut laisser des fichiers orphelins sur le système. Utiliser `-r` pour une suppression complète.

6.5. Gestion des groupes (groupadd, groupdel)

La commande `groupadd` crée un nouveau groupe, et `groupdel` supprime un groupe existant.

```
sudo groupadd developpeurs
# groupadd : crée un nouveau groupe système
# developpeurs : nom du groupe à créer

sudo groupdel developpeurs
# groupdel : supprime un groupe existant
# developpeurs : nom du groupe à supprimer
# Échoue si des utilisateurs ont ce groupe comme groupe principal
```

7. Planification et services (crontab, at, systemctl, journalctl) (25min)

L'automatisation des tâches et la gestion des services sont fondamentales pour l'administration système. `crontab` planifie des tâches récurrentes, `at` planifie des tâches ponctuelles, `systemctl` gère les services systemd, et `journalctl` consulte les journaux système.

7.1. Tâches récurrentes (crontab)

La commande `crontab` (cron table) permet de planifier l'exécution automatique de commandes à intervalles réguliers.

```
crontab -e
# crontab : édite la table de planification de l'utilisateur courant
# -e : édite (ouvre l'éditeur de texte configuré)
# Format : minute heure jour_mois mois jour_semaine commande

crontab -l
# crontab : liste les tâches planifiées de l'utilisateur courant
# -l : liste (list)
# Affiche toutes les entrées du crontab de l'utilisateur
```

Exemple d'entrée crontab :

```
0 2 * * * /home/eleve/sauvegarde.sh
# 0 2 * * * : tous les jours à 2h00 du matin
# /home/eleve/sauvegarde.sh : commande à exécuter
```

CONCEPT CLÉ

Le format cron comporte cinq champs temporels : minute (0-59), heure (0-23), jour du mois (1-31), mois (1-12), jour de la semaine (0-7, où 0 et 7 sont dimanche). L'astérisque `*` signifie "toutes les valeurs".

BON À SAVOIR

Chaque utilisateur a son propre crontab. Le fichier système `/etc/crontab` et le répertoire `/etc/cron.d/` contiennent les tâches planifiées au niveau système, nécessitant des privilèges root.

7.2. Tâches ponctuelles (at)

La commande `at` planifie l'exécution d'une commande à une heure précise, une seule fois.

```
echo "/home/eleve/sauvegarde.sh" | at 02:00
# at : planifie l'exécution d'une commande à une heure donnée
# echo ... | : tube qui envoie la commande vers l'entrée standard de at
# 02:00 : heure d'exécution (format HH:MM)
# at lit la commande depuis l'entrée standard et la planifie

atq
# at : liste les tâches planifiées en attente
# Aucun flag requis : affiche les tâches avec leur numéro d'identification

atrm 5
# at : supprime une tâche planifiée
# atrm : at remove, supprime la tâche numéro 5 de la file d'attente
```

7.3. Gestion des services (systemctl)

La commande `systemctl` contrôle les services et unités du gestionnaire systemd, qui est le système d'initialisation par défaut sur la plupart des distributions Linux modernes.

```
systemctl status sshd
# systemctl : contrôle les services et unités systemd
# status : affiche l'état détaillé d'un service
# sshd : nom du service (Secure Shell Daemon)
# Affiche si le service est actif, inactif ou en échec

systemctl start sshd
# systemctl : contrôle les services systemd
# start : démarre un service immédiatement
# sshd : nom du service à démarrer

systemctl enable sshd
# systemctl : contrôle les services systemd
# enable : active le démarrage automatique d'un service au boot
# sshd : nom du service à activer
```

CONCEPT CLÉ

`systemctl` distingue deux actions : démarrer un service immédiatement (`start`) et l'activer pour qu'il démarre automatiquement à chaque démarrage du système (`enable`). Un service peut être démarré sans être activé, et activé sans être démarré immédiatement.

> **BON À SAVOIR**
>
> `systemctl` nécessite des privilèges root pour la plupart des opérations (start, stop, enable, disable). L'option `status` peut être exécutée sans privilèges pour consulter l'état d'un service.

7.4. Consultation des journaux (journalctl)

La commande `journalctl` consulte les journaux du système collectés par systemd-journald, qui centralise les logs du noyau, des services et des applications.

```
journalctl -u sshd
# journalctl : consulte les journaux systemd
# -u : filtre par unité (service) spécifique
# sshd : nom du service dont on consulte les journaux

journalctl -n 50
# journalctl : consulte les journaux systemd
# -n : affiche les N dernières lignes
# 50 : nombre de lignes à afficher
# Utile pour un diagnostic rapide des événements récents

journalctl --since "1 hour ago"
# journalctl : consulte les journaux systemd
# --since : affiche les entrées à partir d'une date/heure donnée
# "1 hour ago" : expressions de temps relatives supportées
```

> **CONCEPT CLÉ**
>
> Contrairement aux fichiers de log traditionnels dans `/var/log/`, `journalctl` lit les journaux binaires gérés par systemd-journald. Il offre des fonctionnalités de filtrage avancées (par service, par priorité, par période) et un affichage paginé.

8. Accès distant et transfert de fichiers (ssh, scp, rsync, curl, wget) (25min)

L'administration système implique fréquemment de se connecter à des machines distantes et d'y transférer des fichiers. `ssh` permet l'accès distant sécurisé, `scp` et `rsync` transfèrent des fichiers, et `curl` et `wget` téléchargent des ressources depuis le réseau.

8.1. Connexion distante sécurisée (ssh)

La commande `ssh` (Secure Shell) permet d'ouvrir une session distante chiffrée sur une machine distante.

```
ssh utilisateur@192.168.1.10
# ssh : ouvre une session distante sécurisée (Secure Shell)
# utilisateur : nom d'utilisateur sur la machine distante
# 192.168.1.10 : adresse IP de la machine distante
# Le trafic est chiffré, protégeant le mot de passe et les données échangées

ssh utilisateur@192.168.1.10 "uptime"
# ssh : exécute une commande sur la machine distante sans ouvrir de session interactive
# "uptime" : commande à exécuter sur la machine distante
# Résultat : la sortie de la commande distante s'affiche localement
```

CONCEPT CLÉ

SSH utilise le chiffrement asymétrique pour l'authentification et le chiffrement symétrique pour le trafic de données. Il remplace les protocoles non sécurisés comme Telnet et rsh, qui transmettaient les mots de passe en clair.

CONTRE-MESURE

Toujours utiliser des clés SSH (générées avec `ssh-keygen`) plutôt que l'authentification par mot de passe, et désauthentifier l'accès root direct via SSH dans `/etc/ssh/sshd_config`.

8.2. Transfert de fichiers (scp, rsync)

La commande `scp` (secure copy) copie des fichiers entre machines via SSH. La commande `rsync` synchronise des répertoires de manière efficace, ne transférant que les différences.

```
scp fichier.txt utilisateur@192.168.1.10:/home/utilisateur/
# scp : copie sécurisée de fichiers entre machines (Secure Copy)
# fichier.txt : fichier source local
# utilisateur@192.168.1.10:/home/utilisateur/ : destination distante
# Le transfert est chiffré via SSH

rsync -avz donnees_fictives/ utilisateur@192.168.1.10:/home/utilisateur/sauvegarde/
# rsync : synchronise des répertoires de manière efficace
# -a : mode archive (préserve droits, dates, liens symboliques, etc.)
# -v : mode verbeux, affiche les fichiers transférés
# -z : compresse les données pendant le transfert
# donnees_fictives/ : répertoire source
# utilisateur@192.168.1.10:/home/utilisateur/sauvegarde/ : destination distante
# rsync ne transfère que les fichiers modifiés depuis la dernière synchronisation
```

CONCEPT CLÉ

`rsync` est plus efficace que `scp` pour les synchronisations répétées car il utilise un algorithme de delta-transfert : seuls les blocs modifiés sont envoyés, pas les fichiers entiers. Il peut aussi fonctionner en local sans SSH.

8.3. Téléchargement depuis le réseau (curl, wget)

La commande `curl` transfère des données depuis ou vers un serveur en supportant de nombreux protocoles (HTTP, HTTPS, FTP, etc.). La commande `wget` est un téléchargeur non interactif optimisé pour la récupération de fichiers.

```
curl -o page.html https://example.com
# curl : transfère des données depuis un serveur
# -o : écrit la sortie dans un fichier (output)
# page.html : nom du fichier de destination
# https://example.com : URL de la ressource à télécharger

wget https://example.com/fichier.zip
# wget : télécharge des fichiers depuis le réseau (non interactif)
# https://example.com/fichier.zip : URL du fichier à télécharger
# wget reprend automatiquement un téléchargement interrompu avec -c
```

CONCEPT CLÉ

`curl` est un outil polyvalent adapté aux tests d'API et aux transferts complexes (entêtes, authentification, cookies). `wget` est spécialisé dans le téléchargement de fichiers, avec reprise automatique et téléchargement récursif de sites web.

BON À SAVOIR

`curl -I` affiche uniquement les en-têtes HTTP d'une réponse, utile pour vérifier qu'un serveur est en ligne sans télécharger le contenu complet.

Fig 7 — Gestion des utilisateurs, services et accès distant

Diagramme		
B	useradd, usermod	
B	systemctl	
B	journalctl	
Administrateur	ssh	
F	scp, rsync	
Administrateur	crontab	H

Figure — Diagramme Mermaid

Lab 03 — Premier script exécutable (45min)

Environnement

Élément	Valeur
Poste	Linux (Bash)
Fichier	mon_premier_script.sh

Exercices

Exercice 10 — Créer un script avec shebang (15min)

Créez un fichier `backup.sh` contenant :

```
#!/bin/bash
# #!/bin/bash : shebang, indique d'utiliser l'interpréteur Bash
echo "Début de la sauvegarde"
# echo : affiche le message de début
date
# date : affiche la date et l'heure système
echo "Sauvegarde terminée"
# echo : affiche le message de fin
```

- Le fichier backup.sh est créé avec un éditeur de texte
- La première ligne contient le shebang #!/bin/bash

Exercice 11 — Rendre le script exécutable (15min)

```
chmod 755 backup.sh
# chmod : modifie les droits d'accès au fichier
# 755 : lecture/écriture/exécution pour le propriétaire, lecture/exécution pour le groupe et les autres
# backup.sh : fichier cible à rendre exécutable
chmod u+x backup.sh
# chmod : modifie les droits d'accès au fichier
# u+x : ajoute la permission d'exécution pour le propriétaire
# backup.sh : fichier cible
ls -l backup.sh
# ls -l : affiche les droits détaillés du fichier
# backup.sh : fichier cible
```

- Le script est rendu exécutable avec chmod 755
- La permission d'exécution est ajoutée avec chmod u+x
- La commande ls -l confirme la présence du droit d'exécution (x)

Exercice 12 — Exécuter le script de trois façons (15min)

```
./backup.sh
# ./ : exécute le script situé dans le répertoire courant
# backup.sh : nom du script
bash backup.sh
# bash : invoque l'interpréteur Bash pour exécuter le fichier
# backup.sh : fichier de script à exécuter
source backup.sh
# source : exécute le script dans le shell courant sans processus enfant
# backup.sh : fichier de script à exécuter
```

- Le script s'exécute avec ./backup.sh
- Le script s'exécute avec bash backup.sh
- Le script s'exécute avec source backup.sh

Exercice 13 — Vérifier le code de retour (10min)

```
./backup.sh
# ./backup.sh : exécute le script situé dans le répertoire courant
echo $?
# $? : variable spéciale contenant le code de retour de la dernière commande
# 0 indique un succès, toute valeur non-zéro indique une erreur
```

- Quelle valeur affiche `$?` après un succès ?
- Quelle valeur indiquerait une erreur ?

- Le code de retour 0 est observé après succès
- La signification des codes non-zéro est comprise

Solution

Solution — Exercices 10, 11, 12 et 13

Contenu du script `backup.sh` :

```
#!/bin/bash
echo "Début de la sauvegarde"
date
echo "Sauvegarde terminée"
```

Commandes d'exécution :

```
chmod 755 backup.sh
chmod u+x backup.sh
ls -l backup.sh
./backup.sh
bash backup.sh
source backup.sh
./backup.sh
echo $?
```

Résultat attendu : le script affiche « Début de la sauvegarde », la date et l'heure, puis « Sauvegarde terminée ». La commande `echo $?` affiche `0` après un succès.

Exercices supplémentaires — Archivage, compression et gestion des données fictives (donnees_fictives/)

Exercice 13b — Archiver et compresser les fichiers de données (15min)

```
tar -czvf donnees_archive.tar.gz donnees_fictives/
# tar : crée une archive compressée avec gzip
# -c : crée une nouvelle archive
# -z : compresse avec gzip
# -v : mode verbeux, affiche les fichiers traités
# -f : spécifie le nom du fichier archive
# donnees_archive.tar.gz : nom de l'archive compressée à créer
# donnees_fictives/ : répertoire à archiver et compresser

tar -tzvf donnees_archive.tar.gz
# tar : liste le contenu d'une archive compressée sans l'extraire
# -t : liste le contenu de l'archive
# -z : l'archive est compressée avec gzip
# -v : mode verbeux
# -f : spécifie le fichier archive
# donnees_archive.tar.gz : archive à lister
```

- L'archive donnees_archive.tar.gz est créée
- Le contenu de l'archive s'affiche avec tar -tzvf

Exercice 14b — Vérifier l'existence des fichiers de configuration avec test (15min)

Le fichier `donnees_fictives/services.txt` contient une liste de chemins de fichiers de configuration système. Vérifiez quels fichiers existent sur le système.

```
while read FICHIER; do
    # while : lit chaque ligne du fichier services.txt
    # read : lit une ligne depuis l'entrée standard
    # FICHIER : variable contenant le chemin lu
    if [ -f "$FICHIER" ]; then
        # -f : teste si le fichier existe et est un fichier régulier
        echo "$FICHIER : présent"
    else
        echo "$FICHIER : absent"
    fi
done < donnees_fictives/services.txt
# done : termine la boucle while
# < donnees_fictives/services.txt : redirige l'entrée standard depuis le fichier
```

- Chaque fichier listé dans services.txt est testé
- Le message "présent" ou "absent" s'affiche pour chaque fichier

Solution

Solution — Exercices 13b et 14b

```
# Exercice 13b
tar -czvf donnees_archive.tar.gz donnees_fictives/
tar -tzvf donnees_archive.tar.gz

# Exercice 14b
while read FICHIER; do
    if [ -f "$FICHIER" ]; then
        echo "$FICHIER : présent"
    else
        echo "$FICHIER : absent"
    fi
done < donnees_fictives/services.txt
```

Résultat attendu pour l'exercice 13b : l'archive est créée avec les 6 fichiers de donnees_fictives/, puis tar -tzvf liste leur contenu.

Résultat attendu pour l'exercice 14b : pour chaque chemin dans services.txt, un message indique si le fichier est présent ou absent sur le système (ex. `/etc/passwd : présent`, `/etc/ssh/sshd_config : présent` ou absent selon le système).

Démonstration commentée

Le shebang `#!/bin/bash` est lu par le noyau au moment de l'exécution directe (`./backup.sh`). Il désigne l'interpréteur qui traitera le reste du fichier. La commande `chmod 755` accorde les droits de lecture, écriture et exécution au propriétaire, et de lecture et exécution au groupe et aux autres. L'exécution avec `bash backup.sh` contourne le besoin de droits d'exécution car c'est Bash lui-même qui lit le fichier. La commande `source backup.sh` exécute le script dans le shell courant, ce qui rend ses variables disponibles après l'exécution. Le code de retour `$?` vaut `0` lorsque le script se termine sans erreur.

Correction et résolution pas à pas

- Créez `backup.sh` avec un éditeur de texte (nano ou vim).
- Ajoutez `#!/bin/bash` en première ligne.

- Ajoutez les commandes `echo` et `date`.
- Enregistrez le fichier.
- Rendez-le exécutable avec `chmod 755 backup.sh`.
- Vérifiez les droits avec `ls -l backup.sh`.
- Lancez le script avec `./backup.sh`, puis testez `bash backup.sh` et `source backup.sh`.
- Vérifiez le code de retour avec `echo $?` après l'exécution.

Bonnes pratiques

Pratique	Description
Shebang toujours présent	Commencer chaque script par `#!/bin/bash` pour garantir l'interpréteur
Droits restrictifs	Utiliser 755 plutôt que 777 pour limiter l'écriture au propriétaire
Vérification du code de retour	Tester `$?` après l'exécution d'une commande critique

BONNES PRATIQUES

Ne rendez jamais un script writable par tous (777) ; 755 suffit pour l'exécution tout en protégeant contre la modification non autorisée. Vérifiez systématiquement `$?` après une commande dont le succès est nécessaire à la suite du script.

Script de correction : `backup.sh`

Exercices — Chapitre 3

Exercice 14 — Créer un script qui affiche des informations système (10min)

Créez un fichier `infos.sh` contenant :

```
#!/bin/bash
# #!/bin/bash : shebang, indique d'utiliser l'interpréteur Bash
echo "=== Informations système ==="
# echo : affiche un séparateur de section
echo "Utilisateur : $(whoami)"
# $(whoami) : substitution de commande, insère le nom de l'utilisateur
echo "Répertoire : $(pwd)"
# $(pwd) : substitution de commande, insère le chemin du répertoire courant
echo "Date : $(date)"
# $(date) : substitution de commande, insère la date et l'heure
echo "=== Fin ==="
```

- Rendez le script exécutable.
- Exécutez-le et observez le résultat.
- Vérifiez le code de retour avec `echo $?`.

Solution

Solution — Exercice 14

```
chmod 755 infos.sh
./infos.sh
echo $?
```

Résultat attendu :

```
=== Informations système ===
Utilisateur : eleve
Répertoire : /home/eleve
Date : lun 8 nov 2026 14:30:00 CET
=== Fin ===
0
```

Le code de retour `0` confirme que le script s'est exécuté sans erreur.

- Le script infos.sh est créé avec le shebang
- Le script est rendu exécutable
- L'exécution affiche les informations système
- Le code de retour 0 est observé

CE QU'ON A RETENU

Un script Bash commence par un shebang `#!/bin/bash` qui désigne l'interpréteur. La commande `chmod 755` (ou `chmod u+x`) rend le script exécutable. Un script peut s'exécuter de trois façons : `./script.sh` (direct, nécessite droits + shebang), `bash script.sh` (via l'interpréteur, shebang ignoré) et `source script.sh` (dans le shell courant, variables conservées). Le code de retour `$?` vaut `0` en cas de succès et une valeur non-zéro en cas d'erreur.

Chapitre 4 — Structures de contrôle : conditions (if, case), boucles (for, while) (14h45-16h00)

Objectifs

- Utiliser les conditions `if / then / else / fi` avec `[ condition ]`
- Maîtriser les opérateurs de test : `-f`, `-d`, `-eq`, `-ne`, `-gt`, `-lt`, `-z`, `-n`
- Utiliser `case / esac` pour la correspondance de motifs
- Écrire des boucles `for` et `while`
- Utiliser `break` et `continue` pour contrôler les boucles
- Comprendre le rôle des codes de sortie dans les conditionnels

Contenu théorique (1h00)

1. Condition if / then / else / fi (30min)

La structure conditionnelle `if` permet d'exécuter un bloc de commandes uniquement si une condition est vraie. Selon Linux Command Line and Shell Scripting Bible, 5e (Blum & Bresnahan), page 326, les crochets

`[ ]` sont une commande de test qui est un synonyme de la commande `test`. On peut enclore une condition de test dans des crochets dans une instruction `if`-`then`.

La syntaxe générale est :

```
if [ condition ]; then
    commandes_si_vrai
else
    commandes_si_faux
fi
```

Les crochets `[ ]` doivent être entourés d'espaces à l'intérieur (`[ "$AGE" -ge 18 ]`, pas `["$AGE" -ge 18]`). Les variables doivent être protégées par des guillemets doubles pour éviter les erreurs si la variable est vide ou contient des espaces.

Les opérateurs de test les plus courants sont :

Opérateur	Signification	Type de comparaison
`-f`	Le fichier existe et est un fichier régulier	Fichier
`-d`	Le répertoire existe	Répertoire
`-eq`	Égal à	Numérique
`-ne`	Différent de	Numérique
`-gt`	Strictement supérieur à	Numérique
`-lt`	Strictement inférieur à	Numérique
`-z`	La chaîne est vide	Chaîne
`-n`	La chaîne est non vide	Chaîne

```
if [ "$AGE" -ge 18 ]; then
    echo "Vous êtes majeur."
else
    echo "Vous êtes mineur."
fi
# if : débute une condition conditionnelle
# [ "$AGE" -ge 18 ] : teste si la variable AGE est supérieure ou égale à 18
# -ge : opérateur de comparaison numérique (greater or equal)
# then : début du bloc exécuté si la condition est vraie
# else : bloc exécuté si la condition est fausse
# fi : termine la condition (mot-clé if à l'envers)
```

CONCEPT CLÉ

Les crochets `[ ]` sont une commande de test. Il est important de mettre des espaces à l'intérieur des crochets et de protéger les variables avec des guillemets pour éviter les erreurs de syntaxe.

CONTRE-MESURE

Testez toujours l'existence d'un fichier avec `-f` avant de tenter de le lire, et l'existence d'un répertoire avec `-d` avant d'y accéder, afin d'éviter les erreurs d'exécution.

2. Condition case / esac (20min)

La structure `case` permet de comparer une variable contre plusieurs motifs (patterns) et d'exécuter le bloc associé au premier motif qui correspond. Selon Linux Command Line and Shell Scripting Bible, 5e (Blum & Bresnahan), page 325, la commande `case` compare la variable spécifiée contre les différents motifs et exécute les commandes associées au premier motif correspondant.

```
case "$JOUR" in
    lundi) echo "Début de la semaine de travail" ;;
    mardi) echo "Deuxième jour de travail" ;;
    mercredi) echo "Milieu de semaine" ;;
    jeudi) echo "Quatrième jour de travail" ;;
    vendredi) echo "Dernier jour de travail" ;;
    samedi|dimanche) echo "Week-end" ;;
    *) echo "Jour non reconnu" ;;
esac
# case : sélectionne une action selon la valeur de la variable JOUR
# ) : sépare le motif du bloc d'actions
# ;; : termine chaque bloc d'actions
# | : sépare plusieurs motifs pour un même bloc (samedi ou dimanche)
# *) : motif par défaut, correspond à tous les autres cas non listés
# esac : termine la structure case (mot-clé case à l'envers)
```

Le motif `*)` agit comme une clause par défaut (équivalent du `default` dans d'autres langages). Il est recommandé de toujours inclure un motif par défaut pour gérer les valeurs imprévues.

CONCEPT CLÉ

La structure `case` est plus lisible qu'une suite de `if`/`elif` lorsque l'on compare une même variable contre plusieurs valeurs discrètes.

BON À SAVOIR

Le symbole `|` dans un motif de `case` permet de regrouper plusieurs valeurs qui déclenchent la même action (par exemple `samedi|dimanche`).

3. Boucles for et while (30min)

La boucle `for` itère sur une liste de valeurs. Selon Linux Command Line and Shell Scripting Bible, 5e (Blum & Bresnahan), page 347, la boucle `for` devrait normalement itérer à travers toutes les valeurs spécifiées dans la liste.

```
for JOUR in lundi mardi mercredi jeudi vendredi; do
    echo "Jour : $JOUR"
done
# for : itère sur chaque valeur de la liste
# JOUR : variable de boucle qui prend successivement chaque valeur
# in : sépare la variable de boucle de la liste de valeurs
# do : début du corps de la boucle
# done : termine la boucle
```

La boucle `while` répète un bloc de commandes tant qu'une condition est vraie. Selon Linux Command Line and Shell Scripting Bible, 5e (Blum & Bresnahan), page 354, la boucle `while` utilise les codes de

sortie, comme la condition `if`. La boucle continue tant que la commande de test retourne un code de sortie 0.

```
COMPTEUR=1
# COMPTEUR=1 : initialise le compteur à 1
while [ $COMPTEUR -le 5 ]; do
    echo "Itération $COMPTEUR"
    # echo : affiche la valeur actuelle du compteur
    COMPTEUR=$((COMPTEUR + 1))
    # $((...)) : évaluation arithmétique, incrémente le compteur de 1
done
# while : répète tant que la condition est vraie
# [ $COMPTEUR -le 5 ] : teste si le compteur est inférieur ou égal à 5
# -le : lower or equal (inférieur ou égal)
# do : début du corps de la boucle
# done : termine la boucle
```

Les commandes `break` et `continue` permettent de contrôler le flux d'une boucle :

- `break` : sort immédiatement de la boucle
- `continue` : passe à l'itération suivante sans exécuter le reste du corps de la boucle

PIÈGE

L'arithmétique Bash utilise `$(( ))` ; oublier les parenthèses doubles ferait afficher le texte littéralement par le shell.

CONCEPT CLÉ

Les codes de sortie jouent un rôle central dans les structures de contrôle : selon *How Linux Works, 2nd Edition* (Ward, Brian), page 254, le code de sortie de la dernière commande détermine la façon dont le shell traite le conditionnel. Une boucle `while` continue tant que sa commande de test retourne 0.

Fig 4 — Flux de contrôle conditionnel et itératif

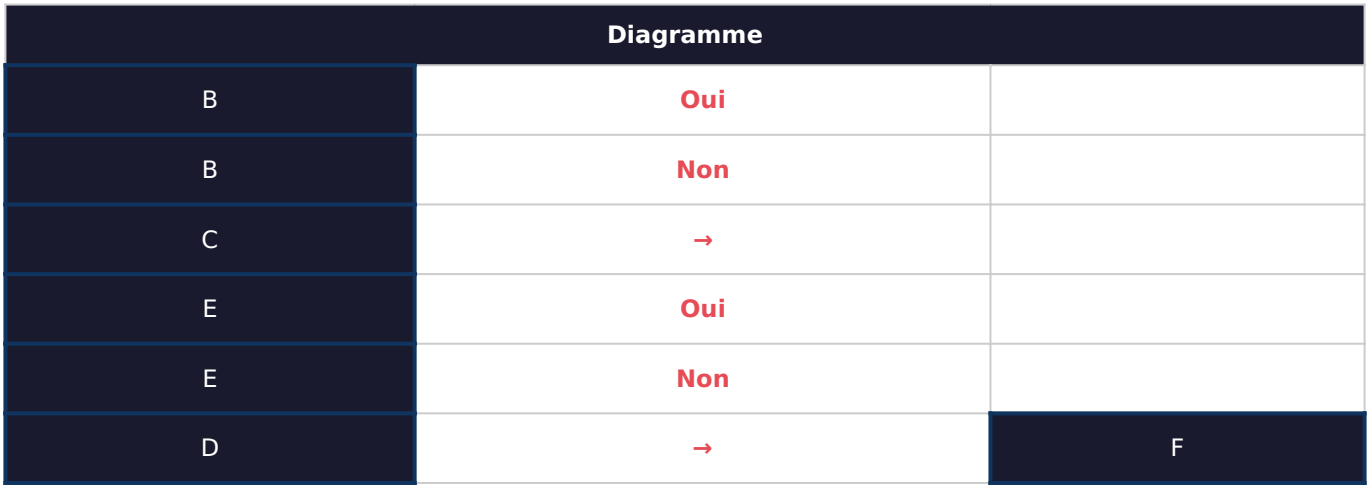


Figure — Diagramme Mermaid

4. Expressions régulières et tests avancés (25min)

Les expressions régulières (regex) permettent de définir des motifs de recherche complexes dans du texte. Bash les utilise dans plusieurs contextes : `grep` pour la recherche, `sed` et `awk` pour la transformation, et les tests conditionnels avec `[[ ]]` pour la correspondance de motifs.

4.1. Notions de base des expressions régulières

Une expression régulière est une suite de caractères qui définit un motif de recherche. Les caractères les plus courants sont :

Symbole	Signification	Exemple	Correspondance
`.`	N'importe quel caractère unique	`c.t`	chat, cot, cut
`*`	Zéro ou plusieurs occurrences du caractère précédent	`ab*`	a, ab, abb, abbb
`+`	Une ou plusieurs occurrences du caractère précédent	`ab+`	ab, abb, abbb
`?`	Zéro ou une occurrence du caractère précédent	`colou?r`	color, colour
`^`	Début de ligne	`^ERROR`	lignes commençant par ERROR
`$`	Fin de ligne	`erreur$`	lignes se terminant par erreur
`[...]`	Un des caractères entre crochets	`[aeiou]`	une voyelle
`[^...]`	Aucun des caractères entre crochets	`[^0-9]`	tout sauf un chiffre

4.2. Utilisation de grep avec des expressions régulières

```
grep "^ERROR" donnees_fictives/system.log
# grep : recherche des lignes correspondant à un motif
# "^ERROR" : expression régulière, ^ signifie début de ligne
# donnees_fictives/system.log : fichier de journal système
# Affiche uniquement les lignes commençant par ERROR

grep -E "ERROR|WARN" donnees_fictives/system.log
# grep -E : expressions régulières étendues (extended regex)
# "ERROR|WARN" : motif correspondant à ERROR ou WARN (| = ou)
# donnees_fictives/system.log : fichier cible
# Affiche les lignes contenant ERROR ou WARN

grep -E "[0-9]+" donnees_fictives/system.log
# grep -E : expressions régulières étendues
# "[0-9]+" : une ou plusieurs chiffres
# donnees_fictives/system.log : fichier cible
# Affiche les lignes contenant au moins un chiffre
```

CONCEPT CLÉ

Par défaut, `grep` utilise des expressions régulières de base (BRE). L'option `-E` active les expressions régulières étendues (ERE), qui simplifient l'écriture des motifs (notamment `+`, `?`, `|` sans échappement).

BON À SAVOIR

Toujours protéger l'expression régulière entre guillemets doubles pour éviter que le shell n'interprète les caractères spéciaux comme `*`, `?`, `$`, `[`, `]`.

4.3. Tests conditionnels avancés avec [[ ]]

Bash propose une syntaxe de test étendue avec `[[ ]]` qui offre des fonctionnalités supérieures aux crochets simples `[ ]`, notamment la correspondance de motifs avec `=~` (expression régulière) et les comparaisons lexicographiques.

```
LOG_LINE="2026-11-08 10:10:05 ERROR httpd[2340] server reached MaxRequestWorkers"
if [[ $LOG_LINE =~ ERROR ]]; then
    # [[ ]] : test conditionnel étendu de Bash
    # $LOG_LINE : variable contenant la ligne de journal
    # =~ : opérateur de correspondance avec expression régulière
    # ERROR : motif à rechercher dans la ligne
    echo "Ligne d'erreur détectée"
fi

if [[ $LOG_LINE =~ ^[0-9]{4}-[0-9]{2}-[0-9]{2} ]]; then
    # ^[0-9]{4}-[0-9]{2}-[0-9]{2} : date au format AAAA-MM-JJ en début de ligne
    # {4} : exactement 4 occurrences, {2} : exactement 2 occurrences
    echo "Format de date valide détecté"
fi
```

CONCEPT CLÉ

`[[ ]]` est une construction propre à Bash (non portable vers `sh`). Elle offre des avantages par rapport à `[ ]` : pas besoin de protéger les variables avec des guillemets (les espaces sont gérés en interne), support des expressions régulières avec `=~`, et opérateurs logiques `&&` et `||` au lieu de `-a` et `-o`.

CONTRE-MESURE

`[[ ]]` n'est pas disponible dans les scripts POSIX (`sh`). Pour un script portable, utiliser `[ ]` avec la commande `test` ou `expr` pour les expressions régulières.

4.4. Tableau récapitulatif des commandes de traitement de texte

Commande	Rôle principal	Options courantes	Exemple d'usage
cat	Afficher le contenu d'un fichier	-n (numéroter les lignes)	`cat fichier.txt`
less	Navigation paginée dans un fichier	/ (recherche), q (quitter)	`less fichier.log`
head	Afficher les premières lignes	-n (nombre de lignes)	`head -20 fichier.txt`
tail	Afficher les dernières lignes	-f (suivi en temps réel)	`tail -f fichier.log`
wc	Compter lignes, mots, octets	-l, -w, -c	`wc -l fichier.txt`
find	Rechercher des fichiers	-name, -type, -mtime	`find . -name "*.log"`
grep	Rechercher un motif dans du texte	-i, -c, -n, -E	`grep "ERROR" fichier.log`
cut	Extraire des colonnes	-d (délimiteur), -f (champs)	`cut -d',' -f1 fichier.csv`
sort	Trier des lignes	-r (inversé), -n (numérique)	`sort fichier.txt`
uniq	Supprimer les doublons consécutifs	-c (compter)	`sort f	uniq -c`
sed	Éditeur de flux, substitutions	s/old/new/g	`sed 's/foo/bar/g' f`
awk	Traitement de colonnes, calculs	-F (séparateur)	`awk -F',' '{print $1}' f`
diff	Comparer deux fichiers	-u (format unifié)	`diff f1 f2`
patch	Appliquer un fichier de différences	-p1 (niveau de répertoire)	`patch < correctif.diff`

Fig 8 — Flux de traitement avec expressions régulières

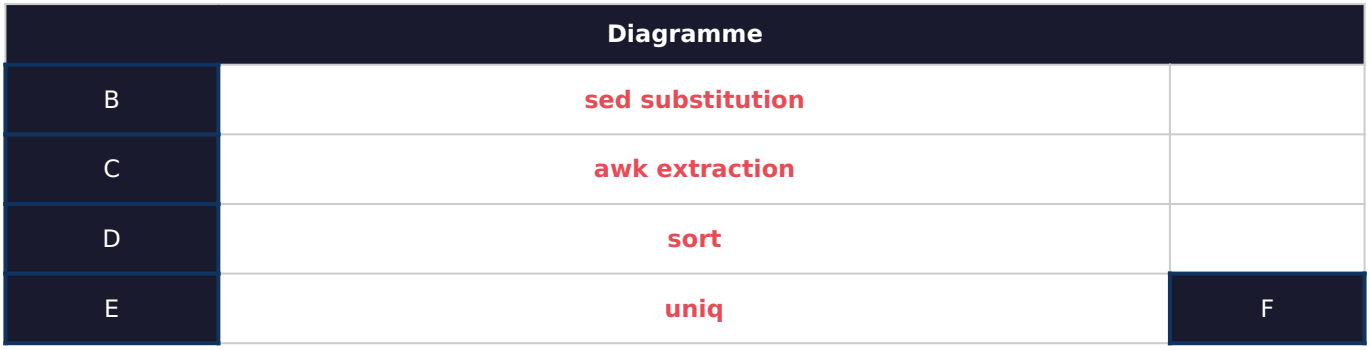


Figure — Diagramme Mermaid

Lab 04 — Conditions et boucles (45min)

Environnement

Élément	Valeur
Poste	Linux (Bash)
Fichier	controle.sh

Exercices

Exercice 15 — Vérifier l'existence d'un fichier avec if (15min)

Créez un script `verif_fichier.sh` contenant :

```
#!/bin/bash
# #!/bin/bash : shebang, indique d'utiliser l'interpréteur Bash
FICHIER="/etc/hostname"
# FICHIER="/etc/hostname" : assigne le chemin du fichier à vérifier
if [ -f "$FICHIER" ]; then
    # -f : teste si le fichier existe et est un fichier régulier
    # "$FICHIER" : variable protégée par des guillemets doubles
    echo "Le fichier $FICHIER existe."
else
    echo "Le fichier $FICHIER n'existe pas."
fi
# if : débute la condition
# then : début du bloc si la condition est vraie
# else : bloc si la condition est fausse
# fi : termine la condition
```

- Le script teste l'existence du fichier /etc/hostname
- Le message approprié s'affiche selon le résultat du test

Exercice 16 — Menu de sélection avec case (15min)

Créez un script `menu.sh` contenant :

```
#!/bin/bash
# #!/bin/bash : shebang, indique d'utiliser l'interpréteur Bash
read -p "Entrez un jour de la semaine : " JOUR
# read : lit une saisie clavier
# -p : affiche une invite avant la saisie
# JOUR : variable qui stocke la saisie utilisateur
case "$JOUR" in
    lundi) echo "Début de la semaine de travail" ;;
    vendredi) echo "Dernier jour de travail avant le week-end" ;;
    samedi|dimanche) echo "C'est le week-end !" ;;
    *) echo "Jour de la semaine ordinaire" ;;
esac
# case : sélectionne une action selon la valeur de JOUR
# read -p : lit une saisie clavier avec invite
# ;; : termine chaque bloc d'actions
# *) : motif par défaut pour tous les autres cas
# esac : termine la structure case
```

- Le script lit une saisie utilisateur avec read -p
- La structure case correspond la saisie à un motif
- Le message adapté s'affiche selon le jour saisi

Exercice 17 — Boucle for sur une liste de fichiers (15min)

```
for FICHIER in /etc/hostname /etc/passwd /etc/group; do
    # for : itère sur chaque fichier de la liste
    # FICHIER : variable de boucle contenant le chemin courant
    if [ -f "$FICHIER" ]; then
        # -f : teste si le fichier existe et est un fichier régulier
        echo "$FICHIER : présent"
    else
        echo "$FICHIER : absent"
    fi
done
# done : termine la boucle for
```

- La boucle itère sur chaque fichier de la liste
- Le test -f vérifie l'existence de chaque fichier
- Le résultat s'affiche pour chaque fichier

Exercice 18 — Boucle while avec compteur (15min)

```
COMPTEUR=1
# COMPTEUR=1 : initialise le compteur à 1
while [ $COMPTEUR -le 5 ]; do
    # while : répète tant que la condition est vraie
    # -le : inférieur ou égal (lower or equal)
    echo "Compteur : $COMPTEUR"
    COMPTEUR=$((COMPTEUR + 1))
    # $((...)) : évaluation arithmétique, incrémente le compteur
done
# done : termine la boucle while
```

  - Combien de fois la boucle s'exécute-t-elle ?
  - Que se passerait-il sans l'incrémentation de COMPTEUR ?

- La boucle s'exécute 5 fois puis s'arrête
- Le risque de boucle infinie est identifié

Solution

Solution — Exercices 15, 16, 17 et 18

Script `verif_fichier.sh` :

```
#!/bin/bash
FICHIER="/etc/hostname"
if [ -f "$FICHIER" ]; then
    echo "Le fichier $FICHIER existe."
else
    echo "Le fichier $FICHIER n'existe pas."
fi
```

Script `menu.sh` :

```
#!/bin/bash
read -p "Entrez un jour de la semaine : " JOUR
case "$JOUR" in
    lundi) echo "Début de la semaine de travail" ;;
    vendredi) echo "Dernier jour de travail avant le week-end" ;;
    samedi|dimanche) echo "C'est le week-end !" ;;
    *) echo "Jour de la semaine ordinaire" ;;
esac
```

Boucle for sur les fichiers :

```
for FICHIER in /etc/hostname /etc/passwd /etc/group; do
    if [ -f "$FICHIER" ]; then
        echo "$FICHIER : présent"
    else
        echo "$FICHIER : absent"
    fi
done
```

Boucle while avec compteur :

```
COMPTEUR=1
while [ $COMPTEUR -le 5 ]; do
    echo "Compteur : $COMPTEUR"
    COMPTEUR=$((COMPTEUR + 1))
done
```

Réponses aux questions de l'exercice 18 :

- La boucle s'exécute 5 fois (compteur de 1 à 5).
- Sans l'incrémentation de COMPTEUR, la condition resterait vraie indéfiniment : boucle infinie.

Exercices supplémentaires — Conditions avancées et expressions régulières sur les données fictives (donnees_fictives/)

Exercice 19b — Filtrer les erreurs et avertissements dans system.log (15min)

```
grep -E "ERROR|WARN" donnees_fictives/system.log
# grep -E : expressions régulières étendues
# "ERROR|WARN" : motif correspondant à ERROR ou WARN
# donnees_fictives/system.log : fichier de journal système
# Affiche les lignes contenant ERROR ou WARN

grep -c -E "ERROR|WARN" donnees_fictives/system.log
# grep -c : compte le nombre de lignes correspondantes
# -E : expressions régulières étendues
# "ERROR|WARN" : motif ERROR ou WARN
# donnees_fictives/system.log : fichier cible
```

- Les lignes contenant ERROR ou WARN s'affichent
- Le nombre total de lignes d'erreur et d'avertissement s'affiche

Exercice 20b — Extraire les adresses IP du fichier reseau.txt (15min)

Le fichier `donnees_fictives/reseau.txt` contient des entrées de journal web avec des adresses IP. Extrayez les adresses IP uniques.

```
cut -d' ' -f1 donnees_fictives/reseau.txt
# cut : extrait des sections de chaque ligne
# -d' ' : définit l'espace comme délimiteur de champs
# -f1 : sélectionne le premier champ (adresse IP)
# donnees_fictives/reseau.txt : fichier de journal web

cut -d' ' -f1 donnees_fictives/reseau.txt | sort | uniq
# cut : extrait les adresses IP
# | : tube qui redirige la sortie vers la commande suivante
# sort : trie les adresses IP pour regrouper les doublons
# uniq : supprime les doublons consécutifs
# Résultat : liste des adresses IP uniques présentes dans le fichier
```

- La première colonne (adresse IP) est extraite
- Les doublons sont supprimés après sort | uniq
- La liste des adresses IP uniques s'affiche

Exercice 21b — Détecter les lignes de journal avec une date au format valide (15min)

```
while read LIGNE; do
    # while : lit chaque ligne du fichier system.log
    # read : lit une ligne depuis l'entrée standard
    # LIGNE : variable contenant la ligne lue
    if [[ $LIGNE =~ ^[0-9]{4}-[0-9]{2}-[0-9]{2} ]]; then
        # [[ ]] : test conditionnel étendu de Bash
        # =~ : opérateur de correspondance avec expression régulière
        # ^[0-9]{4}-[0-9]{2}-[0-9]{2} : date AAAA-MM-JJ en début de ligne
        echo "Date valide : $LIGNE"
    fi
done < donnees_fictives/system.log
# done : termine la boucle while
# < donnees_fictives/system.log : redirige l'entrée standard depuis le fichier
```

- Chaque ligne de system.log est lue et testée
- Les lignes commençant par une date valide sont affichées
- L'expression régulière correspond au format AAAA-MM-JJ

Solution

Solution — Exercices 19b, 20b et 21b

```
# Exercice 19b
grep -E "ERROR|WARN" donnees_fictives/system.log
grep -c -E "ERROR|WARN" donnees_fictives/system.log

# Exercice 20b
cut -d' ' -f1 donnees_fictives/reseau.txt
cut -d' ' -f1 donnees_fictives/reseau.txt | sort | uniq

# Exercice 21b
while read LIGNE; do
    if [[ $LIGNE =~ ^[0-9]{4}-[0-9]{2}-[0-9]{2} ]]; then
        echo "Date valide : $LIGNE"
    fi
done < donnees_fictives/system.log
```

Résultat attendu pour l'exercice 19b : 11 lignes contenant ERROR ou WARN (5 ERROR + 6 WARN dans system.log).

Résultat attendu pour l'exercice 20b : 7 adresses IP uniques (192.168.1.10, 10.0.0.5, 192.168.1.50, 172.16.0.25, 192.168.1.100, 192.168.1.20, 10.0.0.99).

Résultat attendu pour l'exercice 21b : toutes les 23 lignes de system.log commencent par une date au format AAAA-MM-JJ, donc toutes sont affichées.

Démonstration commentée

La commande `read -p` suspend l'exécution et attend une saisie utilisateur, stockée dans la variable spécifiée. La structure `if [ -f "$FICHIER" ]` teste l'existence d'un fichier régulier et affiche un message adapté. La structure `case` compare la saisie utilisateur contre plusieurs motifs et exécute le premier bloc correspondant. La boucle `for` itère sur chaque élément de la liste fournie, exécutant le corps pour chaque valeur. La boucle `while` répète le bloc tant que la condition est vraie, à condition que le compteur soit incrémenté pour éviter la boucle infinie.

Correction et résolution pas à pas

- Créez `verif_fichier.sh` avec le shebang et un test `if [ -f ... ]`.
- Testez avec un fichier existant (`/etc/hostname`) puis un fichier inexistant.
- Créez `menu.sh` avec `read -p` et une structure `case` incluant un motif `*)` par défaut.
- Testez avec différents jours (lundi, vendredi, samedi, mardi).
- Écrivez une boucle `for` sur une liste de fichiers système avec un test `-f` à l'intérieur.
- Écrivez une boucle `while` avec un compteur initialisé à 1, incrémenté avec `$((COMPTEUR + 1))`, et condition `-le 5`.
- Rendez chaque script exécutable avec `chmod 755` et testez-les.

Bonnes pratiques

Pratique	Description
Protéger les variables	Toujours utiliser `"$VAR"` dans les tests `[ ]`
Indentation claire	Indenter les blocs if, for, while pour la lisibilité
Motif par défaut	Toujours inclure `*)` dans une structure case
Incrémentation	S'assurer que toute boucle while modifie sa variable de condition

BONNES PRATIQUES

Protégez systématiquement les variables entre guillemets dans les tests `[ ]` pour éviter les erreurs si la variable est vide ou contient des espaces. Toujours inclure un motif `*)` dans une structure `case` pour gérer les valeurs imprévues.

Script de correction : `controle.sh`

Exercices — Chapitre 4

Exercice 19 — Script combinant if et boucle for (10min)

Écrivez un script qui parcourt une liste de répertoires et affiche pour chacun s'il existe ou non :

```
for REP in /home /tmp /var/log /nonexistent; do
    # for : itère sur chaque répertoire de la liste
    # REP : variable de boucle contenant le chemin courant
    if [ -d "$REP" ]; then
        # -d : teste si le répertoire existe
        echo "$REP : répertoire présent"
    else
        echo "$REP : répertoire absent"
    fi
done
# done : termine la boucle for
```

- Quel opérateur de test utiliser pour vérifier un répertoire ?
- Que se passe-t-il pour le répertoire `/nonexistent` ?

Solution

Solution — Exercice 19

L'opérateur de test `-d` vérifie l'existence d'un répertoire. Pour `/nonexistent`, le test échoue et le message « répertoire absent » s'affiche, car ce répertoire n'existe pas sur le système.

- L'opérateur -d est utilisé pour tester les répertoires
- Le script gère correctement les répertoires existants et inexistants

Exercice 20 — Boucle while avec break (10min)

Écrivez une boucle while qui affiche les nombres de 1 à 10 mais s'arrête à 5 en utilisant `break` :

```
COMPTEUR=1
# COMPTEUR=1 : initialise le compteur à 1
while [ $COMPTEUR -le 10 ]; do
    # while : répète tant que le compteur est inférieur ou égal à 10
    if [ $COMPTEUR -eq 5 ]; then
        # -eq : teste l'égalité numérique
        break
        # break : sort immédiatement de la boucle
    fi
    echo "Compteur : $COMPTEUR"
    COMPTEUR=$((COMPTEUR + 1))
    # $((...)) : évaluation arithmétique, incrémente le compteur
done
# done : termine la boucle while
```

- Quel est le dernier nombre affiché ?
- Pourquoi la boucle s'arrête-t-elle à ce point ?

Solution

Solution — Exercice 20

Le dernier nombre affiché est 4. Lorsque COMPTEUR vaut 5, la condition `[ $COMPTEUR -eq 5 ]` est vraie, la commande `break` est exécutée et la boucle se termine immédiatement. Le nombre 5 n'est donc jamais affiché.

- La commande break est comprise et utilisée correctement

- Le dernier nombre affiché est identifié (4)

CE QU'ON A RETENU

La structure `if / then / else / fi` avec `[ condition ]` permet d'exécuter du code de manière conditionnelle, avec des opérateurs de test tels que `-f`, `-d`, `-eq`, `-ne`, `-gt`, `-lt`, `-z`, `-n`. La structure `case / esac` offre une alternative lisible pour la correspondance de motifs. Les boucles `for` et `while` permettent d'itérer sur des listes ou de répéter des actions, avec `break` et `continue` pour en contrôler le flux. Les codes de retour (0 = succès, non-zéro = erreur) sous-tendent le fonctionnement des conditionnels et des boucles.

Synthèse

Couverture Jour 1

Chapitre	Thème
1	Introduction au shell Bash et à son environnement
2	Syntaxe de base : variables, types, commandes simples
3	Création et exécution d'un script
4	Structures de contrôle : conditions (if, case), boucles (for, while)

Memo — Commandes essentielles

```
# Environnement
env                         # Affiche les variables d'environnement
echo $VAR                   # Affiche la valeur d'une variable
export VAR=valeur           # Exporte une variable vers les processus enfants
echo $SHELL                 # Affiche l'interpréteur par défaut
echo $HOME                  # Affiche le répertoire personnel
echo $PATH                  # Affiche les répertoires de recherche des commandes

# Variables et commandes
VAR="valeur"                # Assignation sans espaces autour de =
echo "$VAR"                 # Interpolation avec guillemets doubles
echo '$VAR'                 # Pas d'interprétation avec guillemets simples
date                        # Affiche la date et l'heure
whoami                      # Affiche l'utilisateur courant
hostname                    # Affiche le nom de la machine
pwd                         # Affiche le répertoire de travail courant
ls -l                       # Liste détaillée d'un répertoire
cat fichier                 # Affiche le contenu d'un fichier

# Redirections
commande > fichier          # Redirige la sortie vers un fichier (écrasement)
commande >> fichier         # Ajoute la sortie à la fin d'un fichier
commande1 | commande2       # Tube : sortie de commande1 vers entrée de commande2

# Script
chmod 755 script.sh         # Rend le script exécutable (755)
chmod u+x script.sh         # Ajoute le droit d'exécution pour le propriétaire
./script.sh                 # Exécute le script directement
bash script.sh              # Exécute via l'interpréteur Bash
source script.sh            # Exécute dans le shell courant
$?                          # Code de retour de la dernière commande (0 = succès)

# Structures de contrôle
if [ condition ]; then ... fi        # Condition simple
case var in motif) ... ;; esac       # Sélection multi-cas
for var in liste; do ... done        # Boucle sur une liste
while [ condition ]; do ... done     # Boucle conditionnelle
read -p "invite " VAR                # Lecture clavier avec invite
break                                # Sortie immédiate d'une boucle
continue                             # Passage à l'itération suivante
```

Ressources supplémentaires

Outils utilisés

Outil	Version	Usage
bash	5.x	Interpréteur de commandes et langage de script
coreutils	—	Commandes de base du système Linux

Scripting Bash et PowerShell — JOUR 2

Bash (2/2) — Automatisation, administration et scripts interactifs (22/09/2026)

FIL CONDUCTEUR

Après avoir maîtrisé les bases de Bash (J1), cette journée traite de l'automatisation système : manipulation de fichiers, gestion des erreurs, administration (utilisateurs, services, journaux), tâches planifiées (cron), scripts interactifs, processus, droits et bonnes pratiques.

Objectifs du Jour

- Manipuler fichiers et répertoires via des scripts Bash
- Gérer les erreurs simplement dans un script
- Automatiser des tâches administratives (utilisateurs, services, journaux)
- Planifier des scripts avec cron
- Écrire des scripts interactifs (lecture clavier, menus)
- Gérer les processus et les droits d'accès
- Appliquer les bonnes pratiques d'écriture et de factorisation

Planning du Jour

Chapitre	Horaires	Thème	Durée
5	9h00-10h30	Manipulation de fichiers et répertoires	1h30
—	10h30-10h45	Pause	15min
6	10h45-12h00	Gestion des erreurs simples	1h15
—	12h00-13h00	Pause déjeuner	1h
7	13h00-14h30	Automatisation de tâches administratives : utilisateurs, services, journaux	1h30
—	14h30-14h45	Pause	15min
8	14h45-16h00	Scripts interactifs, processus, droits, bonnes pratiques et cas pratique	1h15

Chapitre 5 — Manipulation de fichiers et répertoires (9h00-10h30)

Objectifs

- Créer, copier, déplacer et supprimer fichiers et répertoires dans un script
- Vérifier l'existence d'un fichier avant traitement
- Lire et écrire des fichiers ligne par ligne

Contenu Théorique (1h)

1. Commandes de manipulation de fichiers (30min)

```
touch fichier.txt
# touch : crée un fichier vide ou met à jour son horodatage

cp source.txt destination.txt
# cp : copie un fichier d'une source vers une destination

mv fichier.txt nouveau_nom.txt
# mv : déplace ou renomme un fichier

rm fichier.txt
# rm : supprime un fichier
```

```
mkdir dossier
# mkdir : crée un répertoire

rmdir dossier
# rmdir : supprime un répertoire vide

rm -rf dossier
# rm : supprime
# -r : récursif (supprime le contenu)
# -f : forcer (sans confirmation)
```

PIÈGE

`rm -rf` est irréversible et dangereux. Toujours vérifier le chemin cible avant d'exécuter dans un script.

2. Vérifier l'existence d'un fichier (30min)

```
if [ -f /etc/passwd ]; then
    echo "Le fichier existe"
fi
# -f : teste si le chemin est un fichier régulier

if [ -d /etc ]; then
    echo "Le répertoire existe"
fi
# -d : teste si le chemin est un répertoire
```

3. Lire un fichier ligne par ligne (30min)

```
while read -r LIGNE; do
    echo "Ligne : $LIGNE"
done < /etc/hostname
# read -r : lit une ligne sans interpréter les antislashs
# < : redirige le contenu du fichier vers la boucle
```

> **CONCEPT CLÉ**
>
> L'option `-r` de `read` empêche l'interprétation des antislashs (`\`) comme caractères d'échappement, préservant le contenu brut des lignes.

Fig 2.1 — Manipulation de fichiers et répertoires

Diagramme		
Script	**mkdir / rmdir**	
Script	**test -f / test -d**	
Script	**while read**	E

Figure — Diagramme Mermaid

Lab 05 — Manipulation de fichiers (45min)

Environnement

Élément	Valeur
Poste	Linux (Bash)
Répertoire de travail	/tmp/lab_scripting

Exercices

Exercice 13 — Créer et organiser des fichiers (15min)

```
mkdir -p /tmp/lab_scripting/dossier_a
touch /tmp/lab_scripting/fichier1.txt
cp /tmp/lab_scripting/fichier1.txt /tmp/lab_scripting/dossier_a/
mv /tmp/lab_scripting/fichier1.txt /tmp/lab_scripting/fichier_renomme.txt
```

- Le répertoire dossier_a est créé
- Le fichier est copié puis renommé

Exercice 14 — Vérifier l'existence avant traitement (15min)

```
CIBLE="/etc/hostname"
if [ -f "$CIBLE" ]; then
    echo "Le fichier $CIBLE est présent"
else
    echo "Le fichier $CIBLE est absent"
fi
```

- La condition vérifie correctement l'existence du fichier
- Le message adapté s'affiche

Solution

Solution — Exercices 13 et 14

```
mkdir -p /tmp/lab_scripting/dossier_a
touch /tmp/lab_scripting/fichier1.txt
cp /tmp/lab_scripting/fichier1.txt /tmp/lab_scripting/dossier_a/
mv /tmp/lab_scripting/fichier1.txt /tmp/lab_scripting/fichier_renomme.txt
CIBLE="/etc/hostname"
if [ -f "$CIBLE" ]; then
    echo "Le fichier $CIBLE est présent"
else
    echo "Le fichier $CIBLE est absent"
fi
```

Démonstration commentée

L'opérateur de redirection `<` envoie le contenu d'un fichier en entrée standard d'une boucle `while read`. Cela permet de traiter chaque ligne individuellement sans charger le fichier entier en mémoire.

Correction et résolution pas à pas

- Créez le répertoire de travail avec `mkdir -p`.
- Créez un fichier avec `touch`.
- Copiez-le dans un sous-répertoire avec `cp`.
- Renommez-le avec `mv`.
- Testez l'existence d'un fichier système avec `[ -f ]`.

Bonnes pratiques

Pratique	Description
Vérifier avant supprimer	Tester l'existence d'un fichier avant `rm`
Chemins absolus	Privilégier les chemins absolus dans les scripts d'administration

BONNES PRATIQUES

Utilisez des chemins absolus dans les scripts d'administration pour éviter les ambiguïtés liées au répertoire courant.

Script de correction : `fichiers.sh`

Exercices — Chapitre 5

Exercice 15 — Lire un fichier ligne par ligne (10min)

```
while read -r LIGNE; do
    echo "Contenu : $LIGNE"
done < /etc/hostname
```

- Quel est le rôle de l'option `-r` ?
- Que fait la redirection `<` ?

Solution

Solution — Exercice 15

`-r` empêche l'interprétation des antislashs. `<` redirige le contenu du fichier vers l'entrée standard de la boucle.

- L'option -r est comprise
- La redirection < est comprise

> **Ce qu'on a retenu**

> - `touch`, `cp`, `mv` et `rm` permettent de créer, copier, déplacer et supprimer des fichiers.

> - `mkdir`, `rmdir` et `rm -rf` gèrent la création et la suppression des répertoires.

> - `[ -f ]` et `[ -d ]` testent l'existence d'un fichier ou d'un répertoire avant traitement.

> - `read -r` lit une ligne sans interpréter les antislashs comme des caractères d'échappement.

> - `rm -rf` est irréversible : toujours vérifier le chemin cible avant exécution.

Chapitre 6 — Gestion des erreurs simples (10h45-12h00)

Objectifs

- Tester le code de retour d'une commande
- Utiliser `set -e` pour arrêter un script en cas d'erreur
- Afficher des messages d'erreur clairs

Contenu Théorique (45min)

1. Le code de retour $? (25min)

Chaque commande renvoie un code de retour : 0 indique le succès, toute autre valeur une erreur.

```
ls /fichier_inexistant
echo $?
# ls : tente de lister un fichier inexistant (échec)
# echo $? : affiche le code de retour (non-zéro en cas d'erreur)
```

2. Arrêter le script en cas d'erreur avec set -e (20min)

```
#!/bin/bash
set -e
# set -e : arrête le script dès qu'une commande retourne une erreur

cp fichier_inexistant destination/
echo "Cette ligne ne s'exécutera jamais"
```

BON À SAVOIR

`set -e` est utile pour éviter qu'un script continue dans un état incohérent après un échec. Cependant, certaines commandes attendues comme échouant doivent être protégées (ex: `commande || true`).

PIÈGE

`set -e` ne s'applique pas aux commandes dont le code de retour est explicitement testé (dans un `if`, une boucle `while` ou précédées de `!`), ni au dernier élément d'un pipeline à moins d'activer aussi `set -o pipefail`. Une gestion d'erreur explicite reste préférable.

3. Gestion conditionnelle des erreurs (20min)

```
if cp source.txt destination/; then
    echo "Copie réussie"
else
    echo "Échec de la copie" >&2
    exit 1
fi
# >&2 : redirige le message vers la sortie d'erreur standard
# exit 1 : termine le script avec un code d'erreur
```

Fig 2.2 — Gestion des erreurs dans un script

Diagramme		
Commande	**code retour non nul**	
C	**Oui**	
C	**Non**	E

Figure — Diagramme Mermaid

Lab 06 — Gestion des erreurs (45min)

Environnement

Élément	Valeur
Poste	Linux (Bash)
Fichier	erreurs.sh

Exercices

Exercice 16 — Tester le code de retour (15min)

```
ls /fichier_inexistant
CODE_RETOUR=$?
echo "Code de retour : $CODE_RETOUR"
```

- Le code de retour est non-zéro pour un fichier inexistant
- La valeur est capturée dans CODE_RETOUR

Exercice 17 — Script avec gestion d'erreur (15min)

```
#!/bin/bash
set -e
SOURCE="/etc/hostname"
DEST="/tmp/backup_hostname"
if cp "$SOURCE" "$DEST"; then
    echo "Copie réussie vers $DEST"
else
    echo "Échec de la copie" >&2
    exit 1
fi
```

- Le script quitte avec le code 1 si la copie échoue
- Le message d'erreur s'affiche sur la sortie d'erreur

Solution

Solution — Exercices 16 et 17

```
#!/bin/bash
set -e
SOURCE="/etc/hostname"
DEST="/tmp/backup_hostname"
if cp "$SOURCE" "$DEST"; then
    echo "Copie réussie vers $DEST"
else
    echo "Échec de la copie" >&2
    exit 1
fi
```

Démonstration commentée

La redirection `>&2` envoie le message vers `stderr` (sortie d'erreur standard), séparant ainsi les messages d'erreur des résultats normaux (`stdout`). Cela permet de rediriger ou de filtrer les erreurs indépendamment.

Correction et résolution pas à pas

- Testez une commande qui échoue et affichez `$?`.
- Ajoutez `set -e` au début d'un script.
- Enveloppez une commande risquée dans un `if` avec message d'erreur et `exit 1`.
- Vérifiez que le script s'arrête proprement en cas d'échec.

Bonnes pratiques

Pratique	Description
Séparer stdout et stderr	Utiliser `>&2` pour les messages d'erreur
Quitter avec un code significatif	`exit 1` pour erreur, `exit 0` pour succès

BONNES PRATIQUES

Toujours distinguer les messages informatifs (`stdout`) des messages d'erreur (`stderr`) pour faciliter le débogage et l'automatisation.

Script de correction : `erreurs.sh`

Exercices — Chapitre 6

Exercice 18 — Protéger une commande attendue comme échouante (10min)

```
grep "motif" fichier.txt || true
# grep : recherche un motif dans un fichier
# || true : si grep échoue, exécute true pour éviter l'arrêt avec set -e
```

- Pourquoi ajouter `|| true` après `grep` ?
- Quel serait l'effet de `set -e` sans cette protection ?

Solution

Solution — Exercice 18

`grep` renvoie un code non-zéro si le motif n'est pas trouvé. Avec `set -e`, le script s'arrêterait. `|| true` force un code de retour 0 pour continuer l'exécution.

- Le rôle de `|| true` est compris
- L'interaction avec `set -e` est identifiée

> **Ce qu'on a retenu**

> - Le code de retour `$?` vaut 0 en cas de succès et une valeur non-zéro en cas d'erreur.

> - `set -e` arrête l'exécution du script dès qu'une commande échoue.

> - `>&2` redirige les messages d'erreur vers le flux stderr, séparé de stdout.

> - `exit 1` termine le script avec un code d'erreur significatif.

> - `|| true` protège une commande attendue comme échouante contre l'arrêt provoqué par `set -e`.

Chapitre 7 — Automatisation de tâches administratives : utilisateurs, services, journaux (13h00-14h30)

Objectifs

- Automatiser la gestion d'utilisateurs et de groupes
- Gérer les services système dans un script
- Consulter et faire tourner les journaux (logs)

Contenu Théorique (1h)

1. Gestion d'utilisateurs et de groupes (30min)

```
adduser nouvel_utilisateur
# adduser : crée un nouveau compte utilisateur

usermod -aG groupe utilisateur
# usermod : modifie un compte utilisateur
# -aG : ajoute l'utilisateur au groupe supplémentaire (append)

groupadd nouveau_groupe
# groupadd : crée un nouveau groupe
```

CONCEPT CLÉ

Ces commandes nécessitent des privilèges root (via `sudo`). Dans un script d'automatisation, vérifiez les droits avant exécution.

BON À SAVOIR

`systemctl` et `journalctl` s'appuient sur systemd, l'init par défaut de la plupart des distributions Linux modernes. Certains systèmes (Alpine Linux, environnements embarqués) utilisent un autre gestionnaire de services où ces commandes sont absentes.

2. Gestion des services (30min)

```
systemctl status ssh
# systemctl : gère les services systemd
# status : affiche l'état d'un service

systemctl start ssh
# start : lance un service

systemctl enable ssh
# enable : active le démarrage automatique au boot
```

3. Consulter les journaux (30min)

```
tail -n 50 /var/log/syslog
# tail : affiche la fin d'un fichier
# -n 50 : limite aux 50 dernières lignes

journalctl -n 100
# journalctl : consulte les journaux systemd
# -n 100 : affiche les 100 dernières entrées
```

Fig 2.3 — Automatisation administrative (utilisateurs, services, journaux)

Diagramme		
Script admin	**systemctl**	
Script admin	**tail / journalctl**	D

Figure — Diagramme Mermaid

Lab 07 — Automatisation administrative (45min)

Environnement

Élément	Valeur
Poste	Linux (Bash, sudo requis)
Service exemple	ssh

Exercices

Exercice 19 — Consulter le statut d'un service (15min)

```
systemctl status ssh
```

- Le statut du service ssh s'affiche
- L'état (actif/inactif) est identifiable

Exercice 20 — Lire les dernières lignes d'un journal (15min)

```
journalctl -n 30
# journalctl : consulte les journaux systemd
# -n 30 : affiche les 30 dernières entrées
```

- Les 30 dernières entrées du journal s'affichent
- Les horodatages et messages sont lisibles

Solution

Solution — Exercices 19 et 20

```
systemctl status ssh
journalctl -n 30
```

Démonstration commentée

`systemctl` est l'outil de gestion de systemd, l'init système des distributions Linux modernes. Il permet de démarrer, arrêter, activer et consulter l'état des services. `journalctl` centralise les journaux de tous les services systemd.

Correction et résolution pas à pas

- Vérifiez le statut d'un service avec `systemctl status`.
- Démarrez un service avec `systemctl start` (si nécessaire).
- Consultez les journaux récents avec `journalctl -n`.
- Observez les horodatages et les niveaux de sévérité.

Bonnes pratiques

Pratique	Description
Vérifier les privilèges	Contrôler que le script s'exécute avec sudo/root
Consulter avant modifier	Vérifier l'état d'un service avant d'intervenir

BONNES PRATIQUES

Un script d'administration doit vérifier qu'il dispose des privilèges nécessaires (`id -u` doit renvoyer 0 pour root) avant de tenter toute modification.

Script de correction : `admin.sh`

Exercices — Chapitre 7

Exercice 21 — Vérifier les privilèges root (10min)

```
if [ "$(id -u)" -ne 0 ]; then
    echo "Ce script doit être exécuté en tant que root" >&2
    exit 1
fi
# id -u : renvoie l'identifiant utilisateur (0 = root)
# -ne : not equal (différent de)
```

- Pourquoi vérifier `id -u` avant les opérations d'administration ?
- Que retourne `id -u` pour un utilisateur standard ?

Solution

Solution — Exercice 21

`id -u` retourne 0 pour root et une valeur non-zéro pour un utilisateur standard. La vérification empêche l'exécution d'opérations privilégiées sans droits suffisants.

- La vérification des privilèges est comprise
- Le code de retour de id -u pour root est identifié (0)

> **Ce qu'on a retenu**

> - `adduser`, `usermod` et `groupadd` nécessitent des privilèges root (via `sudo`).

> - `systemctl` gère les services systemd : `status`, `start`, `enable`, etc.

> - `journalctl` consulte les journaux centralisés de tous les services systemd.

> - `id -u` retourne 0 pour l'utilisateur root et une valeur non-zéro pour un utilisateur standard.

> - Un script d'administration doit vérifier ses privilèges avant toute opération sensible.

Chapitre 8 — Scripts interactifs, processus, droits, bonnes pratiques et cas pratique (14h45-16h00)

Objectifs

- Écrire des scripts interactifs avec lecture clavier et menus
- Gérer les processus (affichage, arrêt, priorité)
- Gérer les droits et permissions dans un script
- Appliquer les bonnes pratiques d'écriture et de factorisation
- Réaliser un cas pratique Bash complet

Contenu Théorique (45min)

1. Scripts interactifs : lecture clavier et menus (25min)

```
read -p "Choisissez une option (1-3) : " CHOIX
case "$CHOIX" in
    1) echo "Option 1 sélectionnée" ;;
    2) echo "Option 2 sélectionnée" ;;
    3) echo "Option 3 sélectionnée" ;;
    *) echo "Option invalide" ;;
esac
# read -p : lit une saisie avec invite affichée
# case : sélectionne selon la valeur saisie
```

2. Gestion des processus (20min)

```
ps aux
# ps : affiche les processus en cours
# a : tous les utilisateurs
# u : format utilisateur détaillé
# x : processus sans terminal

kill 1234
# kill : envoie un signal à un processus (identifié par son PID)

kill -9 1234
# -9 : signal SIGKILL (arrêt forcé, immédiat)
```

> **BON À SAVOIR**
>
> `kill` sans numéro envoie SIGTERM (demande d'arrêt propre). `kill -9` force l'arrêt immédiat sans nettoyage.

3. Droits et permissions (30min)

```
chmod 700 script_prive.sh
# chmod : modifie les droits
# 700 : lecture/écriture/exécution pour le propriétaire seulement

chown utilisateur:groupe fichier.txt
# chown : modifie le propriétaire et le groupe d'un fichier
```

> **CONCEPT CLÉ**
>
> Les droits s'expriment en octal : 4 (lecture), 2 (écriture), 1 (exécution), additionnés par catégorie (propriétaire, groupe, autres).

Fig 2.4 — Script interactif et gestion des processus

Diagramme		
B	choix valide	
B	choix invalide	
C	→	
D	→	
Gestion processus	kill	H

Figure — Diagramme Mermaid

Lab 08 — Scripts interactifs, processus et droits (45min)

Environnement

Élément	Valeur
Poste	Linux (Bash)
Fichier	menu.sh

Exercices

Exercice 22 — Créer un menu interactif (20min)

```
#!/bin/bash
echo "=== Menu d'administration ==="
echo "1. Afficher la date"
echo "2. Lister les processus"
echo "3. Quitter"
read -p "Votre choix : " CHOIX
case "$CHOIX" in
    1) date ;;
    2) ps aux ;;
    3) echo "Au revoir !" ; exit 0 ;;
    *) echo "Choix invalide" ;;
esac
```

- Le menu s'affiche avec les 3 options
- La saisie utilisateur déclenche l'action correspondante

Exercice 23 — Gérer les droits d'un script (20min)

```
chmod 700 menu.sh
ls -l menu.sh
# chmod 700 : seuls le propriétaire peut lire/écrire/exécuter
# ls -l : affiche les droits détaillés du fichier
```

- Le script est rendu exécutable pour le propriétaire seulement
- Les droits s'affichent avec ls -l

Solution

Solution — Exercices 22 et 23

Contenu du script `menu.sh` :

```
#!/bin/bash
echo "=== Menu d'administration ==="
echo "1. Afficher la date"
echo "2. Lister les processus"
echo "3. Quitter"
read -p "Votre choix : " CHOIX
case "$CHOIX" in
    1) date ;;
    2) ps aux ;;
    3) echo "Au revoir !" ; exit 0 ;;
    *) echo "Choix invalide" ;;
esac
```

Commandes d'exécution :

```
chmod 700 menu.sh
ls -l menu.sh
```

Démonstration commentée

Le menu utilise `case` pour associer chaque saisie numérique à une action. `ps aux` liste tous les processus avec leur identifiant (PID), permettant d'identifier un processus avant de l'arrêter avec `kill`.

Correction et résolution pas à pas

- Créez un fichier `menu.sh` avec le shebang.
- Ajoutez l'affichage du menu et `read -p` pour la saisie.
- Ajoutez un `case` traitant chaque option.
- Rendez le script exécutable avec `chmod 700`.
- Testez chaque option du menu.

Bonnes pratiques

Pratique	Description
Factoriser les tâches répétitives	Créer des fonctions pour éviter la duplication
Menu avec option par défaut	Traiter le cas `*)` pour les saisies invalides
Droits minimaux	Accorder seulement les droits nécessaires (700)

BONNES PRATIQUES

Factorisez le code répété en fonctions pour rendre le script lisible, maintenable et facile à tester.

Cas pratique Bash — Sauvegarde automatisée avec rapport (45min)

Objectif

Écrire un script `sauvegarde.sh` qui :

- Vérifie les privilèges root
- Crée une archive d'un répertoire source
- Enregistre le résultat dans un journal
- Gère les erreurs à chaque étape

```
#!/bin/bash
set -e

SOURCE="/etc"
DEST="/tmp/sauvegarde_$(date +%Y%m%d).tar.gz"
LOG="/var/log/sauvegarde.log"

if [ "$(id -u)" -ne 0 ]; then
    echo "Ce script doit être exécuté en tant que root" >&2
    exit 1
fi

echo "=== Début de la sauvegarde : $(date) ===" >> "$LOG"

if tar -czf "$DEST" "$SOURCE" 2>>"$LOG"; then
    echo "Sauvegarde réussie : $DEST" >> "$LOG"
else
    echo "Échec de la sauvegarde" >&2
    exit 1
fi

echo "=== Fin de la sauvegarde : $(date) ===" >> "$LOG"
```

- Le script vérifie les privilèges root
- L'archive est créée avec tar
- Le journal est mis à jour à chaque étape

> **CONCEPT CLÉ**
>
> `date +%Y%m%d` génère un horodatage au format AAAAMMJJ pour nommer les archives de façon unique.

Script de correction : `sauvegarde.sh`

Exercices — Chapitre 8

Exercice 24 — Identifier et arrêter un processus (10min)

```
ps aux | grep nom_processus
kill "$PID"
# ps : liste les processus
# grep : filtre par nom de processus
# kill : envoie un signal d'arret au PID identifie (stocker le PID dans la variable $PID avant)
```

- Comment identifier le PID d'un processus ?
- Quelle différence entre `kill` et `kill -9` ?

Solution

Solution — Exercice 24

Le PID est identifié avec `ps aux | grep nom`. `kill` demande un arrêt propre (SIGTERM), `kill -9` force l'arrêt immédiat (SIGKILL).

- Le PID d'un processus est identifiable
- La différence SIGTERM / SIGKILL est comprise

> **Ce qu'on a retenu**

> - `read -p` et `case` permettent de construire des menus interactifs réactifs à la saisie clavier.

> - `ps aux` liste les processus avec leur identifiant (PID) ; `kill` envoie un signal d'arrêt.

> - `kill` demande un arrêt propre (SIGTERM) ; `kill -9` force l'arrêt immédiat (SIGKILL).

> - `chmod` modifie les droits d'accès (notation octale) ; `chown` modifie le propriétaire.

> - Factoriser le code en fonctions et appliquer le principe des droits minimaux améliore la maintenabilité.

Synthèse

Couverture Jour 2

Chapitre	Thème
5	Manipulation de fichiers et répertoires
6	Gestion des erreurs simples
7	Automatisation de tâches administratives (utilisateurs, services, journaux)
8	Scripts interactifs, processus, droits, bonnes pratiques, cas pratique

Memo — Commandes Essentielles

```
# Fichiers / répertoires
touch fichier            # Crée un fichier vide
cp src dst               # Copie un fichier
mv src dst               # Déplace / renomme
rm fichier               # Supprime un fichier
mkdir dossier            # Crée un répertoire
rmdir dossier            # Supprime un répertoire vide

# Vérifications
[ -f fichier ]           # Teste si c'est un fichier
[ -d dossier ]           # Teste si c'est un répertoire

# Erreurs
echo $?                  # Code de retour
set -e                   # Arrêt en cas d'erreur
>&2                      # Redirection vers stderr

# Administration
adduser utilisateur      # Crée un utilisateur
systemctl status svc     # État d'un service
journalctl -n 50         # Dernières lignes du journal

# Interactif / processus
read -p "invite " VAR    # Lecture clavier
ps aux                   # Liste des processus
kill PID                 # Arrêt d'un processus
chmod 700 fichier        # Droits propriétaire uniquement
```

Ressources supplémentaires

Outils utilisés

Outil	Version	Usage
bash	5.x	Interpréteur de commandes
coreutils	—	Commandes de base du système Linux

Scripting Bash et PowerShell — JOUR 3

PowerShell (1/2) + Python initiation — Environnement, cmdlets, exécution, structures de contrôle et Python (08/11/2026)

FIL CONDUCTEUR

Cette troisième journée introduit PowerShell sous Windows (chapitres 9 à 12) puis ouvre sur Python pour l'administration système sous Linux (chapitre 17, issu du module EXTRA). L'objectif est de faire comprendre la rupture de paradigme entre Bash (flux de texte) et PowerShell (objets .NET), puis de présenter Python comme un langage interprété à typage dynamique complémentaire des shells.

Objectifs du Jour

- Comprendre l'environnement PowerShell et sa différence fondamentale avec Bash (manipulation d'objets .NET plutôt que de texte brut).
- Maîtriser les cmdlets de base, le pipeline d'objets, les variables, les tableaux, les hashtables et les opérateurs de comparaison.
- Savoir créer, exécuter un script PowerShell (.ps1) et gérer les politiques d'exécution (execution policy).
- Utiliser les structures de contrôle : conditions if/elseif/else, switch, boucles for, foreach, while, do-while, ainsi que break et continue.
- Comprendre ce qu'est Python, l'installer et le vérifier sur un poste Linux, écrire et exécuter un premier script Python avec shebang.

Planning du Jour

Chapitre	Horaires	Thème	Durée
9	9h00-10h30	Présentation de l'environnement PowerShell	1h30
—	10h30-10h45	Pause	15min
10	10h45-12h00	Cmdlets, pipeline, variables et syntaxe de base	1h15
—	12h00-13h00	Pause déjeuner	1h
11	13h00-14h30	Exécution d'un script et gestion des politiques d'exécution	1h30
—	14h30-14h45	Pause	15min
12	14h45-16h00	Structures de contrôle	1h15
—	16h00-16h15	Pause	15min
17	16h15-17h45	Introduction à Python pour l'administration système	1h30

Chapitre 9 — Présentation de l'environnement PowerShell (9h00-10h30)

Objectifs

- Expliquer ce qu'est PowerShell et en quoi il diffère de Bash (approche par objets .NET vs approche par texte).
- Identifier les environnements d'exécution et d'édition : console PowerShell, ISE, VS Code.
- Utiliser les cmdlets de découverte : Get-Command, Get-Help, Get-Member.
- Comprendre le rôle des alias et des profils PowerShell.

Contenu théorique (45min)

1. Qu'est-ce que PowerShell ? (20min)

PowerShell est un shell d'administration système et un langage de scripting développé par Microsoft. Contrairement à Bash, qui traite les données sous forme de flux de texte, PowerShell manipule des objets .NET. Chaque commande native de PowerShell, appelée cmdlet (commande native de PowerShell, suivant la convention Verbe-Nom), produit des objets structurés que l'on peut inspecter, filtrer et formater sans recourir à l'analyse textuelle (grep, awk, cut).

Caractéristique	Bash	PowerShell
Données manipulées	Flux de texte	Objets .NET
Extension de script	.sh	.ps1
Environnement principal	Linux / Unix	Windows (et multiplateforme via PowerShell 7)
Cmdlet / commande	commande externe ou builtin	cmdlet Verbe-Nom

CONCEPT CLÉ

PowerShell manipule des objets .NET, et non du texte brut. Cela permet d'accéder directement aux propriétés d'un objet (par exemple le nom, le statut ou le nom d'affichage d'un service) sans analyser une chaîne de caractères.

BON À SAVOIR

PowerShell existe en deux grandes familles : Windows PowerShell 5.1 (intégré à Windows, basé sur .NET Framework) et PowerShell 7+ (multiplateforme, basé sur .NET, installable sur Linux et macOS).

2. Les environnements PowerShell (15min)

Trois environnements principaux sont utilisés pour écrire et exécuter du PowerShell :

- La console PowerShell (powershell.exe ou pwsh) : interface en ligne de commande interactive.
- PowerShell ISE (Integrated Scripting Environment, éditeur intégré historique de PowerShell) : éditeur graphique avec console intégrée, désormais en mode de maintenance.
- Visual Studio Code avec l'extension PowerShell : environnement de développement moderne, recommandé pour l'écriture de scripts.

3. Cmdlets de base et découverte (25min)

Les cmdlets de découverte permettent d'explorer l'environnement sans connaître à l'avance le nom des commandes :

- Get-Command : liste les cmdlets, fonctions et alias disponibles.
- Get-Help : affiche l'aide détaillée d'une cmdlet.
- Get-Member : inspecte les propriétés et méthodes d'un objet.

```
Get-Command
# Get-Command : cmdlet qui récupère la liste des commandes disponibles (cmdlets, fonctions, alias)
# Aucun paramètre : retourne l'ensemble des commandes de la session courante
```

```
Get-Help Get-Service
# Get-Help : cmdlet qui affiche l'aide d'une commande
# Get-Service : nom de la cmdlet dont on souhaite consulter l'aide
# Pourquoi pédagogique : consulter l'aide avant d'utiliser une cmdlet inconnue évite les erreurs de syntax
```

```
Get-Service | Get-Member
# Get-Service : liste les services du système sous forme d'objets
# | : opérateur de pipeline, transmet l'objet de sortie de Get-Service vers Get-Member
# Get-Member : inspecte les propriétés et méthodes de l'objet transmis
```

4. Alias et profils (15min)

Un alias est un nom court qui fait référence à une cmdlet existante (par exemple `ls` est un alias de `Get-ChildItem`). Un profil PowerShell est un script personnel chargé automatiquement au démarrage de chaque session, utilisé pour définir des alias, des fonctions et des variables d'environnement personnalisées.

> **PIÈGE**
>
> Les alias sont pratiques en console interactive mais doivent être évités dans les scripts de production, car ils réduisent la lisibilité et peuvent masquer la cmdlet réelle appelée.

> **CONTRE-MESURE**
>
> Utiliser systématiquement le nom complet de la cmdlet (Verbe-Nom) dans les scripts, réserver les alias à l'usage interactif uniquement.

Fig 1 — Pipeline PowerShell : de la cmdlet à l'affichage formaté

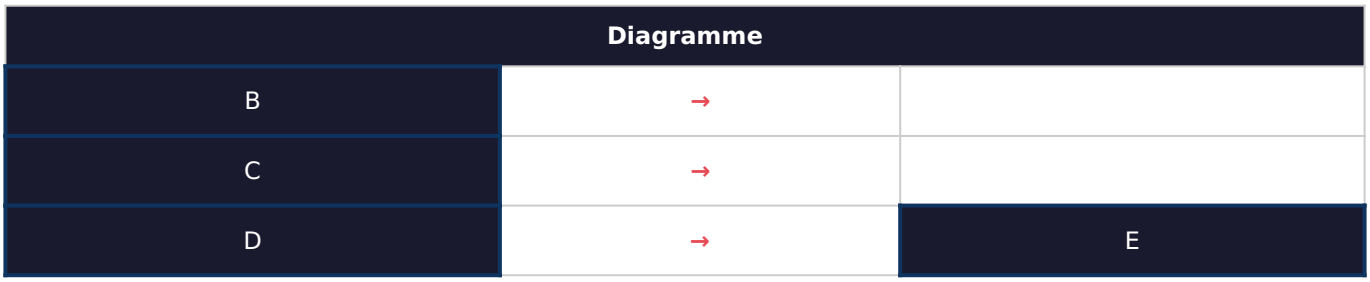


Figure — Diagramme Mermaid

Exemple concret de pipeline :

```
Get-Service | Where-Object { $_.Status -eq 'Running' } | Format-Table -AutoSize
# Get-Service : liste tous les services du système
# Where-Object : filtre les objets selon une condition
# $_.Status : propriété Status de chaque objet service transmis par le pipeline
# -eq : opérateur d'égalité PowerShell
# 'Running' : valeur de statut a comparer
# Format-Table : formate la sortie sous forme de tableau
# -AutoSize : ajuste automatiquement la largeur des colonnes
```

Lab 9 — Découverte de l'environnement PowerShell (45min)

Environnement

Élément	Valeur
Poste	Windows avec PowerShell 5.1 ou PowerShell 7+
Éditeur	Console PowerShell, ISE ou VS Code

Exercices (les exercices s'enchainent pour former un lab complet)

Exercice 9.1 — Lister les cmdlets disponibles (10min)

Utiliser Get-Command pour lister les cmdlets dont le nom commence par Get-.

```
Get-Command -Name Get-*
# Get-Command : recupere les commandes disponibles
# -Name : filtre les commandes par nom
# Get-* : motif generique, l'asterisque remplace n'importe quelle suite de caracteres
```

- La commande retourne une liste de cmdlets
- On identifie au moins Get-Command, Get-Help, Get-Member

Exercice 9.2 — Consulter l'aide d'une cmdlet (10min)

Afficher l'aide de Get-Service et identifier ses paramètres principaux.

```
Get-Help Get-Service
# Get-Help : affiche l'aide detaillee d'une cmdlet
# Get-Service : cible de l'aide
```

- L'aide s'affiche avec une description et une syntaxe
- On identifie le paramètre -Name

Exercice 9.3 — Inspecter un objet avec Get-Member (10min)

Afficher les propriétés d'un objet service.

```
Get-Service | Get-Member
# Get-Service : produit des objets ServiceController
# Get-Member : liste les propriétés et méthodes de l'objet
```

- La liste affiche des propriétés telles que Name, Status, DisplayName

Exercice 9.4 — Créer et utiliser un alias (10min)

Créer un alias monService pour Get-Service et l'utiliser.

```
Set-Alias -Name monService -Value Get-Service
# Set-Alias : crée ou modifie un alias
# -Name : nom de l'alias à créer
# -Value : cmdlet cible de l'alias
monService
# monService : utilise l'alias pour exécuter Get-Service
```

- L'alias est créé sans erreur

- L'exécution de monService retourne les services

Exercice 9.5 — Localiser le profil PowerShell (5min)

Afficher le chemin du profil courant.

```
$PROFILE
# $PROFILE : variable automatique contenant le chemin du profil de l'utilisateur courant
```

- Le chemin du profil s'affiche
- On note si le fichier existe déjà

Démonstration commentée

Exemple de pipeline complet avec interprétation de la sortie attendue :

```
Get-Process | Where-Object { $_.CPU -gt 10 } | Sort-Object CPU -Descending | Select-Object -First 5 -Prope
# Get-Process : liste les processus en cours d'exécution
# Where-Object : filtre les processus dont l'utilisation CPU dépasse 10 secondes
# $_.CPU : propriété CPU de chaque objet processus
# -gt : opérateur "strictement supérieur à"
# Sort-Object : trie les objets
# CPU : propriété utilisée comme clé de tri
# -Descending : ordre décroissant
# Select-Object : sélectionne un sous-ensemble d'objets et de propriétés
# -First 5 : limite aux cinq premiers résultats
# -Property Name, CPU : propriétés à afficher
```

Résultat attendu : un tableau listant les cinq processus les plus gourmands en CPU, avec leurs noms et leur temps CPU. Chaque ligne est un objet, pas une ligne de texte : on pourrait par la suite transmettre ce résultat à une autre cmdlet sans réanalyse textuelle.

Correction et résolution pas à pas

Solution

Solution du Lab 9

```
# Exercice 9.1
Get-Command -Name Get-*
# Exercice 9.2
Get-Help Get-Service
# Exercice 9.3
Get-Service | Get-Member
# Exercice 9.4
Set-Alias -Name monService -Value Get-Service
monService
# Exercice 9.5
$PROFILE
```

Résolution pas à pas :

- **Get-Command -Name Get-*** : on interroge le registre des commandes pour ne conserver que celles dont le nom commence par Get-. L'astérisque est un caractère générique qui correspond à toute suite de caractères, y compris vide.

- **Get-Help Get-Service** : PowerShell charge la documentation intégrée de la cmdlet. Si l'aide n'est pas installée, Get-Help propose de la télécharger en ligne avec le paramètre -Online.
- **Get-Service | Get-Member** : le pipeline transmet chaque objet ServiceController à Get-Member, qui introspecte son type et liste propriétés (Name, Status, DisplayName…) et méthodes (Start, Stop, Pause…).
- **Set-Alias -Name monService -Value Get-Service** : on associe le nom court monService à la cmdlet Get-Service. L'alias n'est valable que pour la session courante sauf s'il est inscrit dans le profil.
- **$PROFILE** : cette variable automatique contient le chemin complet du fichier de profil de l'utilisateur. Si le fichier n'existe pas, il peut être créé avec New-Item.

Bonnes pratiques

Pratique	Description
Nom complet des cmdlets	Utiliser Get-Service plutôt que gsv dans les scripts
Consultation de l'aide	Appeler Get-Help avant d'utiliser une cmdlet inconnue
Inspection des objets	Utiliser Get-Member pour découvrir les propriétés disponibles

BONNES PRATIQUES

Privilégier les noms complets de cmdlets dans les scripts pour garantir la lisibilité et la maintenabilité. Réserver les alias à la console interactive uniquement.

Exercices — Chapitre 9

Exercice 9.6 — Découverte d'une cmdlet inconnue (10min)

Choisir une cmdlet jamais utilisée (par exemple Get-WinEvent ou Get-CimInstance), afficher son aide, puis lister ses membres avec Get-Member.

- Quelle est la description de la cmdlet ?
- Quelles sont ses trois principales propriétés ?

Solution

Solution

```
Get-Help Get-WinEvent
Get-WinEvent -FilterHashtable @{LogName='System'; MaxEvents=5} | Get-Member
# Get-Help : affiche la documentation de Get-WinEvent
# Get-WinEvent : lit les journaux d'événements Windows (compatible PowerShell 5.1 et 7+)
# -FilterHashtable @{LogName='System'; MaxEvents=5} : filtre le journal système et limite aux 5 événemen
# Get-Member : introspecte l'objet d'événement
```

Résolution pas à pas :

- Get-Help Get-WinEvent affiche la description : la cmdlet récupère les événements des journaux d'événements Windows.

- Get-WinEvent -FilterHashtable @{LogName='System'; MaxEvents=5} produit cinq objets d'événement. Get-Member révèle leurs propriétés : Id, Level, Message, ProviderName, TimeCreated, etc.

- La description de la cmdlet est identifiée
- Au moins trois propriétés sont listées

Ce qu'on a retenu

CE QU'ON A RETENU

PowerShell manipule des objets .NET et non du texte brut. Les cmdlets suivent la convention Verbe-Nom. Get-Command, Get-Help et Get-Member sont les cmdlets de découverte essentielles. Les alias sont réservés à l'usage interactif ; les scripts utilisent les noms complets. Le profil PowerShell personnalise chaque session au démarrage.

Chapitre 10 — Cmdlets, pipeline, variables et syntaxe de base (10h45-12h00)

Objectifs

- Maîtriser la syntaxe Verbe-Nom des cmdlets PowerShell.
- Comprendre le pipeline d'objets et son fonctionnement.
- Déclarer et utiliser des variables, des tableaux et des hashtables.
- Utiliser les opérateurs de comparaison (-eq, -ne, -gt, -lt, -like, -match).

Contenu théorique (45min)

1. Syntaxe des cmdlets Verbe-Nom (15min)

Toute cmdlet PowerShell respecte la convention Verbe-Nom (par exemple Get-Service, Stop-Process, New-Item). Le verbe indique l'action, le nom indique la cible. Get-Verb liste les verbes approuvés par Microsoft.

```
Get-Verb
# Get-Verb : liste les verbes approuvés pour la dénomination des cmdlets
# Aucun paramètre : retourne l'ensemble des verbes standards
```

CONCEPT CLÉ

La syntaxe Verbe-Nom rend les cmdlets auto-documentées : Get-Service signifie « obtenir des services », Stop-Process signifie « arrêter un processus ».

> **BON À SAVOIR**
>
> Les verbes approuvés incluent Get, Set, New, Remove, Start, Stop, Out, Format, Select, Sort, Where, Invoke, etc.

2. Pipeline et objets (20min)

Le pipeline (chaîne de traitement) transmet des objets d'une cmdlet à l'autre. Contrairement à Bash où le texte est analysé à chaque étape, PowerShell conserve la structure objet tout au long du pipeline.

```
Get-Service | Where-Object { $_.Status -eq 'Running' } | Format-List
# Get-Service : produit des objets ServiceController
# Where-Object : filtre selon la condition entre accolades
# $_.Status : propriété Status de l'objet courant dans le pipeline
# -eq : opérateur d'égalité
# Format-List : affiche chaque propriété sur une ligne séparée
```

3. Variables, types, tableaux et hashtables (20min)

Une variable se déclare avec le préfixe $. PowerShell est faiblement typé mais permet le typage explicite.

```
$nom = 'Administrateur'
# $nom : déclaration d'une variable
# 'Administrateur' : valeur de type chaîne de caractères affectée

[int]$age = 35
# [int] : typage explicite de la variable en entier
# $age : nom de la variable
# 35 : valeur entière

$services = Get-Service
# $services : variable contenant le résultat d'une cmdlet (collection d'objets)
```

Tableau :

```
$liste = @('Serveur01', 'Serveur02', 'Serveur03')
# @() : opérateur de création de tableau
# $liste : variable contenant le tableau
$liste[0]
# $liste[0] : accès à l'élément d'index 0 (premier élément)
```

Hashtable (table de hachée, collection de paires clé/valeur) :

```
$config = @{ Serveur = 'SRV01'; Port = 8080; Actif = $true }
# @{} : opérateur de création de hashtable
# Serveur, Port, Actif : clés de la table de hachée
# 'SRV01', 8080, $true : valeurs associées
$config['Port']
# $config['Port'] : accès à la valeur associée à la clé Port
```

4. Opérateurs de comparaison (20min)

PowerShell utilise des opérateurs spécifiques, distincts des symboles C-style :

Opérateur	Signification	Exemple
-eq	égal à	$a -eq $b
-ne	différent de	$a -ne $b
-gt	strictement supérieur à	$a -gt $b
-lt	strictement inférieur à	$a -lt $b
-like	correspondance avec motif générique	$nom -like 'Admin*'
-match	correspondance avec expression régulière	$nom -match '^Admin'

```
$statut = 'Running'
if ($statut -eq 'Running') { 'Le service est en cours d'exécution' }
# -eq : compare l'égalité de deux valeurs
# 'Running' : littéral de chaîne comparé
```

PIÈGE

L'opérateur d'égalité en PowerShell est -eq, et non = (réservé à l'affectation). Confondre les deux est une erreur fréquente pour les venus de Bash ou de C.

CONTRE-MESURE

Vérifier systématiquement que -eq est utilisé pour les comparaisons et = pour les affectations.

Fig 2 — Flux d'écriture de script : éditeur, exécution, test

Diagramme		
B	→	
C	→	
D	→	
E	→	Éditeur VS Code / ISE

Figure — Diagramme Mermaid

Lab 10 — Manipulation de variables et pipeline (45min)

Environnement

Élément	Valeur
Poste	Windows avec PowerShell 5.1 ou PowerShell 7+

Exercices (les exercices s'enchainent pour former un lab complet)

Exercice 10.1 — Déclarer des variables typées (10min)

Créer une variable entière et une variable chaîne, puis afficher leurs valeurs.

```
[int]$annee = 2026
# [int] : typage explicite en entier
# $annee : nom de la variable
# 2026 : valeur affectée
[string]$utilisateur = 'Admin'
# [string] : typage explicite en chaîne de caractères
# $utilisateur : nom de la variable
# 'Admin' : valeur affectée
$annee
$utilisateur
```

- Les deux variables s'affichent correctement
- Le typage explicite est utilisé

Exercice 10.2 — Créer un tableau et un hashtable (10min)

Construire un tableau de serveurs et un hashtable de configuration.

```
$serveurs = @('SRV01', 'SRV02', 'SRV03')
# @() : création d'un tableau indexé
# $serveurs : variable contenant le tableau
$config = @{ Environnement = 'Production'; Version = '5.1' }
# @{} : création d'une hashtable (paires clé/valeur)
# $config : variable contenant la table de hachée
$serveurs[1]
$config['Environnement']
```

- Le tableau contient trois éléments
- L'accès par index et par clé fonctionne

Exercice 10.3 — Construire un pipeline de filtrage (15min)

Lister les services arrêtés et les afficher sous forme de liste.

```
Get-Service | Where-Object { $_.Status -eq 'Stopped' } | Format-List Name, DisplayName
# Get-Service : liste tous les services
# Where-Object : filtre les services dont le statut est Stopped
# $_.Status : propriété Status de l'objet service
# -eq : opérateur d'égalité
# Format-List : affiche chaque propriété sur une ligne
# Name, DisplayName : propriétés à afficher
```

- Seuls les services arrêtés s'affichent
- Le format est une liste, pas un tableau

Exercice 10.4 — Utiliser les opérateurs de comparaison (10min)

Tester plusieurs opérateurs sur des variables.

```
$seuil = 50
$valeur = 75
$valeur -gt $seuil
# -gt : retourne Vrai si $valeur est strictement supérieur à $seuil
$nom = 'Administrateur'
$nom -like 'Admin*'
# -like : retourne Vrai si la chaîne correspond au motif Admin*
```

- Les opérateurs retournent des valeurs booléennes ($true / $false)

Démonstration commentée

Exemple d'utilisation combinée de variables, tableau et pipeline :

```
$servicesCritiques = @('Spooler', 'EventSystem')
# $servicesCritiques : tableau contenant les noms des services critiques
foreach ($nom in $servicesCritiques) {
    # foreach : itère sur chaque élément du tableau
    # $nom : variable de boucle contenant le nom courant
    $service = Get-Service -Name $nom
    # Get-Service -Name : récupère le service par son nom exact
    # $nom : valeur transmise comme paramètre de nom
    if ($service.Status -eq 'Running') {
        # -eq : compare le statut à Running
        Write-Host "$nom est en cours d'exécution"
        # Write-Host : affiche un message dans la console
        # "$nom" : interpolation de variable dans une chaîne
    }
}
```

Résultat attendu : pour chaque service du tableau, un message indique s'il est en cours d'exécution.

Correction et résolution pas à pas

Solution

Solution du Lab 10

```
# Exercice 10.1
[int]$annee = 2026
[string]$utilisateur = 'Admin'
$annee
$utilisateur
# Exercice 10.2
$serveurs = @('SRV01', 'SRV02', 'SRV03')
$config = @{ Environnement = 'Production'; Version = '5.1' }
$serveurs[1]
$config['Environnement']
# Exercice 10.3
Get-Service | Where-Object { $_.Status -eq 'Stopped' } | Format-List Name, DisplayName
# Exercice 10.4
$seuil = 50
$valeur = 75
$valeur -gt $seuil
$nom = 'Administrateur'
$nom -like 'Admin*'
```

Résolution pas à pas :

- **[int]$annee = 2026** : le crochet [int] force le typage entier. Toute affectation ultérieure d'une valeur non convertible en entier lèvera une erreur.
- **@('SRV01', 'SRV02', 'SRV03')** : l'opérateur @() crée un tableau indexé à partir de zéro. $serveurs[1] retourne 'SRV02'.
- **@{ Environnement = 'Production'; Version = '5.1' }** : l'opérateur @{} crée une hashtable. L'accès se fait par clé avec $config['Environnement'].
- **Where-Object { $_.Status -eq 'Stopped' }** : le bloc d'accolades est un scriptblock évalué pour chaque objet. $_ fait référence à l'objet courant du pipeline.
- **-gt et -like** : -gt compare des nombres, -like effectue une correspondance de motif générique sur des chaînes.

Bonnes pratiques

Pratique	Description
Typage explicite	Utiliser [int], [string] pour documenter l'intention
Nommage explicite	Préférer $nombreDeServices à $n
Pipeline d'objets	Éviter la conversion en texte tant que possible

BONNES PRATIQUES

Déclarer les variables avec un nom explicite et un typage approprié. Utiliser le pipeline d'objets plutôt que de parser du texte, afin de préserver la structure des données.

Exercices — Chapitre 10

Exercice 10.5 — Comparaison de chaînes et motifs (10min)

Écrire une condition qui vérifie si une variable $nom commence par 'Test' et afficher un message adapté.

- Quel opérateur utiliser ?
- Quel est le résultat si $nom = 'Testeur' ?

Solution

Solution

```
$nom = 'Testeur'
if ($nom -like 'Test*') {
    Write-Host "Le nom commence par Test"
}
# -like : correspondance avec motif générique
# 'Test*' : motif où l'asterisque correspond à toute suite de caractères
```

Résolution pas à pas :

- L'opérateur -like est adapté pour une correspondance de motif générique sur une chaîne.
- Si $nom = 'Testeur', la condition est vraie car 'Testeur' commence par 'Test'. Le message s'affiche.

- L'opérateur -like est identifié
- Le résultat attendu est compris

Ce qu'on a retenu

CE QU'ON A RETENU

Les cmdlets suivent la syntaxe Verbe-Nom. Le pipeline transmet des objets, pas du texte. Les variables utilisent le préfixe $ et acceptent le typage explicite. Les tableaux s'écrivent avec @() et les hashtables avec @{}. Les opérateurs de comparaison sont -eq, -ne, -gt, -lt, -like, -match.

Chapitre 11 — Exécution d'un script et gestion des politiques d'exécution (13h00-14h30)

Objectifs

- Comprendre l'extension .ps1 et l'absence de shebang en PowerShell.
- Maîtriser les politiques d'exécution (Restricted, AllSigned, RemoteSigned, Unrestricted, Bypass).
- Exécuter un script avec .\script.ps1, l'opérateur d'appel & et le point-source ..
- Comprendre le principe des signatures de scripts.

Contenu théorique (45min)

1. Le fichier de script .ps1 (15min)

Un script PowerShell est enregistré dans un fichier d'extension .ps1. Contrairement aux scripts shell Unix, il n'utilise pas de shebang (#!) car l'interpréteur est invoqué explicitement par pwsh ou powershell.exe.

Élément	Description
Extension	.ps1 (PowerShell script)
Shebang	Non applicable (PowerShell)
Interpréteur	pwsh (PowerShell 7+) ou powershell.exe (Windows PowerShell 5.1)

2. Politiques d'exécution (25min)

La politique d'exécution (execution policy) est un mécanisme de sécurité de PowerShell qui détermine quels scripts peuvent s'exécuter et sous quelles conditions.

Politique	Signification
Restricted	Aucun script ne peut s'exécuter (par défaut sur Windows client)
AllSigned	Seuls les scripts signés par un éditeur de confiance s'exécutent
RemoteSigned	Les scripts locaux s'exécutent ; les scripts distants doivent être signés
Unrestricted	Tous les scripts s'exécutent (avec avertissement pour les distants)
Bypass	Aucun blocage, aucun avertissement (usage automatisé)

```
Get-ExecutionPolicy
# Get-ExecutionPolicy : affiche la politique d'exécution courante
# Aucun paramètre : retourne la politique effective
```

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
# Set-ExecutionPolicy : modifie la politique d'exécution
# -ExecutionPolicy : nouvelle politique à appliquer
# RemoteSigned : scripts locaux autorisés, distants doivent être signés
# -Scope CurrentUser : applique la politique uniquement à l'utilisateur courant
```

CONCEPT CLÉ

La politique d'exécution est une barrière de sécurité, pas une mesure de chiffrement ou d'authentification. Elle empêche l'exécution accidentelle de scripts malveillants.

BON À SAVOIR

Get-ExecutionPolicy -List affiche toutes les politiques par scope (MachinePolicy, UserPolicy, CurrentUser, LocalMachine).

3. Modes d'exécution d'un script (20min)

Plusieurs façons d'exécuter un script .ps1 :

- `.\script.ps1` : exécution dans le répertoire courant (nécessite le préfixe .\ pour des raisons de sécurité).
- `& 'C:\chemin\script.ps1'` : opérateur d'appel (&), exécute le script dans un nouveau scope.

- `. 'C:\chemin\script.ps1'` : point-source (dot-sourcing), exécute le script dans le scope courant (les fonctions et variables restent disponibles après l'exécution).

```
.\MonScript.ps1
# .\ : indique que le script se trouve dans le répertoire courant
# MonScript.ps1 : nom du fichier de script
```

```
& 'C:\Scripts\MonScript.ps1'
# & : opérateur d'appel, exécute le script contenu dans la chaîne
# 'C:\Scripts\MonScript.ps1' : chemin complet vers le script
```

```
. 'C:\Scripts\MonScript.ps1'
# . : opérateur de point-source (dot-sourcing)
# Le script s'exécute dans le scope courant, ses fonctions restent disponibles
```

PIÈGE

Le point-source (.) et l'opérateur d'appel (&) ne se comportent pas de la même manière : le point-source importe les fonctions et variables dans le scope courant, tandis que & les isole dans un scope temporaire.

CONTRE-MESURE

Utiliser & pour exécuter un script de manière isolée, et . uniquement lorsque l'on souhaite importer délibérément des fonctions dans la session courante.

4. Signatures de scripts (10min)

Une signature de script permet d'authentifier l'auteur et de garantir l'intégrité du fichier. Elle est requise lorsque la politique est AllSigned ou RemoteSigned pour les scripts distants. La signature repose sur un certificat de code signing.

CONTRE-MESURE

En production, privilégier RemoteSigned avec signature des scripts internes, plutôt que Bypass qui supprime toute protection.

Fig 3 — Politique d'exécution : flux de décision

	Diagramme	
B	**Restricted**	
B	**RemoteSigned**	
B	**Bypass**	
D	**Signe**	
D	**Non signe**	C

Figure — Diagramme Mermaid

Lab 11 — Exécuter un script et gérer la politique d'exécution (45min)

Environnement

Élément	Valeur
Poste	Windows avec PowerShell 5.1 ou PowerShell 7+
Droits	Utilisateur avec droits d'administrateur (pour Set-ExecutionPolicy)

Exercices (les exercices s'enchainent pour former un lab complet)

Exercice 11.1 — Consulter la politique d'exécution (10min)

Afficher la politique courante et l'ensemble des politiques par scope.

```
Get-ExecutionPolicy
# Get-ExecutionPolicy : affiche la politique effective
Get-ExecutionPolicy -List
# -List : affiche toutes les politiques par scope (MachinePolicy, UserPolicy, etc.)
```

- La politique courante s'affiche
- La liste par scope s'affiche

Exercice 11.2 — Créer un premier script .ps1 (10min)

Créer un fichier Bonjour.ps1 affichant un message.

```
'Bonjour, ceci est un script PowerShell.'
# Chaîne de caractères affichée dans la console
# Ce contenu sera enregistré dans le fichier Bonjour.ps1
```

- Le fichier Bonjour.ps1 est créé
- Il contient une commande d'affichage

Exercice 11.3 — Exécuter le script avec & et .\ (15min)

Exécuter Bonjour.ps1 avec l'opérateur d'appel, puis avec le préfixe .\.

```
& '.\Bonjour.ps1'
# & : opérateur d'appel, exécute le script dans un scope isolé
# '.\Bonjour.ps1' : chemin relatif vers le script
.\Bonjour.ps1
# .\ : préfixe de répertoire courant, requis pour exécuter un script local
```

- Le script s'exécute avec &
- Le script s'exécute avec .\

Exercice 11.4 — Modifier la politique d'exécution (10min)

Passer la politique en RemoteSigned pour l'utilisateur courant.

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
# Set-ExecutionPolicy : modifie la politique
# -ExecutionPolicy RemoteSigned : nouvelle politique
# -Scope CurrentUser : portée limitée à l'utilisateur courant
Get-ExecutionPolicy
# Vérification de la nouvelle politique
```

- La politique est passée à RemoteSigned
- La vérification confirme le changement

Démonstration commentée

Exemple de script complet avec point-source et opérateur d'appel :

```
# Contenu de MaFonction.ps1
function Afficher-Date {
    # function : déclare une fonction nommée Afficher-Date
    Get-Date
    # Get-Date : retourne la date et l'heure courantes
}
```

```
. '.\MaFonction.ps1'
# . : point-source, importe la fonction dans le scope courant
Afficher-Date
# Afficher-Date : la fonction est maintenant disponible dans la session
```

Résultat attendu : la date et l'heure courantes s'affichent. La fonction Afficher-Date reste utilisable après l'exécution du script grâce au point-source.

Correction et résolution pas à pas

Solution

Solution du Lab 11

```
# Exercice 11.1
Get-ExecutionPolicy
Get-ExecutionPolicy -List
# Exercice 11.2
# Contenu de Bonjour.ps1 :
'Bonjour, ceci est un script PowerShell.'
# Exercice 11.3
& '.\Bonjour.ps1'
.\Bonjour.ps1
# Exercice 11.4
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
Get-ExecutionPolicy
```

Résolution pas à pas :

- **Get-ExecutionPolicy** : sans paramètre, retourne la politique effective (souvent Restricted sur un poste client Windows).

- **Get-ExecutionPolicy -List** : affiche la politique pour chaque scope. Le scope le plus restrictif applicable l'emporte.
- **Création de Bonjour.ps1** : le fichier contient une simple chaîne. PowerShell affiche toute expression non affectée à une variable.
- **& '.\Bonjour.ps1'** : l'opérateur & exécute le script. Le chemin relatif nécessite le préfixe .\ pour éviter toute ambiguïté avec une commande externe.
- **Set-ExecutionPolicy -Scope CurrentUser** : modifie uniquement la politique de l'utilisateur courant, stockée dans son registre personnel, sans nécessiter forcément des droits d'administrateur pour ce scope.

Bonnes pratiques

Pratique	Description
Politique RemoteSigned	Utiliser RemoteSigned en production plutôt que Bypass
Scope CurrentUser	Appliquer les changements de politique à l'utilisateur courant
Opérateur d'appel	Utiliser & pour exécuter des scripts de manière isolée

BONNES PRATIQUES

Ne jamais utiliser Bypass sur un poste de production sans justification documentée. Préférer RemoteSigned et signer les scripts internes.

Exercices — Chapitre 11

Exercice 11.5 — Différence entre & et . (10min)

Créer un script DefinirVariable.ps1 contenant `$maVariable = 42`, l'exécuter avec & puis avec ., et vérifier si $maVariable existe après chaque exécution.

- Quelle est la différence de comportement ?
- Pourquoi ?

Solution

Solution

```
# Contenu de DefinirVariable.ps1
$maVariable = 42
# Exécution avec &
& '.\DefinirVariable.ps1'
$maVariable
# Exécution avec .
. '.\DefinirVariable.ps1'
$maVariable
```

Résolution pas à pas :

- Avec &, le script s'exécute dans un scope isolé : $maVariable n'existe pas après l'exécution.

- Avec . (point-source), le script s'exécute dans le scope courant : $maVariable reste disponible après l'exécution. Le point-source importe toutes les définitions du script dans la session.

- La différence de comportement est comprise
- Le rôle du scope est expliqué

Ce qu'on a retenu

CE QU'ON A RETENU

Un script PowerShell porte l'extension .ps1 et n'utilise pas de shebang. La politique d'exécution (Restricted, AllSigned, RemoteSigned, Unrestricted, Bypass) contrôle l'exécution des scripts. Get-ExecutionPolicy et Set-ExecutionPolicy permettent de la consulter et de la modifier. L'opérateur & exécute dans un scope isolé, le point-source . importe dans le scope courant.

Chapitre 12 — Structures de contrôle (14h45-16h00)

Objectifs

- Utiliser les conditions if / elseif / else.
- Maîtriser l'instruction switch.
- Utiliser les boucles for, foreach, while et do-while.
- Utiliser break et continue pour contrôler le flux d'exécution.

Contenu théorique (45min)

1. if / elseif / else (20min)

La condition if évalue une expression booléenne et exécute un bloc si elle est vraie.

```
$statut = 'Running'
if ($statut -eq 'Running') {
    # -eq : opérateur d'égalité
    Write-Host 'Le service est en cours d'exécution'
} elseif ($statut -eq 'Stopped') {
    # elseif : condition alternative si la première est fausse
    Write-Host 'Le service est arrêté'
} else {
    # else : bloc par défaut si aucune condition n'est vraie
    Write-Host 'Statut inconnu'
}
```

CONCEPT CLÉ

Les opérateurs de comparaison en PowerShell (-eq, -ne, -gt, -lt, -like, -match) retournent des valeurs booléennes ($true ou $false).

2. switch (15min)

L'instruction switch compare une valeur à plusieurs motifs et exécute le bloc correspondant.

```
$code = 404
switch ($code) {
    200 { 'Requête réussie' }
    404 { 'Ressource introuvable' }
    500 { 'Erreur serveur' }
    default { 'Code non géré' }
    # default : bloc exécuté si aucun motif ne correspond
}
```

3. Boucles for, foreach, while, do-while (25min)

Boucle for avec compteur :

```
for ($i = 0; $i -lt 5; $i++) {
    # $i = 0 : initialisation
    # $i -lt 5 : condition de continuation
    # $i++ : incrémentation après chaque itération
    Write-Host "Iteration $i"
}
```

Boucle foreach sur une collection :

```
$services = Get-Service | Select-Object -First 3
foreach ($s in $services) {
    # foreach : itère sur chaque élément de la collection
    # $s : variable de boucle contenant l'élément courant
    Write-Host $s.Name
}
```

Boucle while :

```
$compteur = 0
while ($compteur -lt 3) {
    # while : répète tant que la condition est vraie
    Write-Host "Compteur : $compteur"
    $compteur++
}
```

Boucle do-while :

```
$compteur = 0
do {
    # do : exécute le bloc au moins une fois
    Write-Host "Do-while : $compteur"
    $compteur++
} while ($compteur -lt 3)
# while : condition évaluée après l'exécution du bloc
```

4. break et continue (15min)

break interrompt immédiatement la boucle. continue passe à l'itération suivante.

```
for ($i = 0; $i -lt 10; $i++) {
    if ($i -eq 5) { break }
    # break : sort de la boucle immédiatement
    if ($i -eq 2) { continue }
    # continue : saute au reste de l'itération courante
    Write-Host $i
}
```

PIÈGE

Une boucle while sans condition de sortie ou avec une condition toujours vraie crée une boucle infinie, bloquant la session PowerShell.

CONTRE-MESURE

Toujours prévoir une modification de la variable de condition à l'intérieur du bloc, et tester la boucle avec des valeurs limites avant exécution.

Fig 4 — Flux de contrôle : condition if/else et boucle

Diagramme		
B	Oui	
B	Non	
C	→	
D	→	E

Figure — Diagramme Mermaid

Lab 12 — Structures de contrôle (45min)

Environnement

Élément	Valeur
Poste	Windows avec PowerShell 5.1 ou PowerShell 7+

Exercices (les exercices s'enchainent pour former un lab complet)

Exercice 12.1 — Condition if/elseif/else (10min)

Écrire un script qui affiche un message selon le statut d'un service.

```
$statut = 'Stopped'
if ($statut -eq 'Running') {
    Write-Host 'Service en cours d'exécution'
} elseif ($statut -eq 'Stopped') {
    Write-Host 'Service arrêté'
} else {
    Write-Host 'Autre statut'
}
```

- Le message correspond au statut testé
- Les trois branches (if, elseif, else) sont présentes

Exercice 12.2 — Instruction switch (10min)

Utiliser switch pour afficher un libellé selon un code HTTP.

```
$codeHttp = 404
switch ($codeHttp) {
    200 { Write-Host 'OK' }
    404 { Write-Host 'Non trouvé' }
    default { Write-Host 'Autre code' }
}
```

- Le code 404 affiche 'Non trouvé'
- Le bloc default est présent

Exercice 12.3 — Boucle foreach sur un tableau (15min)

Itérer sur un tableau de noms de services et afficher chacun.

```
$noms = @('Spooler', 'EventSystem', 'Schedule')
foreach ($nom in $noms) {
    # $nom : élément courant du tableau
    Write-Host "Service : $nom"
}
```

- Chaque nom de service s'affiche
- La boucle foreach est utilisée

Exercice 12.4 — Boucle while avec break et continue (10min)

Écrire une boucle qui s'arrête à une valeur donnée et saute certaines itérations.

```
$i = 0
while ($i -lt 10) {
    $i++
    if ($i -eq 3) { continue }
    if ($i -eq 7) { break }
    Write-Host $i
}
```

- L'itération 3 est sautée
- La boucle s'arrête à 7

Démonstration commentée

Exemple combinant switch et boucle foreach :

```
$services = Get-Service | Select-Object -First 5 -Property Name, Status
foreach ($s in $services) {
    switch ($s.Status) {
        'Running' { Write-Host "$($s.Name) : actif" }
        'Stopped' { Write-Host "$($s.Name) : arrêté" }
        default { Write-Host "$($s.Name) : autre statut" }
    }
    # $($s.Name) : expression sous-ensemble pour interpolation dans une chaîne
}
```

Résultat attendu : pour chacun des cinq premiers services, un message indique son statut (actif, arrêté ou autre).

Correction et résolution pas à pas

Solution

Solution du Lab 12

```
# Exercice 12.1
$statut = 'Stopped'
if ($statut -eq 'Running') {
    Write-Host 'Service en cours d'exécution'
} elseif ($statut -eq 'Stopped') {
    Write-Host 'Service arrêté'
} else {
    Write-Host 'Autre statut'
}
# Exercice 12.2
$codeHttp = 404
switch ($codeHttp) {
    200 { Write-Host 'OK' }
    404 { Write-Host 'Non trouvé' }
    default { Write-Host 'Autre code' }
}
# Exercice 12.3
$noms = @('Spooler', 'EventSystem', 'Schedule')
foreach ($nom in $noms) {
    Write-Host "Service : $nom"
}
# Exercice 12.4
$i = 0
while ($i -lt 10) {
    $i++
    if ($i -eq 3) { continue }
    if ($i -eq 7) { break }
    Write-Host $i
}
```

Résolution pas à pas :

- **if/elseif/else** : chaque branche est évaluée dans l'ordre. Seul le premier bloc dont la condition est vraie s'exécute.

- **switch** : la valeur de $codeHttp est comparée à chaque motif. 404 correspond au second bloc, qui affiche 'Non trouvé'.
- **foreach** : $noms contient trois chaînes. Chaque itération affecte l'élément courant à $nom et affiche le message.
- **while avec continue et break** : $i est incrémenté en début de boucle. À 3, continue saute l'affichage. À 7, break interrompt la boucle. Les valeurs affichées sont 1, 2, 4, 5, 6.

Bonnes pratiques

Pratique	Description
Condition de sortie	Prévoir toujours une condition de sortie dans les boucles
default dans switch	Prévoir un bloc default pour les cas non gérés
break et continue	Utiliser avec parcimonie pour préserver la lisibilité

BONNES PRATIQUES

Éviter les boucles infinies en vérifiant que la variable de condition est modifiée à chaque itération. Utiliser default dans switch pour traiter explicitement les cas imprévus.

Exercices — Chapitre 12

Exercice 12.5 — Boucle for avec condition (10min)

Écrire une boucle for qui affiche uniquement les nombres pairs de 0 à 10.

- Quelle condition utiliser pour tester la parité ?
- Quel opérateur de comparaison ?

Solution

Solution

```
for ($i = 0; $i -le 10; $i++) {
    if ($i % 2 -eq 0) {
        # % : opérateur modulo, retourne le reste de la division
        # -eq : opérateur d'égalité
        Write-Host $i
    }
}
```

Résolution pas à pas :

- L'opérateur modulo % retourne le reste de la division. Si $i % 2 -eq 0, le nombre est pair.
- L'opérateur -eq teste l'égalité avec 0. La boucle affiche 0, 2, 4, 6, 8, 10.

- La boucle affiche uniquement les nombres pairs
- L'opérateur modulo est compris

Ce qu'on a retenu

CE QU'ON A RETENU

Les conditions utilisent if/elseif/else avec les opérateurs -eq, -ne, -gt, -lt. switch compare une valeur à plusieurs motifs avec un bloc default optionnel. Les boucles for, foreach, while et do-while permettent l'itération. break interrompt une boucle, continue passe à l'itération suivante. Toujours prévoir une condition de sortie pour éviter les boucles infinies.

Chapitre 17 — Introduction à Python pour l'administration système (16h15-17h45)

Objectifs

- Comprendre ce qu'est Python et sa différence avec Bash.
- Installer et vérifier Python sur un poste Linux.
- Écrire et exécuter un premier script Python avec shebang.
- Manipuler des variables et les types de base (chaîne, entier, flottant, booléen, liste, dictionnaire).

Contenu théorique (45min)

1. Qu'est-ce que Python ? (30min)

Python est un langage de programmation interprété, de haut niveau, à typage dynamique. Contrairement à Bash qui traite tout comme du texte, Python manipule des objets typés. Le code source est compilé en bytecode puis exécuté par la machine virtuelle Python.

```
python3 --version
# python3 : lance l'interpréteur Python 3
# --version : affiche le numéro de version de Python
```

CONCEPT CLÉ

Python est un langage interprété : le code source est compilé en bytecode puis exécuté par la machine virtuelle Python, sans étape de compilation explicite vers du code machine.

BON À SAVOIR

Python est préinstallé sur la plupart des distributions Linux modernes ; la commande `which python3` permet de localiser l'interpréteur sur le poste.

2. Installation et vérification (15min)

Sur une distribution Linux moderne, Python 3 est généralement déjà présent. On vérifie sa présence et sa version :

```
which python3
# which : localise l'exécutable d'une commande dans le PATH
# python3 : nom de l'interpréteur Python 3 à localiser
python3 --version
# --version : affiche le numéro de version (ex. Python 3.12.x)
```

Si Python n'est pas installé, il peut être ajouté avec le gestionnaire de paquets de la distribution (par exemple `apt install python3` sur Debian/Ubuntu).

3. Shebang et premier script Python (30min)

Un script Python commence par une ligne appelée **shebang** (désignée par `#!`) qui indique au noyau quel interpréteur doit être utilisé pour exécuter le fichier. Cette ligne doit impérativement être la toute première ligne du fichier. Le shebang `#!/usr/bin/env python3` utilise la variable d'environnement PATH pour localiser l'interpréteur, ce qui rend le script portable d'un poste à l'autre.

```
#!/usr/bin/env python3
# #!/usr/bin/env python3 : shebang indiquant l'interpréteur Python via le PATH
print("Bonjour, administration système")
# print : affiche une chaîne de caractères sur la sortie standard
```

CONCEPT CLÉ

Le shebang avec `env` est préférable à un chemin fixe (comme `#!/usr/bin/python3`) car il s'adapte à l'installation de Python sur chaque poste.

BON À SAVOIR

Pour rendre le script exécutable, on utilise `chmod 755` (lecture/écriture/exécution pour le propriétaire, lecture/exécution pour le groupe et les autres), puis on le lance avec `./script.py`.

Fig 5 — Flux d'exécution d'un script Python



Figure — Diagramme Mermaid

4. Variables et types de base (25min)

En Python, une variable se crée par simple affectation, sans déclaration de type explicite (typage dynamique). Les types de base incluent :

- `str` : chaîne de caractères
- `int` : entier
- `float` : nombre à virgule flottante
- `bool` : booléen (True / False)
- `list` : liste ordonnée et modifiable

- `dict` : dictionnaire (paires clé/valeur)

```
nom = "Admin"
# nom : variable de type str (chaîne de caractères)
version = 3
# version : variable de type int (entier)
prix = 19.99
# prix : variable de type float (nombre à virgule flottante)
actif = True
# actif : variable de type bool (booléen)
serveurs = ['SRV01', 'SRV02', 'SRV03']
# serveurs : variable de type list (liste de chaînes)
config = {'Serveur': 'SRV01', 'Port': 8080}
# config : variable de type dict (dictionnaire clé/valeur)
print(f"Utilisateur : {nom}, Python : {version}, Actif : {actif}")
# print : affiche le résultat
# f"..." : f-string, permet l'interpolation de variables avec les accolades {}
```

Lab 17 — Premier script Python (45min)

Environnement

Élément	Valeur
Poste	Linux
Interpréteur	Python 3
Répertoire de travail	/tmp/lab_python

Exercices (les exercices s'enchainent pour former un lab complet)

Exercice 17.1 — Vérifier la version de Python (10min)

Vérifier que Python 3 est installé et afficher sa version.

```
python3 --version
# python3 : lance l'interpréteur Python 3
# --version : affiche le numéro de version
```

- La version de Python s'affiche correctement (ex. Python 3.x.y)
- L'interpréteur est localisé dans le PATH

Exercice 17.2 — Créer et exécuter un premier script (15min)

Créer un fichier hello.py avec un shebang, le rendre exécutable et l'exécuter.

```
nano hello.py
# nano : éditeur de texte en mode terminal
# hello.py : nom du fichier script Python
```

```
#!/usr/bin/env python3
# #!/usr/bin/env python3 : shebang indiquant l'interpréteur Python via le PATH
print("Bonjour, monde")
# print : affiche une chaîne de caractères sur la sortie standard
```

```
chmod 755 hello.py
# chmod : modifie les droits d'accès au fichier
# 755 : lecture/écriture/exécution pour le propriétaire, lecture/exécution pour le groupe et les autres
# hello.py : fichier cible à rendre exécutable
./hello.py
# ./ : exécute le script depuis le répertoire courant
# hello.py : nom du script à exécuter
```

- Le fichier hello.py est créé avec le shebang
- Le script est rendu exécutable avec chmod 755
- L'exécution affiche "Bonjour, monde"

Exercice 17.3 — Afficher des variables avec une f-string (10min)

Déclarer des variables et les afficher avec une f-string.

```
nom = "Admin"
version = 3
print(f"Utilisateur : {nom}, Python : {version}")
# nom : variable de type str
# version : variable de type int
# print : affiche la chaîne formatée
# f"..." : f-string avec interpolation des variables {nom} et {version}
```

- Les variables s'affichent correctement
- La f-string interpole les valeurs

Exercice 17.4 — Manipuler les types de base (15min)

Créer des variables de chaque type de base et afficher leur type avec `type()`.

```
chaine = "Serveur01"
entier = 42
flottant = 3.14
booleen = True
liste = ['SRV01', 'SRV02']
dictionnaire = {'nom': 'SRV01', 'ip': '10.0.0.1'}
print(type(chaine))
print(type(entier))
print(type(flottant))
print(type(booleen))
print(type(liste))
print(type(dictionnaire))
# type() : retourne le type d'un objet
# chaine, entier, flottant, booleen, liste, dictionnaire : variables de types str, int, float, bool, list, dict
```

- Chaque variable est créée avec le type approprié
- La fonction type() affiche le type de chaque variable

Exercice 17.5 — Script complet combinant shebang, variables et types (15min)

Créer un script infos.py qui affiche des informations système de manière formatée.

```
#!/usr/bin/env python3
# #!/usr/bin/env python3 : shebang indiquant l'interpréteur Python via le PATH
import os
# import os : importe le module d'interaction avec le système d'exploitation

utilisateur = os.getenv('USER', 'inconnu')
# os.getenv() : lit une variable d'environnement, avec une valeur par défaut si absente
# 'USER' : nom de la variable d'environnement contenant l'utilisateur courant
# 'inconnu' : valeur par défaut si la variable n'est pas définie

repertoire = os.getcwd()
# os.getcwd() : retourne le répertoire de travail courant

print(f"Utilisateur : {utilisateur}")
print(f"Répertoire : {repertoire}")
print(f"Types : str={type(utilisateur).__name__}, str={type(repertoire).__name__}")
# type(...).__name__ : retourne le nom du type de l'objet
```

```
chmod 755 infos.py
./infos.py
```

- Le script infos.py s'exécute sans erreur
- L'utilisateur et le répertoire courant s'affichent

Démonstration commentée

Le shebang `#!/usr/bin/env python3` permet au noyau Linux de trouver l'interpréteur Python via la variable d'environnement PATH, rendant le script portable. Le bloc `with` (vu dans le module EXTRA, chapitre 18) garantit la fermeture automatique des fichiers, évitant les fuites de descripteurs de fichier. L'interpolation avec f-strings (`f"..."`) est la méthode moderne et lisible pour insérer des variables dans une chaîne de caractères en Python.

Correction et résolution pas à pas

Solution

Solution du Lab 17

```
# Exercice 17.1
# python3 --version

# Exercice 17.2
#!/usr/bin/env python3
print("Bonjour, monde")

# Exercice 17.3
nom = "Admin"
version = 3
print(f"Utilisateur : {nom}, Python : {version}")

# Exercice 17.4
chaine = "Serveur01"
entier = 42
flottant = 3.14
booleen = True
liste = ['SRV01', 'SRV02']
dictionnaire = {'nom': 'SRV01', 'ip': '10.0.0.1'}
print(type(chaine))
print(type(entier))
print(type(flottant))
print(type(booleen))
print(type(liste))
print(type(dictionnaire))

# Exercice 17.5
#!/usr/bin/env python3
import os
utilisateur = os.getenv('USER', 'inconnu')
repertoire = os.getcwd()
print(f"Utilisateur : {utilisateur}")
print(f"Répertoire : {repertoire}")
print(f"Types : str={type(utilisateur).__name__}, str={type(repertoire).__name__}")
```

Résolution pas à pas :

- **python3 --version** : interroge l'interpréteur pour afficher sa version. Confirme que Python 3 est disponible sur le poste.
- **Shebang `#!/usr/bin/env python3`** : première ligne du script, indique au noyau d'utiliser l'interpréteur Python trouvé dans le PATH.
- **chmod 755 hello.py** : rend le script exécutable. Le préfixe `./` est requis car le répertoire courant n'est généralement pas dans le PATH.
- **f-string** : `f"Utilisateur : {nom}"` remplace `{nom}` par la valeur de la variable au moment de l'affichage.
- **type()** : retourne le type de l'objet (str, int, float, bool, list, dict), utile pour vérifier le typage dynamique.
- **os.getenv('USER', 'inconnu')** : lit la variable d'environnement USER avec une valeur de secours, évitant une erreur si la variable est absente.
- **os.getcwd()** : retourne le répertoire de travail courant du processus Python.

Bonnes pratiques

Pratique	Description
Shebang	Toujours commencer par `#!/usr/bin/env python3`
Droits	`chmod 755` pour rendre le script exécutable
f-strings	Préférer les f-strings à la concaténation pour l'interpolation
Valeur par défaut	Utiliser `os.getenv('VAR', 'défaut')` pour les variables d'environnement

BONNES PRATIQUES

Le shebang avec `env` est préférable à un chemin fixe car il s'adapte à l'installation de Python sur chaque poste. Toujours rendre le script exécutable avec `chmod 755` avant de le lancer avec `./`.

Exercices — Chapitre 17

Exercice 17.6 — Script de lecture d'un fichier avec gestion d'erreur (10min)

Écrire un script qui tente d'ouvrir un fichier et affiche un message si le fichier n'existe pas, en utilisant `try/except`.

- Quelle exception est levée si le fichier n'existe pas ?
- Quel bloc s'exécute en cas d'erreur ?

Solution

Solution

```
#!/usr/bin/env python3
chemin = '/tmp/fichier_inexistant.txt'
try:
    with open(chemin, 'r') as f:
        print(f.read())
except FileNotFoundError:
    # FileNotFoundError : exception levée lorsque le fichier n'existe pas
    print(f"Le fichier {chemin} n'existe pas")
# try : bloc contenant le code susceptible de lever une exception
# except : bloc exécuté si l'exception spécifiée se produit
# with : garantit la fermeture automatique du fichier
# open(chemin, 'r') : ouvre le fichier en lecture
```

Résolution pas à pas :

- L'exception levée est `FileNotFoundError` lorsque le fichier n'existe pas.
- Le bloc `except FileNotFoundError` s'exécute et affiche le message d'erreur, évitant l'arrêt brutal du script.

- L'exception FileNotFoundError est identifiée
- Le bloc except s'exécute en cas d'erreur

Ce qu'on a retenu

CE QU'ON A RETENU

Python est un langage interprété à typage dynamique. Le shebang `#!/usr/bin/env python3` rend le script portable. `chmod 755` rend le script exécutable. Les types de base sont str, int, float, bool, list et dict. Les f-strings permettent l'interpolation de variables. `try/except` gère les erreurs de manière contrôlée.

Synthèse

Couverture Jour 3

Chapitre	Thème
9	Présentation de l'environnement PowerShell
10	Cmdlets, pipeline, variables et syntaxe de base
11	Exécution d'un script et gestion des politiques d'exécution
12	Structures de contrôle
17	Introduction à Python pour l'administration système

Mémo — Commandes Essentielles

```
# Découverte PowerShell
Get-Command -Name Get-*   # Liste les cmdlets par motif de nom
Get-Help Get-Service      # Affiche l'aide d'une cmdlet
Get-Member                # Introspecte les propriétés d'un objet
Get-Verb                  # Liste les verbes approuvés

# Variables et données PowerShell
$nom = 'Valeur'           # Déclaration de variable
[int]$age = 35            # Typage explicite
@()                       # Création de tableau
@{ }                      # Création de hashtable

# Pipeline et comparaison PowerShell
Where-Object { condition } # Filtre des objets
Format-Table -AutoSize    # Affichage en tableau ajusté
-eq, -ne, -gt, -lt        # Opérateurs de comparaison
-like, -match             # Correspondance de motif

# Exécution et sécurité PowerShell
Get-ExecutionPolicy       # Consulte la politique d'exécution
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser  # Modifie la politique
& '.\script.ps1'          # Exécute un script (opérateur d'appel)
. '.\script.ps1'          # Point-source (import dans le scope courant)

# Structures de contrôle PowerShell
if / elseif / else        # Condition
switch                    # Sélection multiple
for / foreach / while / do-while  # Boucles
break, continue           # Contrôle de flux d'exécution
```

```
# Python
python3 --version         # Affiche la version de Python
which python3             # Localise l'interpréteur Python
#!/usr/bin/env python3    # Shebang portable
print("message")          # Affiche sur la sortie standard
chmod 755 script.py       # Rend le script exécutable
./script.py               # Exécute le script
type(objet)               # Retourne le type d'un objet
f"{var}"                  # f-string avec interpolation
try/except                # Gestion d'erreurs
import os                 # Module d'interaction avec le système
os.getenv('VAR', 'défaut')# Lit une variable d'environnement
os.getcwd()               # Retourne le répertoire courant
```

Ressources supplémentaires

Outils utilisés

Outil	Version	Usage
PowerShell	7+	Environnement de scripting et d'administration système

Scripting Bash et PowerShell — JOUR 4

PowerShell (2/2) — Fichiers, objets, processus, droits et cas pratique (08/11/2026)

Objectifs du Jour

- Maîtriser la manipulation de fichiers et dossiers avec les cmdlets New-Item, Copy-Item, Move-Item, Rename-Item, Remove-Item, Test-Path et Get-ChildItem.
- Comprendre le modèle objet de PowerShell et utiliser Get-Member, Select-Object, Where-Object, ForEach-Object, Format-*, Export-Csv et ConvertTo-Json.
- Gérer les processus, les services, les listes de contrôle d'accès (ACL) et les utilisateurs/groupes locaux.
- Concevoir un script PowerShell complet avec un bloc param, une gestion d'erreurs (try/catch, trap, $ErrorActionPreference) et une journalisation dans un fichier.

Planning du Jour

Chapitre	Horaires	Thème	Durée
13	9h00-10h30	Manipulation de fichiers et dossiers	1h30
—	10h30-10h45	Pause	15min
14	10h45-12h00	Notions d'objets PowerShell (approche objet VS Bash)	1h15
—	12h00-13h00	Pause déjeuner	1h
15	13h00-14h30	Gestion des processus et des droits	1h30
—	14h30-14h45	Pause	15min
16	14h45-16h00	Cas pratique PowerShell	1h15

Chapitre 13 — Manipulation de fichiers et dossiers (9h00-10h30)

Objectifs

- Créer et supprimer des fichiers et dossiers avec New-Item et Remove-Item en précisant le type d'item (-ItemType).

- Vérifier l'existence d'un chemin avec Test-Path et lister le contenu d'un répertoire avec Get-ChildItem.
- Copier, déplacer, renommer des items et gérer la lecture/écriture de contenu (Copy-Item, Move-Item, Rename-Item, Get-Content, Set-Content, Add-Content) en maîtrisant l'encodage UTF-8.

Contenu Théorique (45min)

1. Création et suppression d'items (New-Item, Remove-Item) (15min)

La cmdlet New-Item crée un nouvel élément dans le système de fichiers : un fichier, un dossier ou un lien symbolique, selon la valeur du paramètre -ItemType. Remove-Item supprime un élément de manière définitive.

```
# Création d'un dossier de travail
New-Item -Path 'C:\Travail\Rapports' -ItemType Directory
# New-Item : crée un nouvel élément dans le système de fichiers
# -Path : chemin de l'élément à créer
# 'C:\Travail\Rapports' : chemin cible du dossier
# -ItemType Directory : précise qu'il s'agit d'un répertoire (valeurs possibles : File, Directory, SymbolicLink)

# Création d'un fichier texte
New-Item -Path 'C:\Travail\Rapports\journal.txt' -ItemType File
# -ItemType File : précise qu'il s'agit d'un fichier texte
```

```
# Suppression définitive du fichier
Remove-Item -Path 'C:\Travail\Rapports\journal.txt'
# Remove-Item : supprime l'élément situé au chemin indiqué
# -Path : cible de la suppression
# Piège : la suppression est définitive, sans passage par la corbeille
```

CONCEPT CLÉ

New-Item distingue le type d'élément à créer via le paramètre -ItemType (File, Directory, SymbolicLink). Sans ce paramètre, PowerShell tente de déduire le type, ce qui peut conduire à des résultats inattendus. Il est donc recommandé de toujours expliciter -ItemType dans un script d'administration.

BON À SAVOIR

Remove-Item accepte le paramètre -Recurse pour supprimer un dossier et tout son contenu de manière récursive, et -Force pour supprimer des fichiers en lecture seule ou masqués. L'utilisation combinée de -Recurse et -Force doit être réservée aux cas où la suppression totale est réellement souhaitée.

2. Copie, déplacement et renommage (Copy-Item, Move-Item, Rename-Item) (15min)

Ces trois cmdlets permettent de réorganiser le système de fichiers sans en modifier le contenu. Copy-Item duplique l'élément source vers une destination, Move-Item le déplace (copie puis suppression de l'original), et Rename-Item modifie uniquement son nom.

```
# Copie du fichier vers une destination
Copy-Item -Path 'C:\Travail\Rapports\journal.txt' -Destination 'C:\Sauvegarde\journal.txt'
# Copy-Item : duplique un fichier ou un dossier vers une nouvelle destination
# -Path : chemin de l'élément source
# -Destination : chemin de l'élément cible
```

```
# Déplacement du fichier vers une archive
Move-Item -Path 'C:\Travail\Rapports\journal.txt' -Destination 'C:\Archive\journal.txt'
# Move-Item : déplace un élément d'un emplacement à un autre (équivalent d'un copier-supprimer)
# L'original n'existe plus dans le répertoire source après le déplacement
```

```
# Renommage du fichier sans déplacement
Rename-Item -Path 'C:\Archive\journal.txt' -NewName 'journal_2026.txt'
# Rename-Item : renomme un élément sans le déplacer
# -NewName : nouveau nom de l'élément, le chemin du dossier reste inchangé
```

PIÈGE

Si le fichier de destination existe déjà, Copy-Item et Move-Item peuvent lever une erreur ou écraser le fichier selon les paramètres et la version de PowerShell. Utiliser -Force pour forcer l'écrasement en connaissance de cause.

CONTRE-MESURE

Toujours vérifier l'existence de la destination avec Test-Path avant une copie ou un déplacement, ou utiliser -Force en connaissance de cause. Le paramètre -WhatIf permet également de prévisualiser l'effet de la commande sans l'exécuter.

3. Vérification et listage (Test-Path, Get-ChildItem) (15min)

Test-Path détermine si un chemin existe et retourne une valeur booléenne. Get-ChildItem liste le contenu d'un répertoire (fichiers et sous-dossiers) sous forme d'objets.

```
# Vérification de l'existence d'un fichier
Test-Path -Path 'C:\Travail\Rapports\journal.txt'
# Test-Path : retourne Vrai ($true) si le chemin existe, Faux ($false) sinon
# -Path : chemin à vérifier
# Résultat attendu : Vrai ($true) si le fichier existe, Faux ($false) sinon
```

```
# Listage du contenu d'un répertoire
Get-ChildItem -Path 'C:\Travail\Rapports'
# Get-ChildItem : liste les éléments d'un répertoire (fichiers et dossiers)
# -Path : répertoire à lister
# Retourne des objets FileInfo et DirectoryInfo dont les propriétés (Name, Length, CreationTime) sont acce
```

CONCEPT CLÉ

Test-Path retourne une valeur booléenne, directement utilisable dans une condition if. Get-ChildItem retourne des objets, pas du texte : leurs propriétés (Name, Length, CreationTime) sont accessibles via Get-Member, ce qui illustre l'approche objet de PowerShell.

4. Lecture et écriture de contenu, encodage UTF-8 (Get-Content, Set-Content, Add-Content) (15min)

Get-Content lit le contenu d'un fichier et le retourne ligne par ligne. Set-Content écrit (ou écrase) le contenu d'un fichier. Add-Content ajoute du contenu à la fin d'un fichier existant sans l'écraser.

```
# Lecture du contenu d'un fichier
Get-Content -Path 'C:\Travail\Rapports\journal.txt'
# Get-Content : lit le contenu d'un fichier et le retourne ligne par ligne
# -Path : chemin du fichier à lire
# Résultat attendu : chaque ligne du fichier s'affiche dans la console
```

```
# Écriture (écrasement) du contenu d'un fichier
Set-Content -Path 'C:\Travail\Rapports\journal.txt' -Value 'Première ligne de rapport.' -Encoding UTF8
# Set-Content : écrit le contenu dans un fichier, en écrasant l'existant s'il est présent
# -Value : contenu à écrire dans le fichier
# -Encoding UTF8 : force l'encodage UTF-8 (sans BOM dans PowerShell 7+)
```

```
# Ajout de contenu à la fin du fichier sans écrasement
Add-Content -Path 'C:\Travail\Rapports\journal.txt' -Value 'Deuxième ligne ajoutée.' -Encoding UTF8
# Add-Content : ajoute du contenu à la fin du fichier existant
# Contrairement à Set-Content, Add-Content préserve le contenu déjà présent
# -Encoding UTF8 : force l'encodage UTF-8 (sans BOM dans PowerShell 7+)
```

L'encodage des fichiers texte est un enjeu d'administration système. Par défaut, PowerShell peut utiliser l'encodage historique de Windows (UTF-16 LE avec BOM ou ANSI selon la version). Pour garantir l'interopérabilité, notamment avec Linux et les outils modernes, il est recommandé d'utiliser l'encodage UTF-8 sans BOM via le paramètre -Encoding. L'encodage UTF-8 constitue le standard universel pour l'échange de fichiers texte entre systèmes hétérogènes (voir The Linux Command Line, William Shotts, p. 261, qui présente UTF-8 comme l'encodage de référence des environnements Linux modernes).

```
# Écriture avec encodage UTF-8 explicite
Set-Content -Path 'C:\Travail\Rapports\journal.txt' -Value 'Rapport UTF-8.' -Encoding UTF8
# -Encoding UTF8 : force l'encodage UTF-8 (sans BOM dans PowerShell 7+)
# Garantit l'interopérabilité avec les outils Linux et les applications modernes
```

> **BON À SAVOIR**
>
> Dans PowerShell 7+, -Encoding UTF8 produit un fichier UTF-8 sans marque BOM (Byte Order Mark), ce qui facilite les échanges avec des outils Linux. Dans Windows PowerShell 5.1, UTF8 peut inclure un BOM ; utiliser utf8NoBOM pour l'éviter.

> **PIÈGE**
>
> Un fichier encodé en UTF-16 lu par un outil s'attendant à de l'UTF-8 affichera des caractères illisibles (mojibake). Toujours vérifier l'encodage avant d'échanger des fichiers entre systèmes.

Fig 1 — Flux de manipulation de fichiers et dossiers

Diagramme		
B	→	
C	→	
D	→	
E	→	
F	→	
G	→	
H	→	
I	→	J

Figure — Diagramme Mermaid

Exemple concret de séquence complète :

```
# Création d'un dossier de travail
New-Item -Path 'C:\Travail\Rapport' -ItemType Directory
# New-Item : crée le répertoire de travail
# -ItemType Directory : précise qu'il s'agit d'un dossier

# Vérification de l'existence du dossier
if (Test-Path -Path 'C:\Travail\Rapport') {
    Write-Host 'Le dossier existe.'
}
# Test-Path : retourne $true si le chemin existe
# Write-Host : affiche un message à la console

# Création d'un fichier et écriture de la première ligne
Set-Content -Path 'C:\Travail\Rapport\journal.txt' -Value 'Début du rapport.' -Encoding UTF8
# Set-Content : crée le fichier et écrit la première ligne
# -Encoding UTF8 : encodage UTF-8 sans BOM (PowerShell 7+)

# Ajout d'une ligne supplémentaire à la fin du fichier
Add-Content -Path 'C:\Travail\Rapport\journal.txt' -Value 'Étape suivante.' -Encoding UTF8
# Add-Content : ajoute du contenu sans écraser l'existant

# Lecture et affichage du contenu complet du fichier
Get-Content -Path 'C:\Travail\Rapport\journal.txt'
# Get-Content : affiche chaque ligne du fichier dans la console

# Copie de sauvegarde du fichier
Copy-Item -Path 'C:\Travail\Rapport\journal.txt' -Destination 'C:\Sauvegarde\journal.txt'
# Copy-Item : duplique le fichier vers le dossier de sauvegarde

# Suppression du fichier original du dossier de travail
Remove-Item -Path 'C:\Travail\Rapport\journal.txt'
# Remove-Item : supprime définitivement le fichier original
```

Résultat attendu : le fichier journal.txt est créé, lu, copié, puis supprimé du dossier de travail. La copie de sauvegarde persiste dans C:\Sauvegarde.

Lab 13 — Manipulation de fichiers et dossiers (45min)

Environnement

Élément	Valeur
Poste	Windows avec PowerShell 5.1 ou PowerShell 7+
Répertoire de travail	C:\Travail\Lab13 (à créer)
Prérequis	Droits d'écriture sur C:\Travail

Exercices (les exercices s'enchaînent pour former un lab complet)

Exercice 13.1 — Créer un dossier et vérifier son existence (10min)

Créer le répertoire C:\Travail\Lab13, puis vérifier qu'il existe avec Test-Path.

```
# Création du répertoire de travail du lab
New-Item -Path 'C:\Travail\Lab13' -ItemType Directory
# New-Item : crée un nouvel élément dans le système de fichiers
# -Path : chemin de l'élément à créer
# 'C:\Travail\Lab13' : chemin cible du répertoire
# -ItemType Directory : précise qu'il s'agit d'un dossier

# Vérification de l'existence du répertoire
Test-Path -Path 'C:\Travail\Lab13'
# Test-Path : détermine si un chemin existe
# -Path : chemin à vérifier
# Retourne Vrai ($true) si le chemin existe, Faux ($false) sinon
# Résultat attendu : Vrai ($true)
```

- Le dossier C:\Travail\Lab13 est créé
- Test-Path retourne $true

Exercice 13.2 — Créer, copier et déplacer un fichier (15min)

Créer un fichier texte dans Lab13, en faire une copie dans un sous-dossier Sauvegarde, puis déplacer l'original vers un sous-dossier Archive.

```
# Création du fichier texte avec encodage UTF-8
Set-Content -Path 'C:\Travail\Lab13\note.txt' -Value 'Contenu de test.' -Encoding UTF8
# Set-Content : crée le fichier et y écrit le contenu
# -Path : chemin du fichier à créer
# -Value : contenu à écrire dans le fichier
# -Encoding UTF8 : encodage UTF-8 sans BOM (PowerShell 7+)

# Création du sous-dossier Sauvegarde
New-Item -Path 'C:\Travail\Lab13\Sauvegarde' -ItemType Directory
# New-Item : crée le sous-dossier de sauvegarde
# -ItemType Directory : précise qu'il s'agit d'un dossier

# Copie du fichier vers le sous-dossier Sauvegarde
Copy-Item -Path 'C:\Travail\Lab13\note.txt' -Destination 'C:\Travail\Lab13\Sauvegarde\note.txt'
# Copy-Item : duplique un fichier ou un dossier vers une nouvelle destination
# -Path : chemin de l'élément source
# -Destination : chemin de l'élément cible
# L'original reste présent dans Lab13 après la copie

# Création du sous-dossier Archive
New-Item -Path 'C:\Travail\Lab13\Archive' -ItemType Directory
# New-Item : crée le sous-dossier d'archive
# -ItemType Directory : précise qu'il s'agit d'un dossier

# Déplacement de l'original vers le sous-dossier Archive
Move-Item -Path 'C:\Travail\Lab13\note.txt' -Destination 'C:\Travail\Lab13\Archive\note.txt'
# Move-Item : déplace un élément d'un emplacement à un autre
# -Path : chemin de l'élément source à déplacer
# -Destination : chemin de l'élément cible
# Après le déplacement, l'original n'existe plus dans Lab13
```

- Le fichier note.txt existe dans le sous-dossier Sauvegarde
- Le fichier note.txt a été déplacé dans Archive (plus présent dans Lab13)

Exercice 13.3 — Renommer et supprimer (10min)

Renommer le fichier déplacé dans Archive, puis le supprimer définitivement.

```
# Renommage du fichier déplacé dans Archive
Rename-Item -Path 'C:\Travail\Lab13\Archive\note.txt' -NewName 'note_archive.txt'
# Rename-Item : renomme un élément sans le déplacer
# -Path : chemin de l'élément à renommer
# -NewName : nouveau nom de l'élément
# Le chemin du dossier reste inchangé, seul le nom change

# Suppression définitive du fichier renommé
Remove-Item -Path 'C:\Travail\Lab13\Archive\note_archive.txt'
# Remove-Item : supprime l'élément situé au chemin indiqué
# -Path : cible de la suppression
# La suppression est définitive, sans passage par la corbeille
```

- Le fichier est renommé en note_archive.txt
- Le fichier est supprimé (Test-Path retourne $false)

Exercice 13.4 — Lister, lire et ajouter du contenu (10min)

Lister le contenu de Lab13, créer un fichier de rapport, y ajouter plusieurs lignes et en afficher le contenu.

```
# Listage du contenu de Lab13
Get-ChildItem -Path 'C:\Travail\Lab13'
# Get-ChildItem : liste les éléments d'un répertoire (fichiers et dossiers)
# -Path : répertoire à lister
# Retourne des objets FileInfo et DirectoryInfo
# Résultat attendu : affichage des sous-dossiers Sauvegarde et Archive

# Création d'un fichier de rapport avec première ligne
Set-Content -Path 'C:\Travail\Lab13\rapport.txt' -Value 'Ligne 1 du rapport.' -Encoding UTF8
# Set-Content : crée le fichier et écrit la première ligne
# -Encoding UTF8 : encodage UTF-8 sans BOM (PowerShell 7+)

# Ajout de lignes supplémentaires à la fin du fichier
Add-Content -Path 'C:\Travail\Lab13\rapport.txt' -Value 'Ligne 2 ajoutée.' -Encoding UTF8
# Add-Content : ajoute du contenu à la fin du fichier existant sans écraser
Add-Content -Path 'C:\Travail\Lab13\rapport.txt' -Value 'Ligne 3 ajoutée.' -Encoding UTF8

# Lecture et affichage du contenu complet du fichier
Get-Content -Path 'C:\Travail\Lab13\rapport.txt'
# Get-Content : lit le contenu d'un fichier et l'affiche ligne par ligne
# Résultat attendu : affichage des trois lignes du rapport
```

- Le listage affiche les sous-dossiers Sauvegarde et Archive
- Le fichier rapport.txt contient trois lignes
- L'encodage du fichier est UTF-8

Démonstration commentée

Exemple de séquence complète avec interprétation ligne par ligne :

```
# 1. Création de l'arborescence de démonstration
New-Item -Path 'C:\Demo\Logs' -ItemType Directory
# New-Item : crée le répertoire de destination pour les journaux
# -Path : chemin du répertoire à créer
# -ItemType Directory : précise qu'il s'agit d'un dossier

# 2. Écriture de la première entrée du journal d'activité
Set-Content -Path 'C:\Demo\Logs\activite.log' -Value "$(Get-Date) : démarrage du service." -Encoding UTF8
# Set-Content : crée le fichier et écrit la première ligne, en écrasant l'existant
# -Path : chemin du fichier cible
# -Value : contenu à écrire, ici une chaîne avec opérateur de sous-expression
# $(Get-Date) : opérateur de sous-expression, exécute Get-Date et insère le résultat (date et heure courant
# -Encoding UTF8 : encodage UTF-8 sans BOM (PowerShell 7+)

# 3. Ajout d'une deuxième entrée à la fin du journal
Add-Content -Path 'C:\Demo\Logs\activite.log' -Value "$(Get-Date) : opération terminée." -Encoding UTF8
# Add-Content : ajoute du contenu à la fin du fichier sans écraser l'existant
# -Path : chemin du fichier cible
# -Value : contenu à ajouter, avec horodatage
# -Encoding UTF8 : encodage UTF-8 sans BOM (PowerShell 7+)

# 4. Lecture et affichage du contenu complet du journal
Get-Content -Path 'C:\Demo\Logs\activite.log'
# Get-Content : lit le contenu du fichier et l'affiche ligne par ligne dans la console
# -Path : chemin du fichier à lire
# Résultat attendu : deux lignes horodatées s'affichent

# 5. Sauvegarde du journal avant nettoyage
Copy-Item -Path 'C:\Demo\Logs\activite.log' -Destination 'C:\Demo\Logs\activite.bak'
# Copy-Item : duplique le fichier journal vers une copie de sauvegarde
# -Path : chemin de l'élément source
# -Destination : chemin de l'élément cible (copie de sauvegarde)
# L'original activite.log existe toujours après cette opération

# 6. Suppression du fichier original après sauvegarde
Remove-Item -Path 'C:\Demo\Logs\activite.log'
# Remove-Item : supprime définitivement le fichier original
# -Path : cible de la suppression
# La suppression est définitive, sans passage par la corbeille
```

Résultat attendu : le fichier activite.log contient deux lignes horodatées (démarrage et opération terminée), puis est copié en activite.bak avant d'être supprimé. La sauvegarde activite.bak persiste dans C:\Demo\Logs.

Interprétation ligne par ligne :

- New-Item crée le répertoire C:\Demo\Logs qui servira de conteneur aux journaux.
- Set-Content crée activite.log et y écrit la première ligne horodatée. L'opérateur $(Get-Date) est évalué avant l'assignation, produisant une date au format local.
- Add-Content ajoute une deuxième ligne à la fin du fichier sans effacer la première, grâce à l'ajout en mode append.
- Get-Content lit et affiche les deux lignes du journal, confirmant que l'écriture et l'ajout ont fonctionné.
- Copy-Item duplique activite.log en activite.bak, constituant une sauvegarde avant toute modification destructive.
- Remove-Item supprime activite.log de manière définitive. Seul activite.bak persiste, démontrant la séparation entre copie et suppression.

Correction et résolution pas à pas

Solution

Solution du Lab 13

```
# Exercice 13.1 — Créer un dossier et vérifier son existence
New-Item -Path 'C:\Travail\Lab13' -ItemType Directory
# New-Item : crée le répertoire de travail
# -Path : chemin du répertoire à créer
# -ItemType Directory : précise qu'il s'agit d'un dossier

Test-Path -Path 'C:\Travail\Lab13'
# Test-Path : vérifie l'existence du répertoire
# -Path : chemin à vérifier
# Résultat attendu : Vrai ($true)

# Exercice 13.2 — Créer, copier et déplacer un fichier
Set-Content -Path 'C:\Travail\Lab13\note.txt' -Value 'Contenu de test.' -Encoding UTF8
# Set-Content : crée le fichier et écrit le contenu
# -Path : chemin du fichier
# -Value : contenu à écrire
# -Encoding UTF8 : encodage UTF-8 sans BOM (PowerShell 7+)

New-Item -Path 'C:\Travail\Lab13\Sauvegarde' -ItemType Directory
# New-Item : crée le sous-dossier Sauvegarde
# -ItemType Directory : précise qu'il s'agit d'un dossier

Copy-Item -Path 'C:\Travail\Lab13\note.txt' -Destination 'C:\Travail\Lab13\Sauvegarde\note.txt
# Copy-Item : duplique le fichier vers Sauvegarde
# -Path : chemin de l'élément source
# -Destination : chemin de l'élément cible

New-Item -Path 'C:\Travail\Lab13\Archive' -ItemType Directory
# New-Item : crée le sous-dossier Archive
# -ItemType Directory : précise qu'il s'agit d'un dossier

Move-Item -Path 'C:\Travail\Lab13\note.txt' -Destination 'C:\Travail\Lab13\Archive\note.txt'
# Move-Item : déplace l'original vers Archive
# -Path : chemin de l'élément source
# -Destination : chemin de l'élément cible

# Exercice 13.3 — Renommer et supprimer
Rename-Item -Path 'C:\Travail\Lab13\Archive\note.txt' -NewName 'note_archive.txt'
# Rename-Item : renomme le fichier sans le déplacer
# -Path : chemin de l'élément à renommer
# -NewName : nouveau nom de l'élément

Remove-Item -Path 'C:\Travail\Lab13\Archive\note_archive.txt'
# Remove-Item : supprime définitivement le fichier
# -Path : cible de la suppression

# Exercice 13.4 — Lister, lire et ajouter du contenu
Get-ChildItem -Path 'C:\Travail\Lab13'
# Get-ChildItem : liste les éléments du répertoire
# -Path : répertoire à lister

Set-Content -Path 'C:\Travail\Lab13\rapport.txt' -Value 'Ligne 1 du rapport.' -Encoding UTF8
# Set-Content : crée le fichier de rapport et écrit la première ligne

Add-Content -Path 'C:\Travail\Lab13\rapport.txt' -Value 'Ligne 2 ajoutée.' -Encoding UTF8
# Add-Content : ajoute une ligne à la fin sans écraser

Add-Content -Path 'C:\Travail\Lab13\rapport.txt' -Value 'Ligne 3 ajoutée.' -Encoding UTF8
# Add-Content : ajoute une deuxième ligne à la fin sans écraser
```

```
Get-Content -Path 'C:\Travail\Lab13\rapport.txt'
# Get-Content : lit et affiche le contenu complet du fichier
```

Résolution pas à pas :

- **New-Item -Path 'C:\Travail\Lab13' -ItemType Directory** : crée le répertoire de travail du lab. Le paramètre -ItemType Directory indique explicitement qu'il s'agit d'un dossier, évitant toute ambiguïté sur le type d'élément créé.
- **Test-Path -Path 'C:\Travail\Lab13'** : retourne $true si le chemin existe. Cette valeur booléenne est directement utilisable dans une condition if pour conditionner la suite du traitement.
- **Set-Content -Path 'C:\Travail\Lab13\note.txt' -Value 'Contenu de test.' -Encoding UTF8** : crée le fichier note.txt et y écrit la première ligne. L'option -Encoding UTF8 garantit un encodage interopérable, sans BOM dans PowerShell 7+.
- **New-Item -Path 'C:\Travail\Lab13\Sauvegarde' -ItemType Directory** : crée le sous-dossier Sauvegarde qui recevra la copie du fichier.
- **Copy-Item -Path 'C:\Travail\Lab13\note.txt' -Destination 'C:\Travail\Lab13\Sauvegarde\note.txt'** : duplique le fichier vers Sauvegarde. L'original reste intact dans Lab13, ce qui distingue la copie du déplacement.
- **New-Item -Path 'C:\Travail\Lab13\Archive' -ItemType Directory** : crée le sous-dossier Archive qui recevra l'original déplacé.
- **Move-Item -Path 'C:\Travail\Lab13\note.txt' -Destination 'C:\Travail\Lab13\Archive\note.txt'** : déplace l'original vers Archive. Après cette opération, le fichier n'existe plus dans Lab13 mais uniquement dans Archive.
- **Rename-Item -Path 'C:\Travail\Lab13\Archive\note.txt' -NewName 'note_archive.txt'** : modifie uniquement le nom du fichier. Le chemin du dossier (Archive) reste inchangé, seul le nom de l'élément est modifié.
- **Remove-Item -Path 'C:\Travail\Lab13\Archive\note_archive.txt'** : supprime définitivement le fichier. Contrairement à une suppression via l'explorateur Windows, le fichier ne passe pas par la corbeille ; l'opération est irréversible.
- **Get-ChildItem -Path 'C:\Travail\Lab13'** : liste le contenu du répertoire de travail. Retourne des objets FileInfo et DirectoryInfo dont les propriétés (Name, Length, CreationTime) sont introspectables avec Get-Member.
- **Set-Content -Path 'C:\Travail\Lab13\rapport.txt' -Value 'Ligne 1 du rapport.' -Encoding UTF8** : crée le fichier rapport.txt et écrit la première ligne en encodage UTF-8.
- **Add-Content -Path 'C:\Travail\Lab13\rapport.txt' -Value 'Ligne 2 ajoutée.' -Encoding UTF8** : ajoute une deuxième ligne à la fin du fichier sans écraser le contenu existant.
- **Add-Content -Path 'C:\Travail\Lab13\rapport.txt' -Value 'Ligne 3 ajoutée.' -Encoding UTF8** : ajoute une troisième ligne à la fin du fichier, préservant les deux lignes précédentes.
- **Get-Content -Path 'C:\Travail\Lab13\rapport.txt'** : lit et affiche les trois lignes du fichier, confirmant que l'ajout sélectif a fonctionné comme attendu.

Bonnes pratiques

Pratique	Description
Vérification avant suppression	Utiliser Test-Path avant Remove-Item pour éviter les erreurs
Encodage UTF-8	Privilégier -Encoding UTF8 pour l'interopérabilité
Paramètre -WhatIf	Utiliser -WhatIf pour prévisualiser une suppression ou un déplacement
Chemins absolus	Préférer les chemins absolus dans les scripts d'administration
Séparation copie/suppression	Privilégier Copy-Item puis Remove-Item plutôt qu'un déplacement non vérifié

Script de vérification avant suppression sécurisée :

```
# Vérification de l'existence avant suppression sécurisée
$chemin = 'C:\Travail\Lab13\note.txt'
if (Test-Path -Path $chemin) {
    Remove-Item -Path $chemin -WhatIf
    # -WhatIf : prévisualise l'effet de la suppression sans l'exécuter
    # Retirer -WhatIf pour effectuer la suppression effective
} else {
    Write-Host 'Le fichier n'existe pas, aucune suppression nécessaire.'
}
# Test-Path : vérifie l'existence du fichier avant toute tentative de suppression
# Remove-Item -WhatIf : affiche ce qui se passerait sans supprimer réellement
# Write-Host : informe l'utilisateur lorsque le fichier est absent
```

BONNES PRATIQUES

Avant toute suppression ou modification en masse, utiliser le paramètre -WhatIf (disponible sur Remove-Item, Move-Item, Copy-Item) pour prévisualiser l'effet de la commande sans l'exécuter. Privilégier l'encodage UTF-8 pour garantir la lisibilité des fichiers sur différents systèmes. Toujours vérifier l'existence d'un chemin avec Test-Path avant d'appliquer Remove-Item, afin d'éviter les erreurs d'exécution et de traiter explicitement le cas où l'élément est absent.

Exercices — Chapitre 13

Exercice 13.5 — Vérification d'existence conditionnelle (10min)

Écrire une condition qui crée un fichier uniquement s'il n'existe pas déjà, puis affiche un message confirmant la création ou l'existence.

- Quelle cmdlet utiliser pour tester l'existence ?
- Quelle cmdlet utiliser pour créer le fichier sans écraser l'existant ?

Solution

Solution

```
$chemin = 'C:\Travail\Lab13\conditionnel.txt'
if (-not (Test-Path -Path $chemin)) {
    Set-Content -Path $chemin -Value 'Fichier créé conditionnellement.' -Encoding UTF8
    Write-Host 'Fichier créé.'
} else {
    Write-Host 'Le fichier existe déjà.'
}
# -not : opérateur de négation booléenne
# Test-Path : retourne $true si le fichier existe, $false sinon
# Set-Content : crée le fichier et y écrit le contenu
# -Encoding UTF8 : encodage UTF-8 sans BOM (PowerShell 7+)
# Write-Host : affiche un message d'information à la console
```

Résolution pas à pas :

- L'opérateur -not inverse le résultat de Test-Path. Si le fichier n'existe pas, Test-Path retourne $false, et -not le transforme en $true, déclenchant l'exécution du bloc if.
- Set-Content crée le fichier et y écrit le contenu. L'option -Encoding UTF8 garantit un encodage interopérable.
- Si le fichier existait déjà, Test-Path retourne $true, -not le transforme en $false, et le bloc else s'exécute : un message d'information s'affiche sans écraser le fichier existant.
- Write-Host affiche le message de confirmation approprié selon la branche exécutée (création ou existence déjà avérée).

Ce qu'on a retenu

CE QU'ON A RETENU

New-Item crée des fichiers et dossiers selon le paramètre -ItemType (File, Directory, SymbolicLink). Test-Path vérifie l'existence d'un chemin et retourne une valeur booléenne ($true/$false) directement utilisable dans une condition. Get-ChildItem liste le contenu d'un répertoire sous forme d'objets (FileInfo, DirectoryInfo), introspectables avec Get-Member. Copy-Item, Move-Item, Rename-Item et Remove-Item permettent de dupliquer, déplacer, renommer et supprimer des éléments ; la suppression par Remove-Item est définitive, sans passage par la corbeille. Get-Content, Set-Content et Add-Content gèrent la lecture et l'écriture de contenu, avec le paramètre -Encoding UTF8 pour maîtriser l'encodage des fichiers texte et garantir l'interopérabilité entre systèmes.

Chapitre 14 — Notions d'objets PowerShell (10h45-12h00)

Objectifs

- Comprendre que PowerShell manipule des objets .NET typés et non du texte brut, contrairement à Bash qui traite des flux de caractères.
- Inspecter les membres d'un objet (propriétés et méthodes) avec Get-Member, et transformer des objets avec Select-Object, Where-Object et ForEach-Object.
- Formater l'affichage avec Format-Table, Format-List et Format-Wide, et sérialiser des objets vers CSV et JSON avec Export-Csv et ConvertTo-Json.

Contenu Théorique (45min)

1. Tout est objet : propriétés, méthodes et introspection avec Get-Member (15min)

Dans PowerShell, chaque commande (cmdlet) retourne un objet .NET typé, et non une chaîne de caractères. Un objet est une structure de données composée de propriétés (données décrivant l'objet) et de méthodes (actions que l'objet peut exécuter). Par exemple, un objet représentant un service Windows possède une propriété `Status` (état du service) et une méthode `Stop()` (arrêter le service).

Contrairement à Bash, où la sortie d'une commande est du texte qu'il faut analyser avec des outils comme `grep`, `awk` ou `cut` (voir The Linux Command Line, William Shotts, p. 281, qui présente ces outils comme des programmes permettant de découper et recombiner des colonnes de texte), PowerShell préserve la structure des données tout au long du pipeline. Chaque objet conserve ses propriétés, accessibles par leur nom, sans dépendre de la position d'une colonne dans un affichage.

La cmdlet `Get-Member` permet l'introspection : elle liste tous les membres (propriétés et méthodes) d'un objet transmis par le pipeline.

```
Get-Service | Get-Member
# Get-Service : retourne des objets ServiceController représentant les services du système
# Get-Member : introspecte l'objet transmis et liste ses propriétés et méthodes
# Résultat attendu : affiche Name, Status, DisplayName (propriétés) et Start, Stop, Pause (méthodes)
```

Pour accéder à une propriété d'un objet, on utilise la notation pointée :

```
$service = Get-Service -Name 'Spooler'
# Get-Service -Name : récupère l'objet ServiceController du service dont le nom est Spooler
# $service : variable stockant l'objet service
$service.Status
# .Status : accède à la propriété Status de l'objet stocké dans $service
# Résultat attendu : Running ou Stopped, selon l'état du service
```

Pour appeler une méthode, on utilise également la notation pointée suivie de parenthèses :

```
$service = Get-Service -Name 'Spooler'
$service.Stop()
# .Stop() : appelle la méthode Stop de l'objet service pour l'arrêter
# Les parenthèses indiquent qu'il s'agit d'une méthode (action) et non d'une propriété (donnée)
```

CONCEPT CLÉ

En PowerShell, tout est objet. Une commande ne retourne pas du texte à analyser, mais un objet structuré dont les propriétés et méthodes sont directement accessibles par leur nom. Cela élimine le besoin de parser du texte avec grep, awk ou cut, et rend les scripts plus robustes face aux changements de format d'affichage.

BON À SAVOIR

`Get-Member` accepte le paramètre `-MemberType` pour filtrer les membres affichés. `-MemberType Property` n'affiche que les propriétés, et `-MemberType Method` n'affiche que les méthodes.

2. Sélection, filtrage et itération : Select-Object, Where-Object, ForEach-Object (15min)

Le pipeline PowerShell transporte des objets d'une cmdlet à l'autre. Trois cmdlets permettent de transformer ces objets :

- `Select-Object` choisit un sous-ensemble de propriétés (projection).
- `Where-Object` filtre les objets selon une condition (sélection).
- `ForEach-Object` exécute une action sur chaque objet du pipeline (itération).

Dans `Where-Object` et `ForEach-Object`, la variable automatique `$_` représente l'objet courant traité par le scriptblock (bloc de script délimité par des accolades `{ }`).

```
Get-Service | Select-Object -Property Name, Status, DisplayName
# Get-Service : retourne les objets ServiceController de tous les services
# Select-Object : sélectionne uniquement les propriétés indiquées, créant un nouvel objet allégé
# -Property : liste des propriétés à conserver dans l'objet de sortie
```

```
Get-Service | Where-Object { $_.Status -eq 'Running' }
# Where-Object : filtre les objets selon la condition évaluée dans le scriptblock
# $_.Status : propriété Status de l'objet courant (représenté par $_)
# -eq : opérateur d'égalité PowerShell (équivalent de == dans d'autres langages)
# Le scriptblock { $_.Status -eq 'Running' } est évalué pour chaque objet ; seuls ceux qui retournent $true
```

```
Get-Service | Where-Object { $_.Status -eq 'Running' } | ForEach-Object { Write-Host $_.DisplayName }
# ForEach-Object : exécute le scriptblock pour chaque objet transmis par le pipeline
# Write-Host : affiche une chaîne de caractères à la console, sans formatter d'objet
# $_.DisplayName : propriété nom d'affichage de l'objet courant
```

PIÈGE

La variable `$_` ne fait référence qu'à l'objet courant dans le scriptblock immédiat. Dans des pipelines imbriqués ou complexes, il est recommandé de nommer explicitement l'objet courant avec un paramètre nommé (par exemple `ForEach-Object -Process { $service = $_ }`) pour éviter toute ambiguïté.

CONTRE-MESURE

Toujours placer les opérations de sélection (`Select-Object`), de filtrage (`Where-Object`) et de traitement (`ForEach-Object`) avant toute cmdlet de formatage (`Format-*`), afin de préserver les objets exploitables par la suite.

3. Formatage et export : Format-*, Export-Csv, ConvertTo-Json (15min)

Les cmdlets `Format-Table`, `Format-List` et `Format-Wide` modifient uniquement la représentation visuelle des objets à l'écran. Elles ne modifient pas l'objet sous-jacent, mais elles transforment l'objet en un objet de formatage qui ne peut plus être transmis utilement à d'autres cmdlets. Elles doivent donc toujours être placées en fin de pipeline.

`Export-Csv` sérialise les objets vers un fichier CSV (Comma-Separated Values, valeurs séparées par des virgules), lisible par des tableurs. `ConvertTo-Json` sérialise les objets vers une chaîne JSON (JavaScript Object Notation, notation d'objets JavaScript), adaptée à l'échange de données avec des applications web ou des API.

```
Get-Service | Select-Object -First 5 -Property Name, Status | Format-Table -AutoSize
# Format-Table : affiche les objets sous forme de tableau à colonnes
# -AutoSize : ajuste automatiquement la largeur des colonnes au contenu des données
# -First 5 : limite le traitement aux cinq premiers objets du pipeline
```

```
Get-Service -Name 'Spooler' | Format-List Name, Status, DisplayName
# Format-List : affiche chaque propriété sélectionnée sur une ligne séparée, utile pour une lectur détail
```

```
Get-Service | Select-Object -First 3 -Property Name, Status | Format-Wide
# Format-Wide : affiche une seule propriété sur plusieurs colonnes, pour un affichage compact
```

```
Get-Service | Select-Object -First 5 -Property Name, Status | Export-Csv -Path 'C:\Travail\service .csv' -
# Export-Csv : sérialise les objets vers un fichier CSV
# -Path : chemin du fichier de sortie
# -NoTypeInformation : omet l'en-tête de type (.NET type) dans le fichier CSV, pour un fichier plu propre
# -Encoding UTF8 : force l'encodage UTF-8 du fichier de sortie (sans BOM dans PowerShell 7+)
```

```
Get-Service | Select-Object -First 3 -Property Name, Status | ConvertTo-Json
# ConvertTo-Json : transforme les objets en une chaîne JSON structurée
# Résultat attendu : une représentation JSON des trois premiers services, exploitable par des appl cations
```

Fig 2 — Approche objet PowerShell VS flux de texte Bash

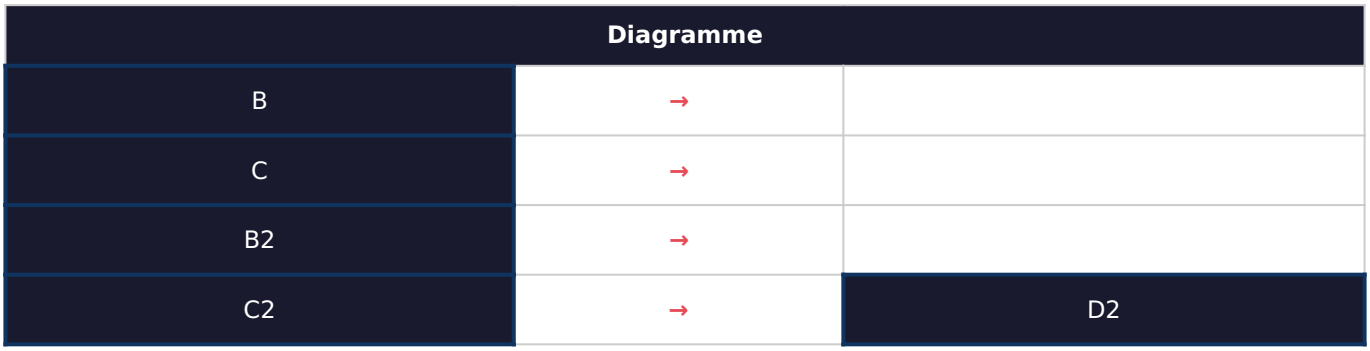


Figure — Diagramme Mermaid

Exemple concret comparant les deux approches pour extraire les services en cours d'exécution :

```
# Approche PowerShell (objets) — robuste, indépendante de l'affichage
Get-Service | Where-Object { $_.Status -eq 'Running' } | Select-Object -Property Name, DisplayName
# Les objets ServiceController conservent leurs propriétés, directement exploitables par une cmdle suivan

# Approche équivalente en Bash (texte) — fragile, dépend du format d'affichage
# systemctl --type=service --state=running | awk '{print $1, $2}'
# Le texte doit être réanalysé à chaque étape ; aucune structure n'est préservée
# MARQUAGE : commande Bash, fournie à titre de comparaison uniquement
```

Résultat attendu : PowerShell retourne des objets avec les propriétés `Name` et `DisplayName`, exploitables par une cmdlet suivante. Bash retourne du texte qui doit être reparsé à chaque étape, rendant le script vulnérable à tout changement de format de sortie.

Lab 14 — Notions d'objets PowerShell (45min)

Environnement

Élément	Valeur
Poste	Windows avec PowerShell 5.1 ou PowerShell 7+
Répertoire de travail	C:\Travail\Lab14 (à créer)
Prérequis	Droits de lecture sur les services et processus du système

Exercices (les exercices s'enchaînent pour former un lab complet)

Exercice 14.1 — Inspecter un objet avec Get-Member (10min)

Lister les propriétés et méthodes d'un objet service, puis filtrer uniquement les propriétés.

```
# Introspection complète de l'objet service Spooler
Get-Service -Name 'Spooler' | Get-Member
# Get-Service -Name : récupère l'objet ServiceController du service Spooler
# Get-Member : liste tous les membres (propriétés et méthodes) de l'objet
# Résultat attendu : Name, Status, DisplayName (propriétés), Start, Stop, Pause (méthodes)

# Introspection filtrée : uniquement les propriétés
Get-Service -Name 'Spooler' | Get-Member -MemberType Property
# -MemberType Property : filtre pour n'afficher que les propriétés de l'objet
```

- La liste affiche des propriétés telles que Name, Status, DisplayName
- La liste affiche des méthodes telles que Start, Stop

Exercice 14.2 — Sélectionner et filtrer des objets (15min)

Sélectionner les propriétés Name et Status des cinq premiers services, puis filtrer uniquement ceux qui sont en cours d'exécution.

```
# Sélection des cinq premiers services avec leurs propriétés Name et Status
Get-Service | Select-Object -First 5 -Property Name, Status
# Select-Object -First 5 : limite le pipeline aux cinq premiers objets
# -Property Name, Status : conserve uniquement ces deux propriétés dans l'objet de sortie

# Filtrage des services en cours d'exécution
Get-Service | Where-Object { $_.Status -eq 'Running' } | Select-Object -Property Name, DisplayName
# Where-Object : filtre les objets dont la propriété Status vaut 'Running'
# $_.Status : propriété Status de l'objet courant dans le pipeline
# -eq : opérateur d'égalité PowerShell
```

- Cinq services s'affichent avec Name et Status
- Seuls les services Running apparaissent après le filtrage

Exercice 14.3 — Itérer et formater (10min)

Itérer sur les trois premiers services et afficher leur nom d'affichage, puis afficher une liste détaillée.

```
# Itération sur les trois premiers services pour afficher leur nom d'affichage
Get-Service | Select-Object -First 3 | ForEach-Object { Write-Host $_.DisplayName }
# ForEach-Object : exécute le scriptblock pour chaque objet transmis
# Write-Host : affiche le nom d'affichage de chaque service à la console
# $_.DisplayName : propriété nom d'affichage de l'objet courant

# Affichage détaillé sous forme de liste
Get-Service | Select-Object -First 3 -Property Name, Status, DisplayName | Format-List
# Format-List : affiche chaque propriété sélectionnée sur une ligne séparée
```

- Trois noms d'affichage s'affichent à la console
- Le format détaillé présente chaque propriété sur une ligne séparée

Exercice 14.4 — Exporter vers CSV et JSON (10min)

Exporter les cinq premiers services vers un fichier CSV et afficher trois services au format JSON.

```
# Création du répertoire de travail du lab s'il n'existe pas
if (-not (Test-Path -Path 'C:\Travail\Lab14')) {
    New-Item -Path 'C:\Travail\Lab14' -ItemType Directory
}
# Test-Path : vérifie l'existence du répertoire
# -not : opérateur de négation booléenne
# New-Item -ItemType Directory : crée le répertoire s'il est absent

# Export des cinq premiers services vers un fichier CSV
Get-Service | Select-Object -First 5 -Property Name, Status, DisplayName | Export-Csv -Path 'C:\Travail\Lab
# Export-Csv : sérialise les objets vers un fichier CSV
# -Path : chemin du fichier de sortie
# -NoTypeInformation : omet l'en-tête de type .NET dans le CSV
# -Encoding UTF8 : encodage UTF-8 du fichier (sans BOM dans PowerShell 7+)

# Affichage de trois services au format JSON
Get-Service | Select-Object -First 3 -Property Name, Status | ConvertTo-Json
# ConvertTo-Json : transforme les objets en une chaîne JSON structurée
```

- Le fichier services.csv est créé et contient les données des cinq premiers services
- La sortie JSON s'affiche dans la console avec les trois premiers services

---

Démonstration commentée

Exemple de pipeline complet d'objets, de la sélection à l'export :

```
# 1. Récupération des processus utilisant plus de 50 Mo de mémoire privée
$processus = Get-Process | Where-Object { $_.PM -gt 50MB } | Select-Object -Property Name, PM, CPU
# Get-Process : retourne des objets Process représentant les processus en cours
# Where-Object { $_.PM -gt 50MB } : filtre les processus dont la mémoire privée (PM) dépasse 50 Mo
# $_.PM : propriété Private Memory (mémoire privée) du processus courant
# -gt : opérateur "strictement supérieur à" PowerShell
# 50MB : suffixe de taille compris par PowerShell (50 mégaoctets)
# Select-Object -Property Name, PM, CPU : conserve uniquement ces trois propriétés

# 2. Tri des processus par consommation mémoire décroissante
$processus | Sort-Object PM -Descending | Format-Table -AutoSize
# Sort-Object : trie les objets selon la propriété indiquée
# PM : propriété de tri (mémoire privée)
# -Descending : ordre décroissant (du plus gourmand au moins gourmand)
# Format-Table -AutoSize : affiche le résultat sous forme de tableau avec colonnes ajustées

# 3. Sérialisation des mêmes objets au format JSON
$processus | ConvertTo-Json
# ConvertTo-Json : transforme la collection d'objets en une chaîne JSON
# Chaque étape préserve la structure objet ; aucune conversion en texte intermédiaire
```

Résultat attendu : un tableau des processus utilisant plus de 50 Mo de mémoire, trié par consommation décroissante, puis une représentation JSON des mêmes données. Chaque étape préserve la structure objet, permettant de réutiliser les données sans les reparser.

Interprétation ligne par ligne :

- `Get-Process` récupère la liste de tous les processus sous forme d'objets `System.Diagnostics.Process`.
- `Where-Object { $_.PM -gt 50MB }` évalue la condition pour chaque processus et ne conserve que ceux dont la mémoire privée dépasse 50 Mo.
- `Select-Object -Property Name, PM, CPU` projette chaque objet sur un sous-ensemble de trois propriétés, allégeant les données transportées.
- `Sort-Object PM -Descending` ordonne les objets par consommation mémoire décroissante, sans altérer leurs propriétés.
- `Format-Table -AutoSize` produit un affichage tabulaire lisible, placé en fin de pipeline pour ne pas altérer les objets destinés à l'export.
- `ConvertTo-Json` sérialise les mêmes objets en JSON, démontrant que la structure objet a été préservée tout au long du traitement.

Correction et résolution pas à pas

Solution

Solution du Lab 14

```
# Exercice 14.1 — Inspecter un objet avec Get-Member
Get-Service -Name 'Spooler' | Get-Member
# Get-Service -Name : récupère l'objet ServiceController du service Spooler
# Get-Member : introspecte l'objet et liste ses membres

Get-Service -Name 'Spooler' | Get-Member -MemberType Property
# -MemberType Property : filtre pour n'afficher que les propriétés

# Exercice 14.2 — Sélectionner et filtrer des objets
Get-Service | Select-Object -First 5 -Property Name, Status
# Select-Object -First 5 : limite aux cinq premiers objets
# -Property Name, Status : propriétés à conserver

Get-Service | Where-Object { $_.Status -eq 'Running' } | Select-Object -Property Name, DisplayName
# Where-Object : filtre selon $_.Status -eq 'Running'
# $_.Status : propriété Status de l'objet courant
# -eq : opérateur d'égalité

# Exercice 14.3 — Itérer et formater
Get-Service | Select-Object -First 3 | ForEach-Object { Write-Host $_.DisplayName }
# ForEach-Object : exécute Write-Host pour chaque objet
# $_.DisplayName : propriété nom d'affichage

Get-Service | Select-Object -First 3 -Property Name, Status, DisplayName | Format-List
# Format-List : affiche chaque propriété sur une ligne séparée

# Exercice 14.4 — Exporter vers CSV et JSON
if (-not (Test-Path -Path 'C:\Travail\Lab14')) {
    New-Item -Path 'C:\Travail\Lab14' -ItemType Directory
}
# Test-Path : vérifie l'existence du répertoire
# -not : opérateur de négation
# New-Item -ItemType Directory : crée le répertoire

Get-Service | Select-Object -First 5 -Property Name, Status, DisplayName | Export-Csv -Path 'C:\Travail\
# Export-Csv : sérialise vers un fichier CSV
# -NoTypeInformation : omet l'en-tête de type .NET
# -Encoding UTF8 : encodage UTF-8

Get-Service | Select-Object -First 3 -Property Name, Status | ConvertTo-Json
# ConvertTo-Json : transforme les objets en chaîne JSON
```

Résolution pas à pas :

- **Get-Member** : introspecte l'objet `ServiceController` et liste ses propriétés (`Name`, `Status`, `DisplayName`, `ServiceType`) et ses méthodes (`Start`, `Stop`, `Pause`, `Continue`, `WaitForStatus`). Cette étape permet de découvrir la structure de l'objet avant de l'exploiter.
- **Select-Object -First 5 -Property Name, Status** : conserve uniquement les cinq premiers objets et les deux propriétés demandées. Le reste de l'objet est ignoré pour la sortie, mais l'objet reste typé et exploitable.
- **Where-Object { $_.Status -eq 'Running' }** : le scriptblock est évalué pour chaque objet transmis. `$_` fait référence à l'objet courant. Seuls les objets dont la propriété `Status` vaut `'Running'` passent le filtre et sont transmis à la cmdlet suivante.
- **ForEach-Object { Write-Host $_.DisplayName }** : exécute le scriptblock pour chaque objet. `Write-Host` affiche le nom d'affichage sans produire d'objet formaté, ce qui convient à un affichage simple à l'écran.

- **Format-List** : affiche chaque propriété sélectionnée sur une ligne séparée, offrant une lecture détaillée des propriétés de chaque objet.
- **Export-Csv -NoTypeInformation -Encoding UTF8** : produit un fichier CSV lisible par des tableurs, encodé en UTF-8 pour l'interopérabilité. L'option `-NoTypeInformation` omet l'en-tête de type .NET pour un fichier plus propre.
- **ConvertTo-Json** : transforme les objets en une chaîne JSON, utile pour l'échange avec des applications web ou des API. La structure des données est préservée dans le format d'échange.

Bonnes pratiques

Pratique	Description
Format-* en fin de pipeline	Placer les cmdlets de formatage en dernière position, car elles transforment les objets en objets de formatage inutilisables par la suite
Préserver les objets	Éviter de convertir les objets en texte avant le dernier moment ; privilégier les opérations sur les propriétés nommées
-NoTypeInformation	Utiliser avec `Export-Csv` pour produire un fichier CSV propre, sans en-tête de type .NET
Encodage UTF-8	Utiliser `-Encoding UTF8` avec `Export-Csv` pour garantir l'interopérabilité entre systèmes
Propriétés nommées	Référencer les propriétés par leur nom (`$_.Status`) plutôt que par leur position textuelle, pour rendre les scripts robustes

Script de vérification de l'intégrité des objets avant export :

```
# Vérification que les objets conservent leurs propriétés avant export
$services = Get-Service | Select-Object -First 5 -Property Name, Status, DisplayName
# Get-Service : retourne les objets ServiceController
# Select-Object : projette sur un sous-ensemble de propriétés

# Vérification de l'intégrité des objets
$services | Get-Member -MemberType Property
# Get-Member -MemberType Property : confirme que les propriétés sont bien présentes

# Export uniquement si des objets sont présents
if (@($services).Count -gt 0) {
    $services | Export-Csv -Path 'C:\Travail\Lab14\services_verifie.csv' -NoTypeInformation -Encoding UTF8
    # Export-Csv : exporte les objets vérifiés vers un fichier CSV
    # -Path : chemin du fichier de sortie
    # -NoTypeInformation : omet l'en-tête de type .NET
    # -Encoding UTF8 : encodage UTF-8
    Write-Host "Export effectué : $($services.Count) services exportés."
    # Write-Host : affiche un message de confirmation
    # $services.Count : nombre d'objets dans la collection
} else {
    Write-Host "Aucun service à exporter."
}
# @($services).Count : nombre d'objets dans la collection (l'opérateur @() force un tableau, même avec un
# if / else : conditionnelle pour n'exporter que si des objets sont présents
```

BONNES PRATIQUES

Les cmdlets `Format-*` ne doivent apparaître qu'en fin de pipeline, car elles transforment les objets en objets de formatage inutilisables par les cmdlets suivantes. Pour produire des fichiers d'échange, préférer `Export-Csv` ou `ConvertTo-Json` qui préservent la structure des données. Toujours référencer les propriétés par leur nom plutôt que par leur position, afin de rendre les scripts indépendants du format d'affichage.

Exercices — Chapitre 14

Exercice 14.5 — Comparaison objet PowerShell vs texte Bash (10min)

Expliquer pourquoi l'approche objet de PowerShell est plus robuste que l'approche texte de Bash pour extraire le nom des services arrêtés.

- Quel est l'avantage de manipuler des objets plutôt que du texte ?
- Quel risque existe-t-il si le format de sortie texte change ?

Solution

Solution

```
# Approche PowerShell (robuste) — référence la propriété par son nom
Get-Service | Where-Object { $_.Status -eq 'Stopped' } | Select-Object -Property Name
# L'objet ServiceController possède une propriété Status typée, indépendante de l'affichage
# $_.Status : propriété Status de l'objet courant
# -eq : opérateur d'égalité
```

Résolution pas à pas :

- L'avantage de l'approche objet est que la propriété `Status` est accessible directement par son nom, quel que soit le mode d'affichage. On n'a pas besoin de connaître la position d'une colonne dans un tableau de texte. La propriété est typée, ce qui rend la comparaison fiable.
- En Bash, l'analyse de texte (par exemple avec `awk`) dépend de la position des colonnes dans la sortie. Si le format de sortie de la commande change (ajout d'une colonne, modification de l'espacement, changement de langue), le script casse car il référence des positions textuelles, pas des propriétés nommées. En PowerShell, le script reste valide car il référence des propriétés nommées de l'objet, indépendamment de leur représentation visuelle.

Ce qu'on a retenu

CE QU'ON A RETENU

PowerShell manipule des objets .NET typés, et non du texte brut. `Get-Member` introspecte les propriétés et méthodes d'un objet. `Select-Object` sélectionne des propriétés, `Where-Object` filtre selon une condition, `ForEach-Object` itère sur chaque objet. `Format-*` modifie uniquement l'affichage et doit être placé en fin de pipeline. `Export-Csv` et `ConvertTo-Json` sérialisent les objets vers des formats d'échange. Contrairement à Bash, l'approche objet préserve la structure des données tout au long du pipeline, rendant les scripts plus robustes et indépendants du format d'affichage.

Chapitre 15 — Gestion des processus et des droits (13h00-14h30)

Objectifs

- Lister, démarrer et arrêter des processus avec Get-Process, Start-Process et Stop-Process.
- Gérer les services Windows avec Get-Service, Start-Service, Stop-Service et Restart-Service.
- Lire et modifier les listes de contrôle d'accès (ACL) avec Get-Acl et Set-Acl, et gérer les utilisateurs et groupes locaux avec Get-LocalUser, New-LocalUser et Add-LocalGroupMember.

Contenu Théorique (45min)

1. Gestion des processus (Get-Process, Start-Process, Stop-Process) (15min)

En administration système, un processus est un programme en cours d'exécution, identifié par un identifiant unique (PID, Process Identifier). Sous Linux, la commande `ps` permet d'en dresser l'instantané et `kill` d'en demander l'arrêt (The Linux Command Line, William Shotts, p. 128 et p. 137). PowerShell applique la même logique avec une approche objet : Get-Process retourne des objets System.Diagnostics.Process, Start-Process lance un nouveau processus, et Stop-Process en interrompt l'exécution.

```
# Liste des processus dont le nom est notepad
Get-Process -Name 'notepad'
# Get-Process : retourne les objets Process représentant les processus en cours d'exécution
# -Name : filtre les processus par leur nom (ici le Bloc-notes)
# Retourne des objets System.Diagnostics.Process avec les propriétés Name, Id, PM, CPU, WS
```

```
# Lancement d'une nouvelle instance du Bloc-notes
Start-Process -FilePath 'notepad.exe'
# Start-Process : lance un nouveau processus sur le système
# -FilePath : chemin ou nom de l'exécutable à lancer (ici notepad.exe)
# Le processus s'exécute de manière indépendante de la console PowerShell
```

```
# Arrêt du processus notepad transmis par le pipeline
Get-Process -Name 'notepad' | Stop-Process
# Stop-Process : arrête immédiatement le processus transmis par le pipeline
# L'arrêt est immédiat, sans enregistrement des données non sauvegardées
# Cette commande nécessite des droits suffisants sur le processus cible
```

CONCEPT CLÉ

Get-Process retourne des objets Process dont les propriétés (Name, Id, CPU, PM, WS) sont directement accessibles par leur nom, sans analyse de texte. Stop-Process interrompt l'exécution immédiatement, ce qui peut entraîner la perte de données non sauvegardées.

BON À SAVOIR

Sous Linux, l'équivalent de Get-Process est `ps`, et celui de Stop-Process est `kill` (The Linux Command Line, William Shotts, p. 128 et p. 137). PowerShell préserve cependant la structure objet des processus, contrairement au flux de texte produit par `ps`.

PIÈGE

Arrêter un processus système critique (par exemple lsass.exe ou winlogon.exe) peut rendre le système instable ou provoquer un redémarrage intempestif. Toujours identifier le processus et son PID avant de l'arrêter.

CONTRE-MESURE

Utiliser Get-Process pour identifier le processus et son identifiant (PID) avant d'appeler Stop-Process. Éviter d'arrêter des processus dont le nom est inconnu ou dont l'importance système n'est pas établie.

2. Gestion des services (Get-Service, Start-Service, Stop-Service, Restart-Service) (15min)

Un service est un programme qui s'exécute en arrière-plan, souvent indépendamment de toute session utilisateur ouverte. Sous Linux, les services sont gérés par systemd (systemctl) ; sous Windows, ils sont représentés par des objets ServiceController manipulables avec Get-Service, Start-Service, Stop-Service et Restart-Service.

```
# Récupération du service d'impression Spooler
Get-Service -Name 'Spooler'
# Get-Service : retourne l'objet ServiceController du service spécifié
# -Name : nom interne du service à récupérer
# Retourne les propriétés Name, Status, DisplayName, etc.
```

```
# Démarrage du service Spooler s'il est arrêté
Start-Service -Name 'Spooler'
# Start-Service : démarre le service spécifié s'il est actuellement arrêté
# -Name : nom interne du service à démarrer
# Ne produit aucune erreur si le service est déjà en cours d'exécution
```

```
# Arrêt du service Spooler s'il est en cours d'exécution
Stop-Service -Name 'Spooler'
# Stop-Service : arrête le service spécifié s'il est en cours d'exécution
# -Name : nom interne du service à arrêter
# L'arrêt peut prendre quelques instants si des dépendances existent
```

```
# Arrêt puis redémarrage du service Spooler
Restart-Service -Name 'Spooler'
# Restart-Service : arrête puis redémarre le service spécifié
# -Name : nom interne du service à redémarrer
# Équivalent à un Stop-Service suivi d'un Start-Service
```

BON À SAVOIR

Get-Service accepte le paramètre -DisplayName pour rechercher un service par son nom d'affichage plutôt que par son nom de service interne.

PIÈGE

Arrêter un service critique (par exemple le service de mise à jour Windows ou le service de sécurité) peut perturber le fonctionnement du système. Vérifier les dépendances du service avant de l'arrêter.

CONTRE-MESURE

Toujours consulter l'état du service avec Get-Service avant d'appliquer Start-Service, Stop-Service ou Restart-Service, et documenter les services modifiés dans un script d'administration.

3. Listes de contrôle d'accès (ACL) avec Get-Acl et Set-Acl (15min)

Une ACL (Access Control List, liste de contrôle d'accès) définit quels utilisateurs ou groupes possèdent quels droits (lecture, écriture, exécution) sur un fichier ou un dossier. Sous Linux, les droits s'administrent avec chmod et chown (The Linux Command Line, William Shotts, p. 14 et p. 125), et les ACL étendues avec setfacl/getfacl. PowerShell propose Get-Acl pour lire l'ACL et Set-Acl pour l'appliquer après modification.

```
# Lecture de l'ACL d'un fichier
$acl = Get-Acl -Path 'C:\Travail\Lab13\rapport.txt'
# Get-Acl : lit la liste de contrôle d'accès d'un fichier ou dossier
# -Path : chemin de la cible dont on lit les droits
# $acl : objet System.Security.AccessControl.FileSecurity contenant le propriétaire et les règles d'accès
```

```
# Affichage du propriétaire du fichier
$acl.Owner
# .Owner : propriété contenant le nom du compte propriétaire du fichier
# Résultat attendu : le nom du compte utilisateur propriétaire (ex. BUILTIN\Administrators)
```

```
# Application d'une ACL modifiée sur le fichier
$acl | Set-Acl -Path 'C:\Travail\Lab13\rapport.txt'
# Set-Acl : applique une liste de contrôle d'accès modifiée à un fichier ou dossier
# -Path : cible sur laquelle appliquer l'ACL
# L'objet ACL modifié est réinjecté sur la cible via le pipeline
```

CONCEPT CLÉ

Get-Acl retourne un objet System.Security.AccessControl.FileSecurity contenant le propriétaire et les règles d'accès. Set-Acl applique une version modifiée de cet objet à la cible. La modification des ACL nécessite des droits suffisants (généralement administrateur ou propriétaire).

PIÈGE

Set-Acl remplace l'intégralité de l'ACL de la cible par l'objet fourni. Toute règle d'accès non présente dans l'objet modifié est perdue. Toujours conserver une copie de l'ACL originale avant modification.

CONTRE-MESURE

Lire l'ACL avec Get-Acl dans une variable, modifier uniquement les règles nécessaires, puis appliquer avec Set-Acl. Tester sur un fichier non critique avant d'appliquer sur des données de production.

4. Utilisateurs et groupes locaux (Get-LocalUser, New-LocalUser, Add-LocalGroupMember) (15min)

PowerShell permet de gérer les comptes d'utilisateurs locaux et les groupes locaux d'une machine Windows. Sous Linux, l'équivalent s'effectue avec useradd, usermod et groupadd. Les cmdlets PowerShell appartiennent au module Microsoft.PowerShell.LocalAccounts.

```
# Listage des comptes d'utilisateurs locaux
Get-LocalUser
# Get-LocalUser : liste les comptes d'utilisateurs locaux de la machine
# Retourne des objets contenant Name, Enabled, LastLogon, PasswordExpires, etc.
```

```
# Création d'un nouvel utilisateur local
New-LocalUser -Name 'LabUtilisateur' -Password (Read-Host -AsSecureString -Prompt 'Mot de passe pour LabUt
# New-LocalUser : crée un nouveau compte utilisateur local
# -Name : nom du compte à créer
# -Password : mot de passe sous forme de chaîne sécurisée (SecureString)
# Read-Host -AsSecureString : saisit le mot de passe sans l'afficher à l'écran
# -Prompt : message affiché à l'utilisateur lors de la saisie
```

```
# Ajout de l'utilisateur au groupe Utilisateurs
Add-LocalGroupMember -Group 'Utilisateurs' -Member 'LabUtilisateur'
# Add-LocalGroupMember : ajoute un utilisateur à un groupe local
# -Group : nom du groupe cible
# -Member : utilisateur à ajouter au groupe
# L'utilisateur hérite des droits associés au groupe
```

BON À SAVOIR

Les cmdlets Get-LocalUser, New-LocalUser et Add-LocalGroupMember nécessitent le module Microsoft.PowerShell.LocalAccounts, disponible par défaut sur Windows 10/11 et Windows Server.

PIÈGE

New-LocalUser requiert un mot de passe complexe respectant la stratégie de mots de passe de la machine. Un mot de passe trop simple provoquera une erreur de création.

CONTRE-MESURE

Vérifier la stratégie de mots de passe avant de créer un utilisateur, ou utiliser un mot de passe respectant les exigences de complexité (majuscules, minuscules, chiffres, caractères spéciaux).

Fig 3 — Gestion des processus et des droits

Diagramme		
B	**Oui**	
B	**Non**	
Get-Process liste les processus	→	
E	→	
F	**Démarrer**	
F	**Arrêter**	
F	**Redémarrer**	
E	→	
J	→	
K	→	
L	→	M

Figure — Diagramme Mermaid

Exemple concret de gestion d'un processus et de ses droits :

```
# Lancement d'un processus de test
Start-Process -FilePath 'notepad.exe'
# Start-Process : lance l'exécutable notepad.exe (Bloc-notes)

# Identification du processus lancé
$p = Get-Process -Name 'notepad'
# Get-Process : récupère l'objet Process du Bloc-notes
# $p : variable stockant l'objet processus

# Lecture de l'ACL d'un fichier associé
$acl = Get-Acl -Path 'C:\Travail\Lab13\rapport.txt'
# Get-Acl : lit la liste de contrôle d'accès du fichier
# $acl : objet contenant les règles d'accès et le propriétaire

# Affichage du propriétaire du fichier
$acl.Owner
# .Owner : propriété contenant le nom du propriétaire du fichier
```

Résultat attendu : le Bloc-notes est lancé, l'objet Process est récupéré dans $p, et le propriétaire du fichier rapport.txt s'affiche.

Lab 15 — Gestion des processus et des droits (45min)

Environnement

Élément	Valeur
Poste	Windows avec PowerShell 5.1 ou PowerShell 7+
Répertoire de travail	C:\Travail\Lab15 (à créer)
Droits	Utilisateur avec droits suffisants pour modifier les ACL et les comptes locaux

Exercices (les exercices s'enchaînent pour former un lab complet)

Exercice 15.1 — Lister et arrêter un processus (10min)

Lancer le Bloc-notes, l'identifier avec Get-Process, puis l'arrêter avec Stop-Process.

```
# Lancement du Bloc-notes
Start-Process -FilePath 'notepad.exe'
# Start-Process : lance l'exécutable notepad.exe
# -FilePath : chemin ou nom de l'exécutable à lancer

# Identification du processus
Get-Process -Name 'notepad'
# Get-Process : liste le processus notepad et ses propriétés (Id, CPU, PM)
# -Name : filtre les processus par nom

# Arrêt du processus
Get-Process -Name 'notepad' | Stop-Process
# Stop-Process : arrête le processus transmis par le pipeline
# L'arrêt est immédiat, sans enregistrement des données non sauvegardées
```

- Le processus notepad est lancé
- Get-Process retourne les informations du processus
- Le processus est arrêté (plus retourné par Get-Process)

Exercice 15.2 — Gérer l'état d'un service (15min)

Vérifier l'état du service Spooler, l'arrêter, puis le redémarrer.

```
# Vérification de l'état du service
Get-Service -Name 'Spooler' | Select-Object -Property Name, Status, DisplayName
# Get-Service : récupère l'objet ServiceController du service Spooler
# -Name : nom interne du service
# Select-Object : sélectionne les propriétés à afficher
# -Property Name, Status, DisplayName : propriétés à conserver

# Arrêt du service
Stop-Service -Name 'Spooler'
# Stop-Service : arrête le service s'il est en cours d'exécution
# -Name : nom interne du service à arrêter

# Redémarrage du service
Start-Service -Name 'Spooler'
# Start-Service : démarre le service s'il est arrêté
# -Name : nom interne du service à démarrer
```

- L'état initial du service s'affiche
- Le service est arrêté puis redémarré

Exercice 15.3 — Lire et afficher une ACL (10min)

Lire l'ACL d'un fichier et afficher son propriétaire.

```
# Lecture de l'ACL du fichier
$acl = Get-Acl -Path 'C:\Travail\Lab13\rapport.txt'
# Get-Acl : lit la liste de contrôle d'accès du fichier
# -Path : chemin du fichier cible
# $acl : objet contenant les règles d'accès et le propriétaire

# Affichage du propriétaire
$acl.Owner
# .Owner : propriété contenant le nom du propriétaire du fichier
# Résultat attendu : le nom du compte propriétaire s'affiche
```

- L'ACL est lue sans erreur
- Le nom du propriétaire s'affiche

Exercice 15.4 — Créer un utilisateur local et l'ajouter à un groupe (10min)

Créer un utilisateur local et l'ajouter au groupe Utilisateurs.

```
# Création de l'utilisateur local
New-LocalUser -Name 'LabUtilisateur' -Password (Read-Host -AsSecureString -Prompt 'Mot de passe pour LabUt
# New-LocalUser : crée un compte utilisateur local
# -Name : nom du compte à créer
# -Password : mot de passe sous forme de chaîne sécurisée (SecureString)
# Read-Host -AsSecureString : saisit le mot de passe de manière masquée
# -Prompt : message affiché lors de la saisie

# Ajout au groupe Utilisateurs
Add-LocalGroupMember -Group 'Utilisateurs' -Member 'LabUtilisateur'
# Add-LocalGroupMember : ajoute l'utilisateur au groupe local
# -Group : nom du groupe cible
# -Member : utilisateur à ajouter
```

- L'utilisateur LabUtilisateur est créé
- L'utilisateur est membre du groupe Utilisateurs

Démonstration commentée

Exemple de gestion complète d'un processus avec vérification des droits :

```
# 1. Lancement d'un processus de test
Start-Process -FilePath 'notepad.exe'
# Start-Process : lance une nouvelle instance du Bloc-notes
# -FilePath : nom de l'exécutable à lancer

# 2. Récupération de l'objet processus
$processus = Get-Process -Name 'notepad'
# Get-Process : récupère l'objet System.Diagnostics.Process du Bloc-notes
# -Name : filtre par nom de processus
# $processus : variable stockant l'objet processus

# 3. Affichage des propriétés principales
$processus | Select-Object -Property Name, Id, PM, CPU
# Select-Object : projette l'objet sur un sous-ensemble de propriétés
# -Property Name, Id, PM, CPU : propriétés à afficher (nom, PID, mémoire privée, CPU)
# PM : mémoire privée consommée par le processus
# CPU : temps processeur consommé

# 4. Arrêt propre du processus
$processus | Stop-Process
# Stop-Process : arrête le processus, libérant les ressources associées
# L'arrêt est immédiat, sans enregistrement des données non sauvegardées
```

Résultat attendu : le Bloc-notes est lancé, ses propriétés (nom, identifiant, mémoire, CPU) s'affichent, puis le processus est arrêté. L'objet processus conserve ses informations jusqu'à l'arrêt.

Interprétation ligne par ligne :

- Start-Process lance une nouvelle instance du Bloc-notes, qui s'exécute indépendamment de la console PowerShell.
- Get-Process récupère l'objet Process correspondant, dont les propriétés Id (identifiant du processus), PM (mémoire privée) et CPU (temps processeur) sont accessibles par leur nom.
- Select-Object projette l'objet sur les quatre propriétés demandées pour un affichage lisible.
- Stop-Process interrompt immédiatement le processus. Le Bloc-notes se ferme sans enregistrement des documents ouverts.

Correction et résolution pas à pas

Solution

> Solution du Lab 15

```
# Exercice 15.1 — Lister et arrêter un processus
Start-Process -FilePath 'notepad.exe'
# Start-Process : lance le Bloc-notes
# -FilePath : nom de l'exécutable à lancer

Get-Process -Name 'notepad'
# Get-Process : liste le processus notepad
# -Name : filtre par nom de processus

Get-Process -Name 'notepad' | Stop-Process
# Stop-Process : arrête le processus transmis par le pipeline

# Exercice 15.2 — Gérer l'état d'un service
Get-Service -Name 'Spooler' | Select-Object -Property Name, Status, DisplayName
# Get-Service : récupère l'objet service
# Select-Object : sélectionne les propriétés à afficher

Stop-Service -Name 'Spooler'
# Stop-Service : arrête le service

Start-Service -Name 'Spooler'
# Start-Service : démarre le service

# Exercice 15.3 — Lire et afficher une ACL
$acl = Get-Acl -Path 'C:\Travail\Lab13\rapport.txt'
# Get-Acl : lit la liste de contrôle d'accès
# -Path : chemin du fichier cible

$acl.Owner
# .Owner : affiche le nom du propriétaire

# Exercice 15.4 — Créer un utilisateur local et l'ajouter à un groupe
New-LocalUser -Name 'LabUtilisateur' -Password (Read-Host -AsSecureString -Prompt 'Mot de passe pour Lab
# New-LocalUser : crée un compte utilisateur local
# -Name : nom du compte
# -Password : mot de passe sécurisé (SecureString)

Add-LocalGroupMember -Group 'Utilisateurs' -Member 'LabUtilisateur'
# Add-LocalGroupMember : ajoute l'utilisateur au groupe
# -Group : nom du groupe cible
# -Member : utilisateur à ajouter
```

Résolution pas à pas :

- **Start-Process -FilePath 'notepad.exe'** : lance une nouvelle instance du Bloc-notes. Le processus s'exécute de manière indépendante de la console PowerShell.
- **Get-Process -Name 'notepad'** : retourne l'objet Process correspondant. Ses propriétés Id (identifiant du processus), PM (mémoire privée) et CPU (temps processeur) sont accessibles par leur nom.
- **Stop-Process** : interrompt immédiatement le processus. Le Bloc-notes se ferme sans enregistrement des documents ouverts. Cette opération est irréversible pour la session en cours.
- **Get-Service -Name 'Spooler'** : récupère l'objet ServiceController du service d'impression. La propriété Status indique Running ou Stopped.
- **Stop-Service puis Start-Service** : modifient l'état du service. Stop-Service arrête le service, Start-Service le redémarre. Ces opérations nécessitent des droits suffisants sur le service.
- **Get-Acl** : lit l'objet FileSecurity contenant le propriétaire et les règles d'accès du fichier. La propriété .Owner retourne le nom du compte propriétaire.

- **New-LocalUser** : crée un compte local. Le mot de passe est saisi de manière sécurisée avec Read-Host -AsSecureString pour ne pas l'afficher en clair. Le mot de passe doit respecter la stratégie de complexité de la machine.
- **Add-LocalGroupMember** : ajoute le nouvel utilisateur au groupe local Utilisateurs, lui conférant les droits associés à ce groupe.

Bonnes pratiques

Pratique	Description
Vérification avant arrêt	Utiliser Get-Process avant Stop-Process pour identifier le processus et éviter d'interrompre un processus critique
Droits suffisants	Vérifier que le compte exécutant dispose des droits pour modifier les ACL et les comptes locaux
Mot de passe sécurisé	Utiliser Read-Host -AsSecureString pour saisir les mots de passe sans les afficher en clair
Sauvegarde de l'ACL	Conserver une copie de l'ACL originale avant toute modification avec Set-Acl
État du service	Consulter Get-Service avant d'appliquer Start-Service, Stop-Service ou Restart-Service

Script de vérification des droits avant modification d'ACL :

```
# Vérification des droits avant modification d'ACL
$chemin = 'C:\Travail\Lab13\rapport.txt'
if (-not (Test-Path -Path $chemin)) {
    Write-Host 'Le fichier n'existe pas.'
    exit
}
$acl = Get-Acl -Path $chemin
# Get-Acl : lit la liste de contrôle d'accès du fichier
# -Path : chemin du fichier cible
Write-Host "Propriétaire actuel : $($acl.Owner)"
# $acl.Owner : propriété contenant le nom du propriétaire
# Vérifier que le compte courant est propriétaire ou administrateur avant Set-Acl
# Test-Path : vérifie l'existence du fichier avant toute opération
# Write-Host : affiche le nom du propriétaire à l'écran
```

BONNES PRATIQUES

Ne jamais arrêter un processus sans l'avoir d'abord identifié avec Get-Process, afin d'éviter d'interrompre un processus système critique. Pour la création d'utilisateurs, toujours saisir le mot de passe de manière sécurisée et respecter la stratégie de complexité de la machine. Avant d'appliquer Set-Acl, conserver une copie de l'ACL originale pour pouvoir revenir en arrière en cas d'erreur.

Exercices — Chapitre 15

Exercice 15.5 — Processus et droits : analyse d'arrêt (10min)

Expliquer pourquoi arrêter un processus avec Stop-Process est plus radical que fermer une application normalement, et quel risque cela présente pour les données.

- Quelle est la différence entre Stop-Process et une fermeture normale ?
- Quel type de données peut être perdu ?

Solution

Solution

```
# Arrêt immédiat d'un processus
Get-Process -Name 'notepad' | Stop-Process
# Stop-Process : envoie un signal d'arrêt immédiat au processus
# L'arrêt est immédiat, sans enregistrement des données non sauvegardées
```

Résolution pas à pas :

- Une fermeture normale (via l'interface utilisateur) permet à l'application d'enregistrer ses données et de se terminer proprement. Stop-Process interrompt le processus immédiatement, sans lui laisser le temps d'enregistrer son état.
- Les données non sauvegardées dans l'application (par exemple un document non enregistré dans le Bloc-notes) sont perdues définitivement. Il est donc recommandé de privilégier la fermeture normale lorsque c'est possible, et de réserver Stop-Process aux cas où l'application ne répond plus.

Ce qu'on a retenu

CE QU'ON A RETENU

Get-Process liste les processus sous forme d'objets Process, et Stop-Process les arrête immédiatement. Start-Process lance un nouveau processus. Get-Service, Start-Service, Stop-Service et Restart-Service gèrent l'état des services Windows. Get-Acl lit la liste de contrôle d'accès d'un fichier, et Set-Acl l'applique après modification. New-LocalUser crée un compte local, et Add-LocalGroupMember l'ajoute à un groupe. L'arrêt d'un processus est radical et peut entraîner la perte de données non sauvegardées. Toutes ces commandes PowerShell sont spécifiques à Windows et nécessitent une machine virtuelle Windows Server pour être testées.

Chapitre 16 — Cas pratique PowerShell (14h45-16h00)

Objectifs

- Concevoir un script PowerShell complet avec un bloc param déclarant des paramètres nommés typés.
- Gérer les erreurs avec try/catch, trap et la variable $ErrorActionPreference.
- Implémenter une journalisation dans un fichier avec horodatage et automatiser la vérification et le démarrage d'un service Windows.

Contenu Théorique (45min)

1. Bloc param et paramètres nommés (15min)

Un script PowerShell peut déclarer ses paramètres en début de fichier, avant toute autre instruction, dans un bloc param. Ce bloc définit l'interface du script : il déclare les paramètres que l'utilisateur peut fournir à l'exécution, avec éventuellement un typage explicite et une valeur par défaut. Lorsqu'un paramètre possède une valeur par défaut, le script peut être appelé sans argument pour ce paramètre ; PowerShell utilisera alors la valeur par défaut.

```
param (
    [string]$CheminLog = 'C:\Travail\rapport.log',
    [string]$NomService = 'Spooler'
)
# param : bloc de déclaration des paramètres du script, placé en tout début de fichier
# [string] : typage explicite du paramètre, PowerShell convertit la valeur fournie en chaîne de caractères
# $CheminLog : chemin du fichier de journalisation, avec valeur par défaut C:\Travail\rapport.log
# $NomService : nom du service à surveiller, avec valeur par défaut Spooler
```

CONCEPT CLÉ

Le bloc param doit apparaître en tout début de script, avant toute autre instruction. Il définit l'interface du script : les paramètres que l'utilisateur peut fournir à l'exécution, avec leur type et leur valeur par défaut.

BON À SAVOIR

Si le bloc param est absent, les paramètres passés à l'exécution sont inaccessibles de manière nommée. Toute valeur fournie sur la ligne de commande serait alors ignorée ou provoquerait une erreur selon le contexte d'appel.

2. Gestion des erreurs (try/catch, trap, $ErrorActionPreference) (15min)

PowerShell distingue deux catégories d'erreurs : les erreurs terminantes (stopping errors), qui interrompent l'exécution du script, et les erreurs non terminantes (non-terminating errors), qui affichent un message d'erreur mais laissent le script se poursuivre. Par défaut, de nombreuses cmdlets génèrent des erreurs non terminantes, qui ne sont pas interceptables par try/catch.

La variable $ErrorActionPreference contrôle le comportement par défaut face aux erreurs. Lorsqu'elle vaut 'Stop', les erreurs non terminantes sont transformées en erreurs terminantes, devenant ainsi attrapables par un bloc try/catch. Le bloc try entoure le code susceptible de lever une erreur ; le bloc catch est exécuté si une erreur terminante survient dans le bloc try. La variable automatique $_ dans le bloc catch fait référence à l'objet d'erreur courant, et $_.Exception.Message contient le message lisible de l'exception.

Le bloc trap, quant à lui, intercepte toute erreur non gérée dans le scope courant, y compris en dehors d'un bloc try. Il constitue un filet de sécurité complémentaire pour les erreurs imprévues. Le mot-clé continue dans un bloc trap reprend l'exécution après le traitement de l'erreur ; break l'arrête.

```
$ErrorActionPreference = 'Stop'
# $ErrorActionPreference : variable de préférence contrôlant le comportement face aux erreurs
# 'Stop' : transforme les erreurs non terminantes en erreurs terminantes, attrapables par try/catch
```

```
try {
    Get-Content -Path 'C:\Fichier\Inexistant.txt' -ErrorAction Stop
    # -ErrorAction Stop : force l'erreur de cette commande à devenir terminante pour ce bloc try
} catch {
    Write-Host "Erreur capturée : $($_.Exception.Message)"
    # $_.Exception.Message : message d'erreur de l'exception courante interceptée par catch
}
# try : bloc contenant le code susceptible de lever une erreur terminante
# catch : bloc exécuté si une erreur terminante survient dans le bloc try
```

```
trap {
    Write-Host "Erreur globale capturée : $($_.Exception.Message)"
    continue
}
# trap : bloc exécuté pour toute erreur non gérée dans le scope courant
# continue : reprend l'exécution après le traitement de l'erreur
```

PIÈGE

Par défaut, de nombreuses cmdlets PowerShell génèrent des erreurs non terminantes qui n'interrompent pas l'exécution et ne sont pas attrapées par try/catch. Il faut utiliser -ErrorAction Stop sur la cmdlet critique, ou définir $ErrorActionPreference = 'Stop' en début de script, pour les rendre attrapables.

CONTRE-MESURE

Toujours ajouter -ErrorAction Stop sur les cmdlets critiques dans un bloc try, ou définir $ErrorActionPreference = 'Stop' en début de script pour une gestion d'erreurs uniforme et prévisible.

3. Journalisation dans un fichier avec horodatage (logging) (15min)

La journalisation consiste à écrire des événements d'exécution dans un fichier texte, avec un horodatage, pour permettre l'audit et le dépannage ultérieur. Chaque ligne de journal comporte généralement une date et une heure au format normalisé, suivies du message descriptif de l'événement. L'opérateur de sous-expression $(Get-Date -Format 'yyyy-MM-dd HH:mm:ss') produit un horodatage lisible et triable (année-mois-jour heure:minute:seconde). La cmdlet Add-Content ajoute la ligne à la fin du fichier sans écraser le contenu existant, ce qui constitue le mode approprié pour un fichier de log.

```
function Ecrire-Log {
    param (
        [string]$Message,
        [string]$CheminLog
    )
    $horodatage = Get-Date -Format 'yyyy-MM-dd HH:mm:ss'
    # Get-Date -Format : formate la date et l'heure courantes selon le modèle indiqué
    # 'yyyy-MM-dd HH:mm:ss' : modèle produisant AAAA-MM-JJ HH:MM:SS, lisible et triable
    $ligne = "[$horodatage] $Message"
    # $ligne : chaîne constituée de l'horodatage entre crochets, suivie du message
    Add-Content -Path $CheminLog -Value $ligne -Encoding UTF8
    # Add-Content : ajoute la ligne à la fin du fichier de journalisation sans écraser l'existant
    # -Path : chemin du fichier de log cible
    # -Value : contenu à ajouter à la fin du fichier
    # -Encoding UTF8 : force l'encodage UTF-8 (sans BOM dans PowerShell 7+)
}
# Ecrire-Log : fonction personnalisée d'écriture de journal, factorisant l'horodatage et l'ajout
```

BON À SAVOIR

Le format 'yyyy-MM-dd HH:mm:ss' produit un horodatage lisible et triable (année-mois-jour heure:minute:seconde), adapté aux journaux d'administration. L'encodage UTF-8 garantit l'interopérabilité des fichiers de log entre systèmes.

4. Script complet RapportServices.ps1 : vérification et démarrage d'un service (15min)

Le script ci-dessous assemble les trois notions précédentes : un bloc param déclare les paramètres nommés ($NomService et $CheminLog), $ErrorActionPreference = 'Stop' rend les erreurs attrapables, la fonction Ecrire-Log journalise chaque étape avec horodatage, et un bloc try/catch entoure le traitement principal. Le script récupère l'état du service spécifié avec Get-Service ; si le service est arrêté (Status vaut 'Stopped'), il le démarre avec Start-Service et journalise l'action. En cas d'erreur (service introuvable, droits insuffisants, etc.), le bloc catch intercepte l'exception, journalise le message et l'affiche à l'écran.

```
param (
    [string]$NomService = 'Spooler',
    [string]$CheminLog = 'C:\Travail\rapport_services.log'
)
# param : bloc de déclaration des paramètres nommés du script
# [string] : typage explicite des paramètres
# $NomService : nom interne du service Windows à vérifier et éventuellement démarrer
# $CheminLog : chemin du fichier de journalisation, avec valeur par défaut

$ErrorActionPreference = 'Stop'
# $ErrorActionPreference : transforme les erreurs non terminantes en erreurs terminantes
# 'Stop' : rend les erreurs attrapables par le bloc try/catch ci-dessous

function Ecrire-Log {
    param (
        [string]$Message,
        [string]$CheminLog
    )
    $horodatage = Get-Date -Format 'yyyy-MM-dd HH:mm:ss'
    # Get-Date -Format : produit l'horodatage au format AAAA-MM-JJ HH:MM:SS
    $ligne = "[$horodatage] $Message"
    Add-Content -Path $CheminLog -Value $ligne -Encoding UTF8
    # Add-Content : ajoute la ligne horodatée à la fin du fichier de log
    # -Path : chemin du fichier de journalisation
    # -Value : ligne de journal à ajouter
    # -Encoding UTF8 : encodage UTF-8 sans BOM (PowerShell 7+)
}

try {
    Ecrire-Log -Message "Démarrage du rapport sur le service $NomService" -CheminLog $CheminLog
    # Ecrire-Log : journalise le début du rapport avec le nom du service
    # -Message : message à journaliser
    # -CheminLog : chemin du fichier de log cible

    $service = Get-Service -Name $NomService
    # Get-Service : récupère l'objet ServiceController du service spécifié
    # -Name : nom interne du service à récupérer
    # $service : variable stockant l'objet service retourné

    Ecrire-Log -Message "Statut du service : $($service.Status)" -CheminLog $CheminLog
    # $service.Status : propriété Status de l'objet service (Running ou Stopped)

    if ($service.Status -eq 'Stopped') {
        # -eq : opérateur d'égalité PowerShell
        # Si le service est arrêté, on le démarre
        Start-Service -Name $NomService
        # Start-Service : démarre le service spécifié s'il est actuellement arrêté
        # -Name : nom interne du service à démarrer
        Ecrire-Log -Message "Service $NomService démarré." -CheminLog $CheminLog
    }

    Ecrire-Log -Message "Rapport terminé avec succès." -CheminLog $CheminLog
} catch {
    Ecrire-Log -Message "Erreur : $($_.Exception.Message)" -CheminLog $CheminLog
    # $_.Exception.Message : message d'erreur de l'exception interceptée
    Write-Host "Une erreur est survenue : $($_.Exception.Message)"
    # Write-Host : affiche le message d'erreur à la console
}
```

```
# try : bloc contenant le code susceptible de lever une erreur (récupération et démarrage du service)
# catch : bloc exécuté si une erreur terminante survient dans le bloc try
```

Résultat attendu : le script vérifie l'état du service spécifié, le démarre si nécessaire, et écrit chaque étape dans le fichier de journalisation avec horodatage. En cas d'erreur, le message d'erreur est journalisé et affiché à la console.

Fig 4 — Flux du cas pratique avec try/catch et logging

Diagramme		
B	→	
C	→	
D	→	
E	**Oui**	
F	→	
G	→	
D	**Erreur**	
I	→	
J	→	
C	→	
K	→	J

Figure — Diagramme Mermaid

Lab 16 — Cas pratique PowerShell (45min)

Environnement

Élément	Valeur
Poste	Windows avec PowerShell 5.1 ou PowerShell 7+
Répertoire de travail	C:\Travail\Lab16 (à créer)
Prérequis	Droits suffisants pour interroger et démarrer les services Windows

Exercices (les exercices s'enchaînent pour former un lab complet)

Exercice 16.1 — Créer le script RapportServices.ps1 avec bloc param et logging (20min)

Créer le fichier C:\Travail\Lab16\RapportServices.ps1 contenant un bloc param avec deux paramètres nommés ($NomService et $CheminLog), la fonction Ecrire-Log avec horodatage, et un bloc try/catch qui récupère l'état du service et le journalise.

```
# Contenu de C:\Travail\Lab16\RapportServices.ps1
param (
    [string]$NomService = 'Spooler',
    [string]$CheminLog = 'C:\Travail\Lab16\rapport.log'
)
# param : bloc de déclaration des paramètres nommés
# [string] : typage explicite des paramètres
# $NomService : nom du service à surveiller, valeur par défaut Spooler
# $CheminLog : chemin du fichier de log, valeur par défaut C:\Travail\Lab16\rapport.log

$ErrorActionPreference = 'Stop'
# $ErrorActionPreference : rend les erreurs attrapables par try/catch
# 'Stop' : transforme les erreurs non terminantes en erreurs terminantes

function Ecrire-Log {
    param (
        [string]$Message,
        [string]$CheminLog
    )
    $horodatage = Get-Date -Format 'yyyy-MM-dd HH:mm:ss'
    # Get-Date -Format : produit l'horodatage au format AAAA-MM-JJ HH:MM:SS
    $ligne = "[$horodatage] $Message"
    Add-Content -Path $CheminLog -Value $ligne -Encoding UTF8
    # Add-Content : ajoute la ligne à la fin du fichier de log
    # -Path : chemin du fichier cible
    # -Value : contenu à ajouter
    # -Encoding UTF8 : encodage UTF-8 sans BOM (PowerShell 7+)
}

try {
    Ecrire-Log -Message "Démarrage du rapport pour le service $NomService" -CheminLog $CheminLog
    # Ecrire-Log : journalise le début du rapport
    # -Message : message à journaliser
    # -CheminLog : chemin du fichier de log

    $service = Get-Service -Name $NomService
    # Get-Service : récupère l'objet ServiceController du service
    # -Name : nom interne du service à récupérer

    Ecrire-Log -Message "Statut actuel : $($service.Status)" -CheminLog $CheminLog
    # $service.Status : propriété Status de l'objet service (Running ou Stopped)

    Ecrire-Log -Message "Rapport terminé." -CheminLog $CheminLog
} catch {
    Ecrire-Log -Message "Erreur : $($_.Exception.Message)" -CheminLog $CheminLog
    # $_.Exception.Message : message d'erreur de l'exception interceptée
    Write-Host "Erreur : $($_.Exception.Message)"
    # Write-Host : affiche le message d'erreur à la console
}
# try : bloc contenant le code susceptible de lever une erreur
# catch : bloc exécuté si une erreur terminante survient dans le bloc try
```

- Le fichier C:\Travail\Lab16\RapportServices.ps1 est créé
- Le script contient un bloc param avec deux paramètres nommés typés
- La fonction Ecrire-Log est définie avec horodatage et encodage UTF-8

- Le bloc try/catch entoure le traitement principal

Exercice 16.2 — Ajouter le démarrage conditionnel du service (10min)

Modifier le script pour qu'il démarre le service si son statut est 'Stopped', puis journalise l'action de démarrage.

```
# Ajout dans le bloc try, après la journalisation du statut actuel
if ($service.Status -eq 'Stopped') {
    # -eq : opérateur d'égalité PowerShell
    # Si le service est arrêté, on procède au démarrage
    Start-Service -Name $NomService
    # Start-Service : démarre le service spécifié
    # -Name : nom interne du service à démarrer
    Ecrire-Log -Message "Service $NomService démarré." -CheminLog $CheminLog
    # Ecrire-Log : journalise l'action de démarrage du service
}
```

- Le script contient une condition if sur $service.Status -eq 'Stopped'
- Start-Service est appelé avec le paramètre -Name lorsque le service est arrêté
- L'action de démarrage est journalisée avec horodatage

Exercice 16.3 — Tester la gestion d'erreurs avec un service inexistant (10min)

Exécuter le script avec un nom de service qui n'existe pas, et observer que le bloc catch intercepte l'erreur, journalise le message et l'affiche à l'écran.

```
# Exécution avec un service inexistant
& 'C:\Travail\Lab16\RapportServices.ps1' -NomService 'ServiceInexistant' -CheminLog 'C:\Travail\Lab16\rapp
# & : opérateur d'appel, exécute le script dans un scope isolé
# -NomService : transmet un nom de service qui n'existe pas sur le système
# -CheminLog : transmet le chemin du fichier de journalisation
```

```
# Lecture du fichier de log pour vérifier l'enregistrement de l'erreur
Get-Content -Path 'C:\Travail\Lab16\rapport.log'
# Get-Content : lit et affiche le contenu du fichier de journalisation
# -Path : chemin du fichier de log à lire
```

- Le bloc catch intercepte l'erreur du service introuvable
- Le message d'erreur est écrit dans le fichier de log avec horodatage
- Le message d'erreur s'affiche aussi à l'écran via Write-Host

Exercice 16.4 — Vérifier le fonctionnement sans erreur sur un service existant (5min)

Exécuter le script avec un service existant (par exemple Spooler) et vérifier que le rapport de succès est journalisé, y compris le démarrage si le service était arrêté.

```
& 'C:\Travail\Lab16\RapportServices.ps1' -NomService 'Spooler' -CheminLog 'C:\Travail\Lab16\rapport.log'
# & : opérateur d'appel, exécute le script avec les paramètres nommés
# -NomService 'Spooler' : cible le service d'impression Spooler
# -CheminLog : indique le fichier de journalisation

Get-Content -Path 'C:\Travail\Lab16\rapport.log'
# Get-Content : affiche le contenu complet du fichier de log
# -Path : chemin du fichier de log à lire
```

- Le statut du service Spooler est journalisé

- Le message "Rapport terminé" apparaît dans le log
- Si le service était arrêté, le message de démarrage apparaît dans le log

Démonstration commentée

Exécution pas à pas du script de cas pratique complet (RapportServices.ps1) :

```
# 1. Déclaration des paramètres nommés
param (
    [string]$NomService = 'Spooler',
    [string]$CheminLog = 'C:\Travail\Lab16\rapport.log'
)
# param : définit l'interface du script avec valeurs par défaut
# [string] : typage explicite de chaque paramètre

# 2. Configuration de la gestion d'erreurs
$ErrorActionPreference = 'Stop'
# $ErrorActionPreference : toute erreur devient terminante et attrapable par try/catch
# 'Stop' : transforme les erreurs non terminantes en erreurs terminantes

# 3. Définition de la fonction de journalisation
function Ecrire-Log {
    param (
        [string]$Message,
        [string]$CheminLog
    )
    $horodatage = Get-Date -Format 'yyyy-MM-dd HH:mm:ss'
    # Get-Date -Format : produit l'horodatage au format AAAA-MM-JJ HH:MM:SS
    $ligne = "[$horodatage] $Message"
    Add-Content -Path $CheminLog -Value $ligne -Encoding UTF8
    # Add-Content : ajoute la ligne horodatée à la fin du fichier de log
}

# 4. Traitement principal dans un bloc try
try {
    Ecrire-Log -Message "Démarrage du rapport pour le service $NomService" -CheminLog $CheminLog
    # Journalise le début du rapport

    $service = Get-Service -Name $NomService
    # Get-Service : récupère l'objet ServiceController du service
    # -Name : nom interne du service à récupérer

    Ecrire-Log -Message "Statut actuel : $($service.Status)" -CheminLog $CheminLog
    # Journalise le statut actuel du service (Running ou Stopped)

    if ($service.Status -eq 'Stopped') {
        # -eq : opérateur d'égalité PowerShell
        Start-Service -Name $NomService
        # Start-Service : démarre le service s'il est arrêté
        # -Name : nom interne du service à démarrer
        Ecrire-Log -Message "Service $NomService démarré." -CheminLog $CheminLog
        # Journalise l'action de démarrage
    }

    Ecrire-Log -Message "Rapport terminé avec succès." -CheminLog $CheminLog
    # Journalise la fin réussie du rapport
} catch {
    Ecrire-Log -Message "Erreur : $($_.Exception.Message)" -CheminLog $CheminLog
    # $_.Exception.Message : message d'erreur de l'exception interceptée
    Write-Host "Une erreur est survenue : $($_.Exception.Message)"
    # Write-Host : affiche le message d'erreur à la console
}
# try : bloc contenant le code susceptible de lever une erreur
# catch : bloc exécuté si une erreur terminante survient dans le bloc try
```

Résultat attendu : le script s'exécute sans interruption brutale, journalise chaque étape avec horodatage dans le fichier de log, démarre le service si nécessaire, et en cas d'erreur (service introuvable par exemple) capture l'exception, l'écrit dans le log et l'affiche à l'écran.

Interprétation ligne par ligne :

- Le bloc param déclare deux paramètres nommés typés avec valeurs par défaut, définissant l'interface du script.
- $ErrorActionPreference = 'Stop' garantit que toute erreur, même non terminante par défaut, devient attrapable par try/catch.
- La fonction Ecrire-Log encapsule l'écriture journalisée : Get-Date -Format produit l'horodatage, et Add-Content ajoute la ligne sans écraser l'existant.
- Dans le bloc try, Get-Service récupère l'objet ServiceController ; sa propriété Status indique si le service est Running ou Stopped.
- Si le service est arrêté, Start-Service le démarre et l'action est journalisée.
- Le bloc catch intercepte toute erreur terminante (par exemple service introuvable), journalise le message d'exception et l'affiche à la console.

Correction et résolution pas à pas

Solution

Solution du Lab 16

```
# Exercice 16.1 — Contenu de C:\Travail\Lab16\RapportServices.ps1
param (
    [string]$NomService = 'Spooler',
    [string]$CheminLog = 'C:\Travail\Lab16\rapport.log'
)
# param : bloc de déclaration des paramètres nommés
# [string] : typage explicite des paramètres

$ErrorActionPreference = 'Stop'
# $ErrorActionPreference : rend les erreurs attrapables par try/catch

function Ecrire-Log {
    param (
        [string]$Message,
        [string]$CheminLog
    )
    $horodatage = Get-Date -Format 'yyyy-MM-dd HH:mm:ss'
    # Get-Date -Format : produit l'horodatage au format AAAA-MM-JJ HH:MM:SS
    $ligne = "[$horodatage] $Message"
    Add-Content -Path $CheminLog -Value $ligne -Encoding UTF8
    # Add-Content : ajoute la ligne à la fin du fichier de log
}

try {
    Ecrire-Log -Message "Démarrage du rapport pour le service $NomService" -CheminLog $CheminLog
    $service = Get-Service -Name $NomService
    # Get-Service : récupère l'objet ServiceController du service
    # -Name : nom interne du service à récupérer
    Ecrire-Log -Message "Statut actuel : $($service.Status)" -CheminLog $CheminLog

    # Exercice 16.2 — Démarrage conditionnel du service (à insérer dans le bloc try, après le journal du
    if ($service.Status -eq 'Stopped') {
        # -eq : opérateur d'égalité PowerShell
        # Si le service est arrêté, on procède au démarrage
        Start-Service -Name $NomService
        # Start-Service : démarre le service spécifié
        # -Name : nom interne du service à démarrer
        Ecrire-Log -Message "Service $NomService démarré." -CheminLog $CheminLog
        # Ecrire-Log : journalise l'action de démarrage du service
    }

    Ecrire-Log -Message "Rapport terminé." -CheminLog $CheminLog
} catch {
    Ecrire-Log -Message "Erreur : $($_.Exception.Message)" -CheminLog $CheminLog
    # $_.Exception.Message : message d'erreur de l'exception interceptée
    Write-Host "Erreur : $($_.Exception.Message)"
}

# Exercice 16.3 — Exécution avec un service inexistant
& 'C:\Travail\Lab16\RapportServices.ps1' -NomService 'ServiceInexistant' -CheminLog 'C:\Travail\Lab16\ra
# & : opérateur d'appel, exécute le script dans un scope isolé
Get-Content -Path 'C:\Travail\Lab16\rapport.log'
# Get-Content : affiche le contenu du fichier de journalisation

# Exercice 16.4 — Exécution avec un service existant
& 'C:\Travail\Lab16\RapportServices.ps1' -NomService 'Spooler' -CheminLog 'C:\Travail\Lab16\rapport.log'
Get-Content -Path 'C:\Travail\Lab16\rapport.log'
```

Résolution pas à pas :

- **Bloc param** : déclare deux paramètres nommés typés ($NomService et $CheminLog) avec valeurs par défaut. Le script peut être appelé sans arguments (valeurs par défaut utilisées) ou avec des arguments personnalisés transmis par nom.
- **$ErrorActionPreference = 'Stop'** : transforme les erreurs non terminantes en erreurs terminantes, permettant leur interception par le bloc try/catch. Sans cette directive, une erreur de Get-Service (service introuvable) ne serait pas interceptée et le script continuerait silencieusement.
- **Function Ecrire-Log** : factorise l'écriture journalisée. Get-Date -Format 'yyyy-MM-dd HH:mm:ss' produit un horodatage normalisé. Add-Content ajoute la ligne à la fin du fichier sans écraser l'existant, préservant l'historique des exécutions.
- **Bloc try** : tente de récupérer le service avec Get-Service -Name et de journaliser son statut. Si Get-Service échoue (service introuvable ou droits insuffisants), l'exécution saute immédiatement au bloc catch.
- **Condition if ($service.Status -eq 'Stopped')** : vérifie si le service est arrêté. Si oui, Start-Service le démarre et l'action est journalisée. Le opérateur -eq compare la propriété Status à la chaîne 'Stopped'.
- **Bloc catch** : journalise le message d'exception ($_.Exception.Message) dans le fichier de log et l'affiche à l'écran avec Write-Host. Le script se termine proprement sans interruption brutale.
- **Opérateur d'appel &** : exécute le script dans un scope isolé, permettant de passer des paramètres nommés (-NomService, -CheminLog) à chaque exécution.

Bonnes pratiques

Pratique	Description
Bloc param en début de script	Déclarer tous les paramètres nommés typés avant toute autre instruction
$ErrorActionPreference = 'Stop'	Rendre les erreurs attrapables par try/catch pour une gestion uniforme
Journalisation avec horodatage	Écrire chaque étape significative dans un fichier de log avec Add-Content
Fonction de log dédiée	Factoriser l'écriture de log pour éviter la répétition et garantir la cohérence du format
Vérification avant action	Tester l'état du service avec Get-Service avant d'appliquer Start-Service
Encodage UTF-8	Utiliser -Encoding UTF8 pour garantir l'interopérabilité des fichiers de log

Script de vérification de l'état d'un service avant démarrage sécurisé :

```powershell
# Vérification de l'état d'un service avant démarrage sécurisé
param (
    [string]$NomService = 'Spooler'
)
# param : déclare le paramètre nommé du service à vérifier
# [string] : typage explicite du paramètre

$ErrorActionPreference = 'Stop'
# $ErrorActionPreference : rend les erreurs attrapables par try/catch

try {
    $service = Get-Service -Name $NomService
    # Get-Service : récupère l'objet ServiceController du service
    # -Name : nom interne du service à récupérer

    if ($service.Status -eq 'Running') {
        # -eq : opérateur d'égalité PowerShell
        Write-Host "Le service $NomService est déjà en cours d'exécution."
        # Write-Host : affiche un message d'information à la console
    } else {
        Start-Service -Name $NomService
        # Start-Service : démarre le service s'il est arrêté
        # -Name : nom interne du service à démarrer
        Write-Host "Le service $NomService a été démarré."
    }
} catch {
    Write-Host "Impossible de gérer le service $NomService : $($_.Exception.Message)"
    # $_.Exception.Message : message d'erreur de l'exception interceptée
}
# try : bloc contenant le code susceptible de lever une erreur
# catch : bloc exécuté si une erreur terminante survient dans le bloc try
```

BONNES PRATIQUES

Un script d'administration robuste déclare ses paramètres dans un bloc param, configure une gestion d'erreurs uniforme avec $ErrorActionPreference = 'Stop', vérifie l'état de la cible avant d'agir, et journalise chaque étape significative avec horodatage pour faciliter l'audit et le dépannage.

Exercices — Chapitre 16

Exercice 16.5 — Ajout d'un bloc trap pour les erreurs non gérées (10min)

Ajouter un bloc trap au script RapportServices.ps1 pour capturer toute erreur non gérée dans le scope courant, et expliquer son rôle complémentaire par rapport à try/catch.

- Quelle est la différence entre try/catch et trap ?
- Où placer le bloc trap dans le script ?

Solution

Solution

```
# Ajout en début de script, après le bloc param et avant le bloc try
trap {
    $horodatage = Get-Date -Format 'yyyy-MM-dd HH:mm:ss'
    # Get-Date -Format : produit l'horodatage au format AAAA-MM-JJ HH:MM:SS
    $ligne = "[$horodatage] Erreur non gérée : $($_.Exception.Message)"
    Add-Content -Path $CheminLog -Value $ligne -Encoding UTF8
    # Add-Content : ajoute la ligne d'erreur à la fin du fichier de log
    # -Path : chemin du fichier de journalisation
    # -Value : ligne d'erreur horodatée à ajouter
    # -Encoding UTF8 : encodage UTF-8 sans BOM (PowerShell 7+)
    Write-Host "Erreur critique : $($_.Exception.Message)"
    # Write-Host : affiche l'erreur critique à la console
    continue
    # continue : reprend l'exécution après le traitement de l'erreur
}
# trap : bloc exécuté pour toute erreur non gérée dans le scope courant
# continue : reprend l'exécution après le traitement de l'erreur
```

Résolution pas à pas :

- try/catch intercepte les erreurs survenues dans un bloc try spécifique. trap intercepte toute erreur non gérée dans le scope courant, y compris en dehors d'un bloc try. Ils sont complémentaires : try/catch pour les erreurs anticipées (récupération du service, démarrage), trap pour les erreurs imprévues survenues en dehors du bloc try.
- Le bloc trap peut être placé n'importe où dans le script, mais il est recommandé de le déclarer en début de script, après le bloc param et la définition de $ErrorActionPreference, pour qu'il soit actif sur toute l'exécution. Le mot-clé continue permet de reprendre l'exécution après le traitement de l'erreur ; break l'arrêterait.

Ce qu'on a retenu

CE QU'ON A RETENU

Le bloc param déclare les paramètres nommés typés d'un script en début de fichier. $ErrorActionPreference = 'Stop' rend les erreurs attrapables par try/catch. Le bloc try entoure le code susceptible d'échouer, et catch intercepte l'exception pour journaliser et afficher le message. trap gère les erreurs non gérées dans le scope. La journalisation avec Add-Content et un horodatage normalisé permet d'auditer l'exécution du script. Un script d'administration robuste combine paramètres nommés, gestion d'erreurs (try/catch, trap, $ErrorActionPreference) et logging, et vérifie l'état d'un service avant de le démarrer si nécessaire.

Synthèse

Couverture Jour 4

Chapitre	Thème
13	Manipulation de fichiers et dossiers
14	Notions d'objets PowerShell (approche objet VS Bash)
15	Gestion des processus et des droits
16	Cas pratique PowerShell

Mémo — Commandes Essentielles

```
# Fichiers et dossiers
New-Item -Path 'chemin' -ItemType Directory|File   # Crée un dossier ou un fichier
Test-Path -Path 'chemin'                           # Vérifie l'existence d'un chemin
Get-ChildItem -Path 'chemin'                       # Liste le contenu d'un répertoire
Copy-Item -Path 'source' -Destination 'cible'      # Duplique un fichier ou dossier
Move-Item -Path 'source' -Destination 'cible'      # Déplace un élément
Rename-Item -Path 'chemin' -NewName 'nom'          # Renomme un élément
Remove-Item -Path 'chemin'                         # Supprime un élément
Get-Content -Path 'fichier'                        # Lit le contenu d'un fichier
Set-Content -Path 'fichier' -Value 'texte'         # Écrit (écrase) le contenu d'un fichier
Add-Content -Path 'fichier' -Value 'texte'         # Ajoute du contenu à la fin d'un fichier
-Encoding UTF8                                     # Force l'encodage UTF-8

# Objets
Get-Member                                         # Introspecte les propriétés et méthodes d'un objet
Select-Object -Property nom1, nom2                 # Sélectionne des propriétés
Where-Object { condition }                         # Filtre les objets selon une condition
ForEach-Object { action }                          # Exécute une action sur chaque objet
Format-Table -AutoSize                             # Affiche sous forme de tableau
Format-List                                        # Affiche chaque propriété sur une ligne
Format-Wide                                        # Affiche sur plusieurs colonnes
Export-Csv -Path 'fichier.csv' -NoTypeInformation  # Exporte vers un fichier CSV
ConvertTo-Json                                     # Convertit en format JSON

# Processus et services
Get-Process -Name 'nom'                            # Liste un processus
Start-Process -FilePath 'exécutable'               # Lance un processus
Stop-Process                                       # Arrête un processus
Get-Service -Name 'nom'                            # Récupère un service
Start-Service -Name 'nom'                          # Démarre un service
Stop-Service -Name 'nom'                           # Arrête un service
Restart-Service -Name 'nom'                        # Redémarre un service

# Droits et comptes locaux
Get-Acl -Path 'chemin'                             # Lit la liste de contrôle d'accès
Set-Acl -Path 'chemin'                             # Applique une liste de contrôle d'accès
Get-LocalUser                                      # Liste les utilisateurs locaux
New-LocalUser -Name 'nom' -Password (Read-Host -AsSecureString)  # Crée un utilisateur local
Add-LocalGroupMember -Group 'groupe' -Member 'user' # Ajoute un utilisateur à un groupe

# Script et erreurs
param (...)                                        # Bloc de déclaration des paramètres
$ErrorActionPreference = 'Stop'                    # Rend les erreurs attrapables
try { ... } catch { ... }                          # Intercepte les erreurs terminantes
trap { ... }                                       # Intercepte les erreurs non gérées
Get-Date -Format 'yyyy-MM-dd HH:mm:ss'             # Horodatage pour la journalisation
```

Ressources supplémentaires

Outils utilisés

Outil	Version	Usage
PowerShell	7+	Environnement de scripting et d'administration système

Scripting Bash et PowerShell — EXTRA

Python et Cryptographie — Modules complémentaires (17/09/2026)

FIL CONDUCTEUR

Ce module complémentaire prolonge les 4 jours de formation (Bash J1-J2, PowerShell J3-J4) avec l'administration système en Python (chapitres 17-20) et la cryptographie symétrique/asymétrique (chapitres 21-24).

Objectifs du Module

- Écrire des scripts d'administration système en Python
- Manipuler fichiers, processus et environnement avec Python
- Comprendre et appliquer la cryptographie symétrique et asymétrique
- Mettre en œuvre une PKI avec OpenSSL

Planning du Module

Chapitre	Horaires	Thème	Durée
17	9h00-10h30	Introduction à Python pour l'administration système	1h30
—	10h30-10h45	Pause	15min
18	10h45-12h00	Manipulation de fichiers, processus et système avec Python	1h15
—	12h00-13h00	Pause déjeuner	1h
19	13h00-14h30	Automatisation et scripting système avec Python	1h30
—	14h30-14h45	Pause	15min
20	14h45-16h00	Cas pratique Python — script d'administration complet	1h15
—	16h00-16h15	Pause	15min
21	16h15-17h45	Cryptographie symétrique — principes et algorithmes	1h30
—	17h45-18h00	Pause	15min
22	18h00-19h15	Cryptographie symétrique — mise en pratique (OpenSSL, Python)	1h15
—	19h15-20h15	Pause déjeuner	1h
23	20h15-21h45	Cryptographie asymétrique — principes et algorithmes	1h30
—	21h45-22h00	Pause	15min
24	22h00-23h15	Cryptographie asymétrique — PKI, certificats, signatures numériques	1h15

Chapitre 17 — Introduction à Python pour l'administration système (9h00-10h30)

Objectifs

- Comprendre ce qu'est Python et sa différence avec Bash
- Installer et vérifier Python sur un poste Linux
- Écrire et exécuter un premier script Python

Contenu Théorique (1h30)

1. Qu'est-ce que Python ? (30min)

Python est un langage de programmation interprété, de haut niveau, à typage dynamique. Contrairement à Bash qui traite tout comme du texte, Python manipule des objets typés.

```
# python3 --version : affiche la version de Python installée
# python3 : lance l'interpréteur Python
```

CONCEPT CLÉ

Python est un langage interprété : le code source est compilé en bytecode puis exécuté par la machine virtuelle Python.

BON À SAVOIR

Python est préinstallé sur la plupart des distributions Linux modernes ; la commande `which python3` permet de localiser l'interpréteur sur le poste.

2. Premier script Python (45min)

```
#!/usr/bin/env python3
# #!/usr/bin/env python3 : shebang indiquant l'interpréteur Python
print("Bonjour, administration système")
# print : affiche une chaîne de caractères sur la sortie standard
```

Fig 1 — Flux d'exécution d'un script Python



Figure — Diagramme Mermaid

Lab 17 — Premier script Python (45min)

Environnement

Élément	Valeur
Poste	Linux
Interpréteur	Python 3

Exercices

Exercice 17.1 — Vérifier la version de Python (10min)

```
python3 --version
# python3 : lance l'interpréteur Python 3
# --version : affiche le numéro de version
```

- La version de Python s'affiche correctement

Exercice 17.2 — Créer et exécuter un premier script (15min)

```
nano hello.py
# nano : éditeur de texte en mode terminal
# hello.py : nom du fichier script Python
```

```
#!/usr/bin/env python3
print("Bonjour, monde")
```

```
chmod 755 hello.py
# chmod : modifie les droits d'accès
# 755 : lecture/écriture/exécution pour le propriétaire, lecture/exécution pour les autres
./hello.py
# ./ : exécute le script depuis le répertoire courant
```

- Le script hello.py s'exécute et affiche "Bonjour, monde"

Démonstration commentée

Le shebang `#!/usr/bin/env python3` permet au noyau Linux de trouver l'interpréteur Python via la variable d'environnement PATH, rendant le script portable.

Correction et résolution pas à pas

Solution

Solution Exercice 17.2

```
#!/usr/bin/env python3
print("Bonjour, monde")
```

Bonnes pratiques

Pratique	Description
Shebang	Toujours commencer par `#!/usr/bin/env python3`
Droits	`chmod 755` pour rendre le script exécutable

BONNES PRATIQUES

Le shebang avec `env` est préférable à un chemin fixe car il s'adapte à l'installation de Python sur chaque poste.

Exercices — Chapitre 17

Exercice 17.3 — Afficher des variables (10min)

```
nom = "Admin"
version = 3
print(f"Utilisateur : {nom}, Python : {version}")
```

- Les variables s'affichent correctement

  Solution

  Solution Exercice 17.3

  ```
  nom = "Admin"
  version = 3
  print(f"Utilisateur : {nom}, Python : {version}")
  ```

Ce qu'on a retenu

> **Ce qu'on a retenu**

> - Python est un langage interprété à typage dynamique

> - Le shebang `#!/usr/bin/env python3` rend le script portable

> - `chmod 755` rend le script exécutable

Chapitre 18 — Manipulation de fichiers, processus et système avec Python (10h45-12h00)

Objectifs

- Utiliser les modules os, shutil, subprocess
- Lire et écrire des fichiers avec open() et with
- Accéder aux variables d'environnement

Contenu Théorique (1h15)

1. Module os (30min)

```
import os
# import os : importe le module d'interaction avec le système d'exploitation

cwd = os.getcwd()
# os.getcwd() : retourne le répertoire de travail courant

print(os.listdir('.'))
# os.listdir() : liste le contenu d'un répertoire
```

CONCEPT CLÉ

Le module `os` fournit une interface portable aux appels système du noyau Linux.

POURQUOI

Il offre une couche d'abstraction portable : un même script Python fonctionne sur Linux, Windows ou macOS sans modification, car Python traduit les appels vers l'API du système hôte.

2. Lecture/écriture de fichiers (45min)

```
with open('fichier.txt', 'w') as f:
    # open() : ouvre un fichier
    # 'w' : mode écriture (écrase le contenu existant)
    # with : garantit la fermeture automatique du fichier
    f.write("Contenu du fichier\n")
    # f.write() : écrit une chaîne dans le fichier
```

Fig 2 — Interaction Python avec le système d'exploitation

Diagramme		
Script Python	**subprocess**	
Script Python	**os.environ**	D

Figure — Diagramme Mermaid

Lab 18 — Manipulation de fichiers (45min)

Environnement

Élément	Valeur
Poste	Linux
Répertoire de travail	/tmp/lab_python

Exercices

Exercice 18.1 — Lister un répertoire (10min)

```
import os
print(os.listdir('/tmp'))
```

- Le contenu de /tmp s'affiche

Exercice 18.2 — Créer et écrire un fichier (15min)

```
mkdir -p /tmp/lab_python
# mkdir -p : crée le répertoire de travail (et ses parents) s'il n'existe pas
# /tmp/lab_python : cible du laboratoire (prérequis indispensable avant toute écriture de fichier)
```

```
with open('/tmp/lab_python/test.txt', 'w') as f:
    f.write("Test Python\n")
```

- Le fichier test.txt est créé et contient "Test Python"

Démonstration commentée

Le bloc `with` garantit que le fichier est fermé même en cas d'erreur, évitant les fuites de descripteurs de fichier.

Correction et résolution pas à pas

Solution

Solution Exercice 18.2

```
with open('/tmp/lab_python/test.txt', 'w') as f:
    f.write("Test Python\n")
```

Bonnes pratiques

Pratique	Description
with statement	Toujours utiliser `with` pour ouvrir des fichiers
Chemins absolus	Utiliser des chemins absolus en lab pour éviter les ambiguïtés

BONNES PRATIQUES

Le `with` statement est la méthode recommandée pour gérer les ressources (fichiers, connexions) en Python.

Exercices — Chapitre 18

Exercice 18.3 — Lire un fichier (10min)

```
with open('/tmp/lab_python/test.txt', 'r') as f:
    print(f.read())
```

- Le contenu du fichier s'affiche

Solution

Solution Exercice 18.3

```
with open('/tmp/lab_python/test.txt', 'r') as f:
    print(f.read())
```

Ce qu'on a retenu

> **Ce qu'on a retenu**

> - Le module `os` permet d'interagir avec le système d'exploitation

> - Le `with` statement garantit la fermeture des fichiers

> - `os.listdir()` liste le contenu d'un répertoire

Chapitre 19 — Automatisation et scripting système avec Python (13h00-14h30)

Objectifs

- Gérer les erreurs avec try/except/finally
- Configurer le logging avec le module logging
- Utiliser argparse pour les arguments de ligne de commande

Contenu Théorique (1h30)

1. Gestion des erreurs (45min)

```
try:
    # try : bloc contenant le code susceptible de lever une exception
    fichier = open('inexistant.txt', 'r')
except FileNotFoundError:
    # except : bloc exécuté si l'exception spécifiée se produit
    print("Le fichier n'existe pas")
finally:
    # finally : bloc toujours exécuté, avec ou sans exception
    print("Nettoyage terminé")
```

CONCEPT CLÉ

La gestion d'erreurs avec `try/except` permet de contrôler le flux du programme en cas d'exception, évitant l'arrêt brutal du script.

BON À SAVOIR

Les niveaux de logging, du plus détaillé au plus critique, sont : DEBUG, INFO, WARNING, ERROR et CRITICAL. Ils permettent de filtrer les messages selon l'importance de l'événement.

2. Logging et argparse (45min)

```
import logging
import argparse

logging.basicConfig(level=logging.INFO)
# logging.basicConfig() : configure le système de logging

parser = argparse.ArgumentParser()
# argparse.ArgumentParser() : crée un parseur d'arguments de ligne de commande
parser.add_argument('--nom', required=True)
# add_argument() : ajoute un argument attendu
args = parser.parse_args()
# parse_args() : analyse les arguments passés en ligne de commande
```

Fig 3 — Flux de gestion d'erreurs et de logging

Diagramme		
B	erreur	
B	succès	
C	niveau	E

Figure — Diagramme Mermaid

Lab 19 — Automatisation (45min)

Environnement

Élément	Valeur
Poste	Linux
Python	3.x

Exercices

Exercice 19.1 — Gestion d'erreurs (15min)

```
try:
    x = 10 / 0
except ZeroDivisionError:
    print("Division par zéro interdite")
```

- L'erreur est interceptée et le message s'affiche

Exercice 19.2 — Script avec argparse (15min)

```
import argparse
parser = argparse.ArgumentParser()
parser.add_argument('--message', default='Hello')
args = parser.parse_args()
print(args.message)
```

- Le script accepte l'argument --message

Démonstration commentée

Le module `logging` permet de tracer l'exécution d'un script avec différents niveaux (DEBUG, INFO, WARNING, ERROR, CRITICAL), essentiel pour le débogage en production.

Correction et résolution pas à pas

Solution

Solution Exercice 19.1

```
try:
    x = 10 / 0
except ZeroDivisionError:
    print("Division par zéro interdite")
```

Bonnes pratiques

Pratique	Description
Logging	Utiliser logging au lieu de print pour les scripts de production
Arguments	Utiliser argparse pour les scripts en ligne de commande

BONNES PRATIQUES

Le logging est préférable à `print` car il permet de contrôler le niveau de détail et de rediriger les sorties vers un fichier.

Exercices — Chapitre 19

Exercice 19.3 — Logging (10min)

```
import logging
logging.basicConfig(level=logging.INFO)
logging.info("Démarrage du script")
```

- Le message de log s'affiche

Solution

Solution Exercice 19.3

```
import logging
logging.basicConfig(level=logging.INFO)
logging.info("Démarrage du script")
```

Ce qu'on a retenu

> **Ce qu'on a retenu**

> - `try/except/finally` gère les erreurs de manière contrôlée

> - `logging` trace l'exécution avec différents niveaux

> - `argparse` parse les arguments de ligne de commande

Chapitre 20 — Cas pratique Python — script d'administration complet (14h45-16h00)

Objectifs

- Intégrer plusieurs concepts dans un script complet
- Implémenter une sauvegarde automatisée avec rotation
- Tester l'exécution et vérifier les logs

Contenu Théorique (1h15)

1. Script de sauvegarde complet (45min)

```
#!/usr/bin/env python3
import os
import shutil
import logging
from datetime import datetime

logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s - %(message)s')

def sauvegarde(source, destination):
    # def : définit une fonction
    horodatage = datetime.now().strftime('%Y%m%d_%H%M%S')
    # datetime.now() : retourne la date et l'heure actuelles
    # strftime() : formate la date selon un modèle
    nom_archive = f"sauvegarde_{horodatage}.tar.gz"
    chemin_archive = os.path.join(destination, nom_archive)
    # os.path.join() : construit un chemin de fichier de manière portable

    logging.info(f"Début de la sauvegarde : {source} -> {chemin_archive}")
    shutil.make_archive(chemin_archive.replace('.tar.gz', ''), 'gztar', source)
    # shutil.make_archive() : crée une archive compressée
    logging.info(f"Sauvegarde terminée : {chemin_archive}")
```

Fig 4 — Architecture du script d'administration complet

Diagramme		
B	os/shutil	
C	logging	
C	rotation	E

Figure — Diagramme Mermaid

CONCEPT CLÉ

Un script d'administration complet combine plusieurs modules (os, shutil, logging, datetime) pour être modulaire, traçable et robuste face aux erreurs.

CONTRE-MESURE

Toujours valider l'existence des répertoires source et destination avant d'écrire, afin d'éviter les erreurs d'exécution en production lorsque les chemins sont absents.

Lab 20 — Cas pratique (45min)

Environnement

Élément	Valeur
Poste	Linux
Répertoire source	/tmp/lab_python/source
Répertoire destination	/tmp/lab_python/destination

Exercices

Exercice 20.1 — Créer le script de sauvegarde (20min)

```
#!/usr/bin/env python3
import os, shutil, logging
from datetime import datetime

logging.basicConfig(level=logging.INFO)

source = '/tmp/lab_python/source'
destination = '/tmp/lab_python/destination'

os.makedirs(source, exist_ok=True)
os.makedirs(destination, exist_ok=True)
with open(os.path.join(source, 'document.txt'), 'w') as f:
    f.write("Données à sauvegarder\n")
horodatage = datetime.now().strftime('%Y%m%d_%H%M%S')
archive = os.path.join(destination, f'sauvegarde_{horodatage}.tar.gz')
shutil.make_archive(archive.replace('.tar.gz', ''), 'gztar', source)
logging.info(f"Sauvegarde créée : {archive}")
```

- Le script s'exécute sans erreur
- Une archive est créée dans le répertoire de destination

Démonstration commentée

Le script combine plusieurs modules : `os` pour la gestion des répertoires, `shutil` pour la compression, `logging` pour le traçage, et `datetime` pour l'horodatage des archives.

Correction et résolution pas à pas

Solution

Solution Exercice 20.1

```
#!/usr/bin/env python3
import os, shutil, logging
from datetime import datetime

logging.basicConfig(level=logging.INFO)

source = '/tmp/lab_python/source'
destination = '/tmp/lab_python/destination'

os.makedirs(source, exist_ok=True)
os.makedirs(destination, exist_ok=True)
with open(os.path.join(source, 'document.txt'), 'w') as f:
    f.write("Données à sauvegarder\n")
horodatage = datetime.now().strftime('%Y%m%d_%H%M%S')
archive = os.path.join(destination, f'sauvegarde_{horodatage}.tar.gz')
shutil.make_archive(archive.replace('.tar.gz', ''), 'gztar', source)
logging.info(f"Sauvegarde créée : {archive}")
```

Bonnes pratiques

Pratique	Description
Modularité	Organiser le code en fonctions réutilisables
Logging	Traçer chaque étape importante du script

BONNES PRATIQUES

Un script d'administration doit être modulaire, traçable et robuste face aux erreurs.

Exercices — Chapitre 20

Exercice 20.2 — Vérifier l'archive (10min)

```
ls -lh /tmp/lab_python/destination/
# ls -lh : liste les fichiers avec tailles lisibles
```

- L'archive de sauvegarde est présente

Solution

Solution Exercice 20.2

```
ls -lh /tmp/lab_python/destination/
```

Ce qu'on a retenu

> **Ce qu'on a retenu**

- Un script complet intègre fichiers, processus, logging et gestion d'erreurs

- `shutil.make_archive` crée des archives compressées

- Le logging permet de tracer l'exécution en production

Chapitre 21 — Cryptographie symétrique — principes et algorithmes (16h15-17h45)

Objectifs

- Comprendre la cryptographie symétrique (clé unique partagée)
- Connaître les algorithmes DES, 3DES, AES
- Comprendre les modes d'opération et le vecteur d'initialisation

Contenu Théorique (1h30)

1. Principes de la cryptographie symétrique (45min)

La cryptographie symétrique utilise une seule clé secrète pour chiffrer et déchiffrer. L'expéditeur et le destinataire doivent partager cette clé de manière sécurisée.

CONCEPT CLÉ

La sécurité de la cryptographie symétrique repose entièrement sur la confidentialité de la clé partagée.

BON À SAVOIR

Le mode CBC nécessite un vecteur d'initialisation (IV) aléatoire et unique par chiffrement, afin que deux messages identiques chiffrés avec la même clé produisent des textes chiffrés différents.

2. Algorithmes et modes (45min)

Algorithme	Taille de clé	Statut
DES	56 bits	Obsolète
3DES	112/168 bits	Déconseillé
AES	128/192/256 bits	Standard actuel

Fig 5 — Flux de chiffrement symétrique

Diagramme		
B	IV	
C	clé secrète	
D	IV	Texte clair

Figure — Diagramme Mermaid

Lab 21 — Chiffrement symétrique (45min)

Environnement

Élément	Valeur
Outil	OpenSSL 3.x
Algorithme	AES-256-CBC

Exercices

Exercice 21.1 — Générer une clé symétrique (10min)

```
openssl rand -hex 32
# openssl : boîte à outils cryptographique
# rand : génère des octets aléatoires
# -hex 32 : affiche 32 octets en hexadécimal (256 bits)
```

- Une clé de 256 bits s'affiche

Exercice 21.2 — Chiffrer un fichier (15min)

```
echo "Données sensibles" > fichier.txt
openssl enc -aes-256-cbc -salt -pbkdf2 -in fichier.txt -out fichier.enc
# openssl enc : commande de chiffrement/déchiffrement
# -aes-256-cbc : algorithme AES-256 en mode CBC
# -salt : ajoute un sel pour renforcer la clé dérivée
# -pbkdf2 : fonction de dérivation de clé
# -in : fichier d'entrée
# -out : fichier de sortie chiffré
```

- Le fichier fichier.enc est créé

Démonstration commentée

L'option `-pbkdf2` dérive une clé cryptographique à partir d'un mot de passe, rendant l'attaque par dictionnaire plus difficile.

Correction et résolution pas à pas

Solution

Solution Exercice 21.2

```
openssl enc -aes-256-cbc -salt -pbkdf2 -in fichier.txt -out fichier.enc
```

Bonnes pratiques

Pratique	Description
Taille de clé	Utiliser AES-256 pour une sécurité maximale
IV unique	Ne jamais réutiliser le même IV avec la même clé

BONNES PRATIQUES

AES-256-CBC avec PBKDF2 est une combinaison robuste pour le chiffrement de fichiers.

Exercices — Chapitre 21

Exercice 21.3 — Déchiffrer un fichier (10min)

```
openssl enc -d -aes-256-cbc -salt -pbkdf2 -in fichier.enc -out fichier_dechiffre.txt
# -d : mode déchiffrement
```

- Le fichier déchiffré contient le texte original

Solution

Solution Exercice 21.3

```
openssl enc -d -aes-256-cbc -salt -pbkdf2 -in fichier.enc -out fichier_dechiffre.txt
```

Ce qu'on a retenu

> **Ce qu'on a retenu**

> - La cryptographie symétrique utilise une clé unique partagée

> - AES-256 est l'algorithme standard actuel

> - Le mode CBC avec un IV aléatoire renforce la sécurité

Chapitre 22 — Cryptographie symétrique — mise en pratique (OpenSSL, Python) (18h00-19h15)

Objectifs

- Utiliser OpenSSL pour chiffrer/déchiffrer des fichiers
- Utiliser le module Python cryptography (Fernet)
- Gérer sécuriséement les clés (chmod 600)

Contenu Théorique (1h15)

1. OpenSSL en pratique (45min)

```
openssl enc -aes-256-cbc -salt -pbkdf2 -in secret.txt -out secret.enc -pass pass:MonMotDePasse
# -pass pass: : spécifie le mot de passe directement
```

PIÈGE

Les mots de passe contenant des caractères spéciaux (`!`, `$`, `*`) peuvent être interprétés par le shell. Utiliser des guillemets simples ou échapper les caractères.

BON À SAVOIR

Fernet utilise le chiffrement symétrique AES-128 en mode CBC avec une authentification HMAC, garantissant à la fois la confidentialité et l'intégrité du message.

2. Python cryptography (Fernet) (30min)

```
from cryptography.fernet import Fernet
# from cryptography.fernet import Fernet : importe la classe Fernet

cle = Fernet.generate_key()
# Fernet.generate_key() : génère une clé symétrique Fernet

f = Fernet(cle)
jeton = f.encrypt(b"Message secret")
# f.encrypt() : chiffre un message (bytes requis)
```

Fig 6 — Pipeline de chiffrement avec OpenSSL

Diagramme		
B	IV aléatoire	
C	même clé	
D	fichier original	E

Figure — Diagramme Mermaid

Lab 22 — Mise en pratique (45min)

Environnement

Élément	Valeur
Outil	OpenSSL 3.x, Python 3.x
Module	cryptography

Exercices

Exercice 22.1 — Chiffrement OpenSSL avec mot de passe (15min)

```
echo "Confidentiel" > secret.txt
# echo : écrit la chaîne sur la sortie standard
# > secret.txt : redirige la sortie vers le fichier secret.txt (écrasement)
openssl enc -aes-256-cbc -salt -pbkdf2 -in secret.txt -out secret.enc -pass pass:Secret123
# openssl enc : commande de chiffrement/déchiffrement OpenSSL
# -aes-256-cbc : algorithme AES-256 en mode CBC
# -salt : ajoute un sel pour renforcer la clé dérivée
# -pbkdf2 : fonction de dérivation de clé à partir d'un mot de passe
# -in : fichier d'entrée en clair
# -out : fichier de sortie chiffré
# -pass pass: : spécifie le mot de passe directement en ligne de commande
```

- Le fichier secret.enc est créé

Exercice 22.2 — Chiffrement Python Fernet (15min)

```
from cryptography.fernet import Fernet
cle = Fernet.generate_key()
f = Fernet(cle)
jeton = f.encrypt(b"Message secret")
print(jeton)
```

- Le jeton chiffré s'affiche

Démonstration commentée

Fernet garantit que le message ne peut pas être lu ou modifié sans la clé. Il combine chiffrement et authentification (HMAC).

Correction et résolution pas à pas

Solution

Solution Exercice 22.2

```
from cryptography.fernet import Fernet
cle = Fernet.generate_key()
f = Fernet(cle)
jeton = f.encrypt(b"Message secret")
print(jeton)
```

Bonnes pratiques

Pratique	Description
Protection des clés	`chmod 600` sur les fichiers contenant des clés
Ne pas coder en dur	Utiliser des variables d'environnement pour les mots de passe

BONNES PRATIQUES

Une clé privée doit toujours être protégée par des droits restrictifs (`chmod 600`) et jamais codée en dur dans le script.

Exercices — Chapitre 22

Exercice 22.3 — Déchiffrement Python Fernet (10min)

```
from cryptography.fernet import Fernet
cle = Fernet.generate_key()
f = Fernet(cle)
jeton = f.encrypt(b"Message secret")
message = f.decrypt(jeton)
# f.decrypt() : déchiffre un jeton Fernet
print(message)
```

- Le message original s'affiche

Solution

Solution Exercice 22.3

```
from cryptography.fernet import Fernet
f = Fernet(cle)
message = f.decrypt(jeton)
print(message)
```

Ce qu'on a retenu

> **Ce qu'on a retenu**

> - OpenSSL permet le chiffrement symétrique en ligne de commande

- Fernet (Python) combine chiffrement et authentification

- Les clés doivent être protégées (chmod 600)

Chapitre 23 — Cryptographie asymétrique — principes et algorithmes (20h15-21h45)

Objectifs

- Comprendre la cryptographie asymétrique (paire de clés)
- Connaître RSA, ECC, Diffie-Hellman
- Comprendre les signatures numériques

Contenu Théorique (1h30)

1. Principes de la cryptographie asymétrique (45min)

La cryptographie asymétrique utilise une paire de clés : une clé publique (pour chiffrer ou vérifier) et une clé privée (pour déchiffrer ou signer).

CONCEPT CLÉ

La clé publique peut être diffusée librement, tandis que la clé privée doit rester secrète.

BON À SAVOIR

RSA 2048 bits est le minimum recommandé aujourd'hui ; 4096 bits offre une marge de sécurité supérieure pour les clés à longue durée de vie.

2. Algorithmes (45min)

Algorithme	Usage	Taille de clé recommandée
RSA	Chiffrement, signatures	2048 ou 4096 bits
ECC	Chiffrement, signatures	256 ou 384 bits
Diffie-Hellman	Échange de clés	2048 bits minimum

Fig 7 — Flux de chiffrement asymétrique

Diagramme		
B	texte chiffré	
C	clé privée	
D	texte clair	E

Figure — Diagramme Mermaid

Lab 23 — Clés asymétriques (45min)

Environnement

Élément	Valeur
Outil	OpenSSL 3.x
Algorithme	RSA 2048 bits

Exercices

Exercice 23.1 — Générer une paire de clés RSA (15min)

```
openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:2048 -out cle_privee.pem
# openssl genpkey : génère une clé privée
# -algorithm RSA : algorithme RSA
# -pkeyopt rsa_keygen_bits:2048 : taille de clé 2048 bits
# -out : fichier de sortie

openssl rsa -in cle_privee.pem -pubout -out cle_publique.pem
# openssl rsa : commande de gestion des clés RSA
# -in : fichier de clé privée en entrée
# -pubout : extrait la clé publique
# -out : fichier de sortie de la clé publique
```

- Les fichiers cle_privee.pem et cle_publique.pem sont créés

Exercice 23.2 — Chiffrer avec la clé publique (15min)

```
echo "Message secret" > message.txt
# echo : écrit la chaîne sur la sortie standard
# > message.txt : redirige la sortie vers le fichier message.txt
openssl pkeyutl -encrypt -inkey cle_publique.pem -pubin -in message.txt -out message.enc
# openssl pkeyutl : utilitaire d'opérations sur clés publiques
# -encrypt : mode chiffrement
# -inkey : fichier de clé
# -pubin : la clé est une clé publique
# -in : fichier d'entrée
# -out : fichier de sortie chiffré
```

- Le fichier message.enc est créé

Démonstration commentée

La clé publique chiffre le message, mais seule la clé privée correspondante peut le déchiffrer. Cela permet une communication sécurisée sans échange préalable de clé secrète.

Correction et résolution pas à pas

Solution

Solution Exercice 23.1

```
openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:2048 -out cle_privee.pem
openssl rsa -in cle_privee.pem -pubout -out cle_publique.pem
```

Bonnes pratiques

Pratique	Description
Taille de clé	RSA 2048 bits minimum, 4096 bits recommandé
Protection clé privée	`chmod 600` sur le fichier de clé privée

BONNES PRATIQUES

La clé privée doit être protégée avec des droits restrictifs et stockée de manière sécurisée.

Exercices — Chapitre 23

Exercice 23.3 — Déchiffrer avec la clé privée (10min)

```
openssl pkeyutl -decrypt -inkey cle_privee.pem -in message.enc -out message_dechiffre.txt
# -decrypt : mode déchiffrement
```

- Le fichier message_dechiffre.txt contient le message original

Solution

Solution Exercice 23.3

```
openssl pkeyutl -decrypt -inkey cle_privee.pem -in message.enc -out message_dechiffre.txt
```

Ce qu'on a retenu

> **Ce qu'on a retenu**

> - La cryptographie asymétrique utilise une paire de clés (publique/privée)

- RSA 2048 bits est le minimum recommandé

- La clé publique chiffre, la clé privée déchiffre

Chapitre 24 — Cryptographie asymétrique — PKI, certificats, signatures numériques (22h00-23h15)

Objectifs

- Comprendre l'infrastructure à clés publiques (PKI)
- Créer une autorité de certification (CA)
- Générer et signer des certificats X.509
- Comprendre les signatures numériques

Contenu Théorique (1h15)

1. PKI et certificats (45min)

Une PKI (Public Key Infrastructure) permet de lier une clé publique à une identité via un certificat signé par une Autorité de Certification (CA).

CONCEPT CLÉ

La chaîne de confiance repose sur une CA racine de confiance qui signe les certificats intermédiaires et utilisateurs.

BON À SAVOIR

Une CA auto-signée est utile pour les tests et les environnements internes, mais les navigateurs et systèmes externes ne lui feront pas confiance sans installation préalable de son certificat racine.

2. Signatures numériques (30min)

Une signature numérique prouve l'intégrité, l'authenticité et la non-répudiation d'un document.

Fig 8 — Chaîne de confiance PKI

Diagramme		
B	signe	
C	utilise	D

Figure — Diagramme Mermaid

Lab 24 — PKI et certificats (45min)

Environnement

Élément	Valeur
Outil	OpenSSL 3.x
Type de CA	Auto-signée

Exercices

Exercice 24.1 — Créer une CA auto-signée (15min)

```
openssl req -x509 -newkey rsa:2048 -keyout cle_ca.pem -out certificat_ca.pem -days 365 -nodes -sub "/CN=M
# openssl req : gestion des demandes de certificat
# -x509 : génère un certificat auto-signé
# -newkey rsa:2048 : génère une nouvelle clé RSA 2048 bits
# -keyout : fichier de sortie de la clé privée
# -out : fichier de sortie du certificat
# -days 365 : durée de validité en jours
# -nodes : ne pas chiffrer la clé privée (No DES)
# -subj : sujet du certificat (CN = Common Name)
```

- Le certificat CA et sa clé privée sont créés

Exercice 24.2 — Générer un CSR et le signer (15min)

```
openssl req -newkey rsa:2048 -keyout cle_utilisateur.pem -out requete.csr -nodes -subj "/CN=Utilis
# openssl req : gestion des demandes de certificat
# -newkey rsa:2048 : génère une nouvelle clé RSA 2048 bits
# -keyout : fichier de sortie de la clé privée
# -out : fichier de sortie de la demande de certificat (CSR)
# -nodes : ne pas chiffrer la clé privée (No DES)
# -subj : sujet du certificat (CN = Common Name)
openssl x509 -req -in requete.csr -CA certificat_ca.pem -CAkey cle_ca.pem -CAcreateserial -out cer
# openssl x509 -req : signe une demande de certificat (CSR)
# -CA : certificat de l'autorité de certification
# -CAkey : clé privée de la CA
# -CAcreateserial : crée un fichier de numéro de série
# -out : fichier de sortie du certificat signé
```

- Le certificat utilisateur est signé par la CA

Démonstration commentée

Le certificat signé par la CA permet à quiconque possédant le certificat de la CA de vérifier l'authenticité de la clé publique de l'utilisateur.

Correction et résolution pas à pas

Solution

Solution Exercice 24.1

```
openssl req -x509 -newkey rsa:2048 -keyout cle_ca.pem -out certificat_ca.pem -days 365 -nodes -subj "/CN
```

Bonnes pratiques

Pratique	Description
Durée de validité	Limiter la durée de vie des certificats (1 an maximum recommandé)
Protection clé CA	Protéger la clé privée de la CA avec des droits restrictifs

BONNES PRATIQUES

La clé privée de la CA est l'élément le plus critique : sa compromission invalide toute la chaîne de confiance.

Exercices — Chapitre 24

Exercice 24.3 — Vérifier un certificat (10min)

```
openssl verify -CAfile certificat_ca.pem certificat_utilisateur.pem
# openssl verify : vérifie la signature d'un certificat
# -CAfile : fichier contenant le certificat de la CA
```

- La vérification retourne "OK"

Solution

Solution Exercice 24.3

```
openssl verify -CAfile certificat_ca.pem certificat_utilisateur.pem
```

Ce qu'on a retenu

> **Ce qu'on a retenu**

> - Une PKI repose sur une chaîne de confiance (CA racine → CA intermédiaire → utilisateur)

> - Les certificats X.509 lient une identité à une clé publique

> - `openssl verify` permet de vérifier la chaîne de signature

Chapitre 25 — MinIO S3 — Sauvegarde et restauration vers un bucket S3 (23h30-00h15)

Objectifs

- Vérifier la présence de Docker et l'installer si nécessaire
- Déployer MinIO via Docker (conteneur serveur S3 compatible)
- Configurer le client mc (MinIO Client)
- Créer un bucket, sauvegarder un fichier, restaurer un fichier
- Vérifier l'intégrité (sha256sum avant/après)

Contenu Théorique (1h30)

1. Vérification de Docker (15min)

Docker est une plateforme de conteneurisation : elle permet d'empaqueter une application avec toutes ses dépendances dans une image, puis d'exécuter cette image dans un conteneur isolé du système hôte. Avant de déployer MinIO, il faut s'assurer que le moteur Docker est présent sur le poste.

```
docker --version
# docker : client en ligne de commande du moteur Docker
# --version : affiche le numéro de version de Docker installé
```

Si Docker est installé, la commande affiche un numéro de version (par exemple `Docker version 27.x`). Dans le cas contraire, le shell signale que la commande est introuvable et il faut procéder à l'installation décrite à la section suivante.

> **POURQUOI**
>
> Déployer MinIO dans un conteneur isole le serveur S3 du reste du système : ses dépendances, ses ports réseau et ses données sont encapsulés. Le déploiement devient reproductible d'un poste à l'autre, et le nettoyage après le laboratoire se limite à arrêter et supprimer le conteneur.

> **CONCEPT CLÉ**
>
> Une image Docker est un modèle immuable contenant le système de fichiers et la configuration d'une application. Un conteneur est l'instance d'exécution isolée de cette image.

2. Installation de Docker (si absent) (20min)

Sur Debian/Ubuntu, l'installation s'appuie sur le dépôt officiel de Docker.

Mise à jour des listes de paquets :

```
sudo apt-get update
# sudo : exécute la commande avec les privilèges du superutilisateur (root)
# apt-get : gestionnaire de paquets des distributions Debian et Ubuntu
# update : recharge la liste des paquets disponibles depuis les dépôts configurés
```

Installation des prérequis :

```
sudo apt-get install -y ca-certificates curl gnupg
# -y : répond automatiquement oui aux questions de confirmation durant l'installation
# ca-certificates : fournit les certificats racines nécessaires aux connexions HTTPS
# curl : outil de transfert de données depuis une URL
# gnupg : suite d'outils de chiffrement et de gestion de clés GPG
```

Ajout de la clé GPG officielle de Docker :

```
sudo install -m 0755 -d /etc/apt/keyrings
# install : copie des fichiers et définit leurs droits d'accès
# -m 0755 : droits lecture/écriture/exécution pour le propriétaire, lecture/exécution pour le groupe et le
# -d : crée un répertoire
# /etc/apt/keyrings : répertoire stockant les clés GPG des dépôts APT

curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /etc/apt/keyrings/docker.g
# curl -fsSL : transfert silencieux, suivi des redirections, détection d'erreurs HTTP, sortie brute
# https://download.docker.com/linux/ubuntu/gpg : adresse de la clé GPG officielle de Docker
# | : redirige la sortie de curl vers l'entrée de la commande gpg
# sudo gpg --dearmor -o : convertit la clé GPG au format binaire utilisable par APT
# /etc/apt/keyrings/docker.gpg : fichier de sortie contenant la clé de Docker
```

Déclaration du dépôt Docker :

```
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] https://downl
# echo : affiche la ligne de configuration du dépôt
# deb : déclare un dépôt de paquets binaires
# arch=$(dpkg --print-architecture) : restreint le dépôt à l'architecture matérielle du poste
# signed-by=/etc/apt/keyrings/docker.gpg : clé GPG utilisée pour vérifier la signature des paquets
# https://download.docker.com/linux/ubuntu : adresse du dépôt Docker
# $(. /etc/os-release && echo "$VERSION_CODENAME") : détecte le nom de code de la version d'Ubuntu ou Debi
# stable : branche stable du dépôt Docker
# | sudo tee : écrit le contenu dans le fichier cible avec les privilèges root
# /etc/apt/sources.list.d/docker.list : fichier de déclaration du dépôt Docker
# > /dev/null : masque l'affichage de la commande tee sur la sortie standard
```

Installation du moteur Docker :

```
sudo apt-get update
# recharge les listes de paquets en prenant en compte le nouveau dépôt Docker

sudo apt-get install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
# docker-ce : moteur Docker Community Edition
# docker-ce-cli : interface en ligne de commande de Docker
# containerd.io : démon de gestion des conteneurs de bas niveau
# docker-buildx-plugin : extension de construction d'images Docker
# docker-compose-plugin : orchestration d'applications multi-conteneurs
```

Démarrage du service et ajout de l'utilisateur au groupe docker :

```
sudo systemctl start docker
# systemctl : outil de gestion des services du gestionnaire systemd
# start : démarre le service immédiatement
# docker : nom du service Docker

sudo systemctl enable docker
# enable : active le démarrage automatique du service Docker à chaque boot du système

sudo usermod -aG docker $USER
# usermod : modifie un compte utilisateur existant
# -aG docker : ajoute l'utilisateur au groupe docker sans le retirer des autres groupes
# $USER : nom de l'utilisateur courant
```

Après `usermod`, l'utilisateur doit se déconnecter puis se reconnecter pour que l'appartenance au groupe docker soit prise en compte.

CONTRE-MESURE

L'appartenance au groupe docker confère des privilèges équivalents à root, car le groupe peut piloter le moteur Docker. En production, restreindre ce groupe aux seuls administrateurs.

BON À SAVOIR

La commande `systemctl enable docker` garantit que Docker démarre automatiquement avec le système, évitant de devoir lancer le service manuellement après chaque redémarrage.

3. Déploiement de MinIO via Docker (20min)

MinIO est un serveur de stockage objet compatible avec l'API Amazon S3. L'image officielle `quay.io/minio/minio` contient le serveur MinIO prêt à être exécuté.

Création du répertoire de données persistantes :

```
mkdir -p /tmp/lab_minio/data
# mkdir -p : crée le répertoire et ses parents s'ils n'existent pas encore
# /tmp/lab_minio/data : répertoire de l'hôte qui recevra les données du conteneur (persistance)
```

Lancement du conteneur MinIO :

```
# Nettoyage d'un éventuel déploiement précédent (évite l'erreur --name minio déjà utilisé)
docker stop minio 2>/dev/null || true
# docker stop : arrête le conteneur nommé minio s'il est en cours d'exécution
# minio : nom du conteneur à arrêter
# 2>/dev/null : masque le message d'erreur si le conteneur n'existe pas
# || true : retourne un code de succès même si l'arrêt échoue (conteneur absent)

docker rm minio 2>/dev/null || true
# docker rm : supprime le conteneur nommé minio s'il existe
# minio : nom du conteneur à supprimer
# 2>/dev/null : masque le message d'erreur si le conteneur n'existe pas
# || true : retourne un code de succès même si la suppression échoue (conteneur absent)
```

```
docker run -d \
  --name minio \
  -p 9000:9000 \
  -p 9001:9001 \
  -e MINIO_ROOT_USER=admin \
  -e MINIO_ROOT_PASSWORD=MinioPass123 \
  -v /tmp/lab_minio/data:/data \
  quay.io/minio/minio server /data --console-address ":9001"
# docker run : crée et démarre un conteneur à partir d'une image
# -d : exécute le conteneur en arrière-plan (mode détaché, le terminal reste disponible)
# --name minio : assigne le nom "minio" au conteneur pour le référencer facilement par la suite
# -p 9000:9000 : publie le port 9000 du conteneur sur le port 9000 de l'hôte (API S3)
# -p 9001:9001 : publie le port 9001 du conteneur sur le port 9001 de l'hôte (console d'administration web
# -e MINIO_ROOT_USER=admin : définit la variable d'environnement du login administrateur de MinIO
# -e MINIO_ROOT_PASSWORD=MinioPass123 : définit le mot de passe administrateur de MinIO
# -v /tmp/lab_minio/data:/data : monte un volume persistant (répertoire de l'hôte vers le répertoire du co
# quay.io/minio/minio : nom de l'image officielle de MinIO sur le registre de conteneurs Quay
# server : sous-commande lançant le mode serveur de MinIO
# /data : répertoire de stockage des objets à l'intérieur du conteneur
# --console-address ":9001" : adresse et port de la console d'administration web de MinIO
```

Vérification du conteneur :

```
docker ps
# docker ps : liste les conteneurs Docker en cours d'exécution
```

CONCEPT CLÉ

Un volume Docker (`-v`) lie un répertoire de l'hôte à un répertoire du conteneur. Les données écrites dans `/data` à l'intérieur du conteneur sont ainsi conservées sur le disque de l'hôte, même après l'arrêt ou la suppression du conteneur.

PIÈGE

Les identifiants `admin` et `MinioPass123` sont des valeurs de démonstration. En production, ils doivent être remplacés par des identifiants forts et uniques, et le serveur doit être exposé via HTTPS avec un reverse proxy.

4. Configuration du client mc (15min)

Le client mc (MinIO Client) est l'outil en ligne de commande officiel pour interagir avec un serveur compatible S3.

Téléchargement et installation :

```
wget https://dl.min.io/client/mc/release/linux-amd64/mc
# wget : télécharge un fichier depuis une URL
# https://dl.min.io/client/mc/release/linux-amd64/mc : adresse du binaire mc pour Linux 64 bits

chmod +x mc
# chmod +x : ajoute le droit d'exécution au fichier mc
# mc : nom du binaire client MinIO
```

Configuration d'un alias de connexion :

```
./mc alias set myminio http://localhost:9000 admin MinioPass123
# ./mc : exécute le client MinIO depuis le répertoire courant
# alias set : configure un alias de connexion nommé
# myminio : nom court de l'alias, référence à ce serveur MinIO
# http://localhost:9000 : URL de l'API S3 du serveur MinIO local
# admin : identifiant d'accès (MINIO_ROOT_USER)
# MinioPass123 : mot de passe d'accès (MINIO_ROOT_PASSWORD)
```

L'alias est enregistré dans le fichier de configuration `~/.mc/config.json` et évite de ressaisir l'URL et les identifiants à chaque commande.

5. Opérations bucket et fichiers (20min)

Création d'un bucket :

```
./mc mb myminio/sauvegardes
# ./mc : client MinIO
# mb : make bucket, crée un nouveau bucket
# myminio/sauvegardes : alias du serveur suivi du nom du bucket à créer
```

Sauvegarde d'un fichier vers le bucket :

```
echo "Données de sauvegarde" > document.txt
# echo : écrit la chaîne sur la sortie standard
# > document.txt : redirige la sortie vers le fichier (création ou écrasement)

sha256sum document.txt
# sha256sum : calcule l'empreinte cryptographique SHA-256 d'un fichier
# document.txt : fichier dont on calcule l'empreinte avant envoi

./mc cp document.txt myminio/sauvegardes/
# ./mc : client MinIO
# cp : copie un fichier d'une source vers une destination
# document.txt : fichier source local
# myminio/sauvegardes/ : destination (bucket sur l'alias myminio)
```

Restauration d'un fichier depuis le bucket :

```
./mc cp myminio/sauvegardes/document.txt document_restaure.txt
# myminio/sauvegardes/document.txt : fichier source dans le bucket
# document_restaure.txt : fichier de destination local (restauré)
```

Vérification d'intégrité :

```
sha256sum document.txt document_restaure.txt
# calcule l'empreinte SHA-256 des deux fichiers pour les comparer
```

Les deux empreintes doivent être identiques pour garantir que le fichier restauré est une copie exacte de l'original.

Synchronisation d'un répertoire :

```
./mc mirror /tmp/lab_minio/dossier_source myminio/sauvegardes/dossier_source
# mc mirror : synchronise un répertoire local avec un bucket (copie incrémentielle des fichiers nouveaux ou
```

Suppression d'un objet :

```
./mc rm myminio/sauvegardes/document.txt
# mc rm : supprime un objet du bucket
```

BON À SAVOIR

La commande `mc mirror` ne copie que les fichiers nouveaux ou modifiés, ce qui optimise les sauvegardes incrémentielles et réduit le temps de transfert.

CONTRE-MESURE

Toujours vérifier l'intégrité des fichiers restaurés avec `sha256sum` avant de les considérer comme valides, notamment après un transfert réseau.

Fig 10 — Déploiement du conteneur MinIO sur l'hôte Docker

Diagramme		
B	**télécharge**	
C	**lance**	
D	**expose**	
D	**expose**	
D	**stocke dans**	G

Figure — Diagramme Mermaid

Fig 11 — Flux de sauvegarde et restauration avec MinIO via Docker

Diagramme		
B	**stockage**	
C	**téléchargement mc cp**	
Fichier local	**sha256sum**	
C	**sha256sum**	
E	**comparaison**	F

Figure — Diagramme Mermaid

Lab 25 — Sauvegarde et restauration avec MinIO S3 via Docker (45min)

Environnement

Élément	Valeur
Poste	Linux
Outils	Docker, MinIO server (conteneur), MinIO client (mc), sha256sum
Répertoire de travail	/tmp/lab_minio
Port API S3	9000
Port console	9001

Exercices (les exercices s'enchaînent pour former un lab complet)

Exercice 25.1 — Vérifier et installer Docker si nécessaire (10min)

Objectif : s'assurer que Docker est disponible sur le poste ; dans le cas contraire, l'installer.

```
docker --version
# docker : client en ligne de commande de Docker
# --version : affiche la version de Docker installée
```

- La commande `docker --version` retourne un numéro de version
- Sinon, Docker est installé via le dépôt officiel (apt)
- Le service Docker est démarré (`systemctl start docker`)
- L'utilisateur courant est ajouté au groupe docker

Solution

Solution Exercice 25.1

Étape 1 — Vérifier la présence de Docker :

```
docker --version
# docker : client Docker
# --version : affiche la version installée
```

Justification : si Docker est présent, le numéro de version s'affiche ; sinon, le shell indique que la commande est introuvable.

Étape 2 — Installer Docker si absent (Debian/Ubuntu) :

```
sudo apt-get update
sudo apt-get install -y ca-certificates curl gnupg
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /etc/apt/keyrings/docker
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] https://download.do
sudo apt-get update
sudo apt-get install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
```

Justification : ces commandes ajoutent le dépôt officiel de Docker, puis installent le moteur Docker et ses extensions.

Étape 3 — Démarrer et activer le service :

```
sudo systemctl start docker
sudo systemctl enable docker
```

Justification : `start` lance le moteur immédiatement ; `enable` assure son démarrage automatique au prochain boot.

Étape 4 — Ajouter l'utilisateur au groupe docker :

```
sudo usermod -aG docker $USER
```

Justification : permet d'exécuter `docker` sans préfixer `sudo` à chaque commande. Une déconnexion puis une reconnexion de session sont nécessaires pour prendre en compte le nouveau groupe.

Exercice 25.2 — Déployer MinIO via Docker (10min)

```
mkdir -p /tmp/lab_minio/data
# mkdir -p : crée le répertoire de données et ses parents
# /tmp/lab_minio/data : volume persistant pour les données MinIO

# Nettoyage d'un éventuel déploiement précédent (évite l'erreur --name minio déjà utilisé)
docker stop minio 2>/dev/null || true
# docker stop : arrête le conteneur nommé minio s'il est en cours d'exécution
# minio : nom du conteneur à arrêter
# 2>/dev/null : masque le message d'erreur si le conteneur n'existe pas
# || true : retourne un code de succès même si l'arrêt échoue (conteneur absent)

docker rm minio 2>/dev/null || true
# docker rm : supprime le conteneur nommé minio s'il existe
# minio : nom du conteneur à supprimer
# 2>/dev/null : masque le message d'erreur si le conteneur n'existe pas
# || true : retourne un code de succès même si la suppression échoue (conteneur absent)

docker run -d \
  --name minio \
  -p 9000:9000 \
  -p 9001:9001 \
  -e MINIO_ROOT_USER=admin \
  -e MINIO_ROOT_PASSWORD=MinioPass123 \
  -v /tmp/lab_minio/data:/data \
  quay.io/minio/minio server /data --console-address ":9001"
# docker run : crée et démarre un conteneur à partir d'une image
# -d : exécute le conteneur en arrière-plan (mode détaché)
# --name minio : assigne le nom "minio" au conteneur pour le référencer facilement
# -p 9000:9000 : publie le port 9000 du conteneur sur le port 9000 de l'hôte (API S3)
# -p 9001:9001 : publie le port 9001 du conteneur sur le port 9001 de l'hôte (console web)
# -e MINIO_ROOT_USER=admin : définit le login administrateur de MinIO via variable d'environnement
# -e MINIO_ROOT_PASSWORD=MinioPass123 : définit le mot de passe administrateur de MinIO
# -v /tmp/lab_minio/data:/data : monte un volume persistant (hôte vers conteneur)
# quay.io/minio/minio : image officielle de MinIO sur le registre Quay
# server /data : lance le mode serveur de MinIO avec le répertoire de stockage /data
# --console-address ":9001" : adresse et port de la console d'administration web

docker ps
# docker ps : liste les conteneurs en cours d'exécution
```

- Le répertoire /tmp/lab_minio/data est créé
- Le conteneur minio est créé et démarré
- `docker ps` affiche le conteneur minio en cours d'exécution
- Les ports 9000 et 9001 sont publiés

Solution

Solution Exercice 25.2

Étape 1 — Créer le répertoire de données persistantes :

```
mkdir -p /tmp/lab_minio/data
# mkdir -p : crée le répertoire et ses parents si nécessaire
# /tmp/lab_minio/data : répertoire de l'hôte recevant les données du conteneur
```

Justification : isoler les données dans un répertoire dédié permet de les conserver après l'arrêt du conteneur.

Étape 2 — Lancer le conteneur MinIO :

```
# Nettoyage d'un éventuel déploiement précédent (évite l'erreur --name minio déjà utilisé)
docker stop minio 2>/dev/null || true
# docker stop : arrête le conteneur nommé minio s'il est en cours d'exécution
# minio : nom du conteneur à arrêter
# 2>/dev/null : masque le message d'erreur si le conteneur n'existe pas
# || true : retourne un code de succès même si l'arrêt échoue (conteneur absent)

docker rm minio 2>/dev/null || true
# docker rm : supprime le conteneur nommé minio s'il existe
# minio : nom du conteneur à supprimer
# 2>/dev/null : masque le message d'erreur si le conteneur n'existe pas
# || true : retourne un code de succès même si la suppression échoue (conteneur absent)
```

```
docker run -d \
  --name minio \
  -p 9000:9000 \
  -p 9001:9001 \
  -e MINIO_ROOT_USER=admin \
  -e MINIO_ROOT_PASSWORD=MinioPass123 \
  -v /tmp/lab_minio/data:/data \
  quay.io/minio/minio server /data --console-address ":9001"
```

Justification : `-d` détache le conteneur, `-p` publie les ports API et console, `-e` définit les identifiants root, `-v` monte le volume persistant, et `server /data` lance MinIO avec le répertoire de stockage.

Étape 3 — Vérifier le conteneur :

```
docker ps
# docker ps : liste les conteneurs en cours d'exécution
```

Justification : confirmer que le conteneur minio est bien actif avant de configurer le client mc.

Exercice 25.3 — Configurer mc et créer un bucket (10min)

```
wget https://dl.min.io/client/mc/release/linux-amd64/mc
# wget : télécharge le binaire client mc
# https://dl.min.io/client/mc/release/linux-amd64/mc : adresse du binaire mc pour Linux 64 bits

chmod +x mc
# chmod +x : rend le binaire mc exécutable
# mc : nom du binaire client MinIO

./mc alias set myminio http://localhost:9000 admin MinioPass123
# ./mc : exécute le client MinIO depuis le répertoire courant
# alias set : configure un alias de connexion nommé
# myminio : nom court de l'alias (référence à ce serveur)
# http://localhost:9000 : URL de l'API S3 du serveur MinIO local
# admin : identifiant d'accès (MINIO_ROOT_USER)
# MinioPass123 : mot de passe d'accès (MINIO_ROOT_PASSWORD)

./mc mb myminio/sauvegardes
# ./mc : client MinIO
# mb : make bucket, crée un nouveau bucket
# myminio/sauvegardes : alias du serveur suivi du nom du bucket

./mc ls myminio
# ./mc : client MinIO
# ls : liste les buckets de l'alias myminio
```

- Le binaire mc est téléchargé et rendu exécutable
- L'alias myminio est configuré vers le serveur local
- Le bucket sauvegardes est créé
- `./mc ls myminio` affiche le bucket

Solution

Solution Exercice 25.3

Étape 1 — Télécharger et rendre mc exécutable :

```
wget https://dl.min.io/client/mc/release/linux-amd64/mc
chmod +x mc
```

Justification : mc est l'outil officiel en ligne de commande pour interagir avec un serveur S3 compatible.

Étape 2 — Configurer l'alias de connexion :

```
./mc alias set myminio http://localhost:9000 admin MinioPass123
```

Justification : l'alias évite de ressaisir l'URL et les identifiants à chaque commande ; il est enregistré dans `~/.mc/config.json`.

Étape 3 — Créer le bucket et le vérifier :

```
./mc mb myminio/sauvegardes
./mc ls myminio
```

Justification : `mb` crée le conteneur de stockage objet ; `ls` confirme sa présence.

Exercice 25.4 — Sauvegarder et restaurer un fichier (10min)

```
echo "Données de sauvegarde" > document.txt
# echo : écrit la chaîne sur la sortie standard
# > document.txt : redirige la sortie vers le fichier (création ou écrasement)

sha256sum document.txt
# sha256sum : calcule l'empreinte cryptographique SHA-256 d'un fichier
# document.txt : fichier dont on calcule l'empreinte avant envoi

./mc cp document.txt myminio/sauvegardes/
# ./mc : client MinIO
# cp : copie un fichier d'une source vers une destination
# document.txt : fichier source local
# myminio/sauvegardes/ : destination (bucket sur l'alias myminio)

./mc cp myminio/sauvegardes/document.txt document_restaure.txt
# ./mc : client MinIO
# cp : copie un fichier d'une source vers une destination
# myminio/sauvegardes/document.txt : fichier source dans le bucket
# document_restaure.txt : fichier de destination local (restauré)
```

- Le fichier document.txt est créé localement
- L'empreinte SHA-256 est calculée avant envoi
- Le fichier est copié dans le bucket sauvegardes
- Le fichier est restauré depuis le bucket vers le poste local

Solution

Solution Exercice 25.4

Étape 1 — Créer le fichier source et calculer son empreinte de référence :

```
echo "Données de sauvegarde" > document.txt
sha256sum document.txt
```

Justification : l'empreinte SHA-256 avant envoi sert de référence pour la vérification d'intégrité après restauration.

Étape 2 — Sauvegarder le fichier dans le bucket :

```
./mc cp document.txt myminio/sauvegardes/
```

Justification : `cp` téléverse le fichier local vers le bucket MinIO.

Étape 3 — Restaurer le fichier depuis le bucket :

```
./mc cp myminio/sauvegardes/document.txt document_restaure.txt
```

Justification : `cp` télécharge le fichier depuis le bucket vers le poste local, dans le sens inverse de la sauvegarde.

Exercice 25.5 — Vérifier l'intégrité avec sha256sum (5min)

```
sha256sum document.txt document_restaure.txt
# sha256sum : calcule l'empreinte SHA-256 des deux fichiers
# document.txt : fichier original (avant sauvegarde)
# document_restaure.txt : fichier restauré (après téléchargement)
```

- Les deux empreintes SHA-256 sont affichées
- Les empreintes sont identiques
- L'intégrité du fichier restauré est confirmée

Solution

Solution Exercice 25.5

```
sha256sum document.txt document_restaure.txt
# sha256sum : calcule l'empreinte SHA-256 des deux fichiers pour comparaison
# document.txt : fichier original
# document_restaure.txt : fichier restauré
```

Justification : la comparaison des deux empreintes confirme que le transfert réseau n'a pas altéré le contenu du fichier. Si les empreintes diffèrent, le fichier restauré est corrompu et doit être retéléchargé.

Démonstration commentée

Le résultat attendu de la séquence complète est le suivant :

```
$ docker --version
Docker version 27.x, build xxxx
$ docker run -d --name minio -p 9000:9000 -p 9001:9001 \
  -e MINIO_ROOT_USER=admin -e MINIO_ROOT_PASSWORD=MinioPass123 \
  -v /tmp/lab_minio/data:/data quay.io/minio/minio server /data --console-address ":9001"
a1b2c3d4... (identifiant du conteneur)
$ ./mc alias set myminio http://localhost:9000 admin MinioPass123
Added `myminio` successfully.
$ ./mc mb myminio/sauvegardes
Bucket created successfully `myminio/sauvegardes`.
$ echo "Données de sauvegarde" > document.txt
$ sha256sum document.txt
a3f5c3...  document.txt
$ ./mc cp document.txt myminio/sauvegardes/
document.txt: 1.00 KiB / 1.00 KiB
$ ./mc cp myminio/sauvegardes/document.txt document_restaure.txt
document_restaure.txt: 1.00 KiB / 1.00 KiB
$ sha256sum document.txt document_restaure.txt
a3f5c3...  document.txt
a3f5c3...  document_restaure.txt
```

Ligne par ligne :

- `docker --version` confirme que le moteur Docker est disponible.
- `docker run ...` crée et démarre le conteneur MinIO ; l'identifiant du conteneur s'affiche.
- `./mc alias set ...` configure l'alias myminio ; le message `Added ... successfully` confirme l'opération.
- `./mc mb myminio/sauvegardes` crée le bucket ; le message `Bucket created successfully` confirme sa création.
- `echo ... > document.txt` crée le fichier source avec un contenu de test.
- `sha256sum document.txt` calcule l'empreinte de référence avant tout transfert.
- `./mc cp document.txt myminio/sauvegardes/` téléverse le fichier dans le bucket MinIO. La sortie indique le nombre d'octets transférés.

- `./mc cp myminio/sauvegardes/document.txt document_restaure.txt` télécharge le fichier depuis le bucket vers le poste local.
- `sha256sum document.txt document_restaure.txt` affiche deux empreintes identiques, prouvant que le fichier restauré est une copie exacte de l'original.

Correction et résolution pas à pas

Solution

Correction complète du Lab 25

Étape 1 — Vérifier et installer Docker si nécessaire :

```
docker --version
# si absent : sudo apt-get update && sudo apt-get install -y docker-ce ...
sudo systemctl start docker
sudo systemctl enable docker
sudo usermod -aG docker $USER
```

Justification : Docker est le prérequis pour exécuter MinIO en conteneur. Le service doit être démarré et l'utilisateur ajouté au groupe docker pour piloter Docker sans `sudo`.

Étape 2 — Créer le répertoire de données persistantes :

```
mkdir -p /tmp/lab_minio/data
```

Justification : isoler les données dans un répertoire dédié permet de les conserver après l'arrêt du conteneur.

Étape 3 — Déployer MinIO via Docker :

```
# Nettoyage d'un éventuel déploiement précédent (évite l'erreur --name minio déjà utilisé)
docker stop minio 2>/dev/null || true
# docker stop : arrête le conteneur nommé minio s'il est en cours d'exécution
# minio : nom du conteneur à arrêter
# 2>/dev/null : masque le message d'erreur si le conteneur n'existe pas
# || true : retourne un code de succès même si l'arrêt échoue (conteneur absent)

docker rm minio 2>/dev/null || true
# docker rm : supprime le conteneur nommé minio s'il existe
# minio : nom du conteneur à supprimer
# 2>/dev/null : masque le message d'erreur si le conteneur n'existe pas
# || true : retourne un code de succès même si la suppression échoue (conteneur absent)
```

```
docker run -d --name minio -p 9000:9000 -p 9001:9001 \
  -e MINIO_ROOT_USER=admin -e MINIO_ROOT_PASSWORD=MinioPass123 \
  -v /tmp/lab_minio/data:/data quay.io/minio/minio server /data --console-address ":9001"
```

Justification : chaque flag a un rôle précis (détachement, publication des ports, identifiants, volume persistant, image officielle, répertoire de stockage, console web).

Étape 4 — Vérifier le conteneur :

```
docker ps
```

Justification : confirmer que le conteneur minio est actif avant de configurer le client mc.

Étape 5 — Installer et configurer le client mc :

```
wget https://dl.min.io/client/mc/release/linux-amd64/mc
chmod +x mc
./mc alias set myminio http://localhost:9000 admin MinioPass123
```

Justification : mc est l'outil officiel pour interagir avec un serveur S3 compatible ; l'alias évite de répéter l'URL et les identifiants.

Étape 6 — Créer le bucket :

```
./mc mb myminio/sauvegardes
```

Justification : le bucket est le conteneur de stockage objet qui regroupera les fichiers sauvegardés.

Étape 7 — Sauvegarder un fichier :

```
echo "Données de sauvegarde" > document.txt
sha256sum document.txt
./mc cp document.txt myminio/sauvegardes/
```

Justification : l'empreinte SHA-256 avant envoi sert de référence pour la vérification d'intégrité après restauration.

Étape 8 — Restaurer le fichier :

```
./mc cp myminio/sauvegardes/document.txt document_restaure.txt
```

Justification : `cp` télécharge le fichier depuis le bucket vers le poste local.

Étape 9 — Vérifier l'intégrité :

```
sha256sum document.txt document_restaure.txt
```

Justification : la comparaison des deux empreintes confirme que le transfert n'a pas altéré le contenu du fichier.

Bonnes pratiques

Pratique	Description
Identifiants forts	Remplacer admin / MinioPass123 par des valeurs uniques en production
Volume persistant	Monter un volume Docker (`-v`) pour conserver les données après l'arrêt du conteneur
Vérification d'intégrité	Comparer systématiquement sha256sum avant et après transfert
Alias nommé	Utiliser un alias mc pour éviter de répéter l'URL et les identifiants
Empreinte de référence	Conserver l'empreinte SHA-256 des fichiers critiques hors du bucket

BONNES PRATIQUES

Le stockage objet S3 avec MinIO est adapté aux sauvegardes, mais la vérification d'intégrité (sha256sum) et la protection des identifiants d'accès sont indispensables pour garantir la confidentialité et l'exactitude des données restaurées.

Exercices — Chapitre 25

Exercice 25.6 — Synchroniser un répertoire avec mc mirror (10min)

```
mkdir -p /tmp/lab_minio/dossier_source
echo "Fichier A" > /tmp/lab_minio/dossier_source/fichier_a.txt
echo "Fichier B" > /tmp/lab_minio/dossier_source/fichier_b.txt
./mc mirror /tmp/lab_minio/dossier_source myminio/sauvegardes/dossier_source
# mc mirror : synchronise le répertoire local vers le bucket (copie incrémentielle)
# /tmp/lab_minio/dossier_source : source locale
# myminio/sauvegardes/dossier_source : destination dans le bucket
./mc ls myminio/sauvegardes/dossier_source
# ls : vérifie la présence des fichiers dans le bucket
```

- Le répertoire source contient deux fichiers
- mc mirror copie tous les fichiers vers le bucket
- Le listage du bucket affiche le dossier et ses fichiers

Solution

Solution Exercice 25.6

```
mkdir -p /tmp/lab_minio/dossier_source
# mkdir -p : crée le répertoire source
echo "Fichier A" > /tmp/lab_minio/dossier_source/fichier_a.txt
# echo : crée le fichier fichier_a.txt
echo "Fichier B" > /tmp/lab_minio/dossier_source/fichier_b.txt
# echo : crée le fichier fichier_b.txt
./mc mirror /tmp/lab_minio/dossier_source myminio/sauvegardes/dossier_source
# mirror : synchronise le répertoire local vers le bucket (copie incrémentielle)
./mc ls myminio/sauvegardes/dossier_source
# ls : vérifie la présence des fichiers dans le bucket
```

Exercice 25.7 — Supprimer un objet du bucket (10min)

```
./mc rm myminio/sauvegardes/document.txt
# mc rm : supprime un objet du bucket
# myminio/sauvegardes/document.txt : cible de suppression
./mc ls myminio/sauvegardes/
# ls : vérifie que l'objet a été supprimé du listage
```

- Le fichier document.txt est supprimé du bucket
- Le listage du bucket ne contient plus document.txt

Solution

Solution Exercice 25.7

```
./mc rm myminio/sauvegardes/document.txt
# rm : supprime un objet du bucket
# myminio/sauvegardes/document.txt : cible de suppression
./mc ls myminio/sauvegardes/
# ls : vérifie que l'objet a été supprimé du listage
```

Ce qu'on a retenu

> **Ce qu'on a retenu**

> - Docker permet de déployer MinIO dans un conteneur isolé et reproductible

> - Le client mc configure un alias et manipule les buckets (mb, cp, mirror, rm)

> - La vérification d'intégrité avec sha256sum garantit l'exactitude des fichiers restaurés

> - Les identifiants d'accès doivent être protégés et modifiés par rapport aux valeurs par défaut

---

Synthèse du Module Extra

Couverture du Module Extra

Chapitre	Thème
17	Introduction à Python pour l'administration système
18	Manipulation de fichiers, processus et système avec Python
19	Automatisation et scripting système avec Python
20	Cas pratique Python — script d'administration complet
21	Cryptographie symétrique — principes et algorithmes
22	Cryptographie symétrique — mise en pratique (OpenSSL, Python)
23	Cryptographie asymétrique — principes et algorithmes
24	Cryptographie asymétrique — PKI, certificats, signatures numériques

Mémo — Commandes et Fonctions Essentielles

```
# Python
import os, shutil, subprocess, logging, argparse
open(fichier, 'mode')  # Ouvrir un fichier
with ... as f:         # Gestion automatique de fermeture
try/except/finally     # Gestion d'erreurs
```

```
# OpenSSL
openssl enc -aes-256-cbc -salt -pbkdf2  # Chiffrement symétrique
openssl genpkey -algorithm RSA          # Génération de clé
openssl req -x509                       # Certificat auto-signé
openssl verify                          # Vérification de certificat
```

Ressources supplémentaires

Outils utilisés

Outil	Version	Usage
Python	3.x	Scripting d'administration
OpenSSL	3.x	Cryptographie et PKI

Fin du cours « Scripting Bash et PowerShell ». Bonne formation !